Optimization

Last modified: 2026-11-10 20:00:58 (PST)

Critical points and optimization

Definition 0.1 (Flat point (stationary point))**.** Let f be differentiable at c. If $f'(c) = 0$, then c is a **flat point** of f (also called a **stationary point**) [1].

Example 0.1 (Flat points, and a point that is not one)**.**

- $f(w) = w^3$ has $f'(w) = 3w^2$, which is 0 at $w = 0$, so 0 is a flat point. Yet $f(0) = 0$ is neither the minimum nor the maximum of the values f takes on any open interval around 0: $f(w) < 0$ for $w < 0$, and $f(w) > 0$ for $w > 0$.
- $h(w) = w^2$ has $h'(1) = 2 \neq 0$, so 1 is not a flat point.

Exercise 0.1 (Differentiate a quadratic)**.** A flat point of a function is where its derivative is zero. The next exercises use the parabola $f(w) = w^2 - 4w + 7$ to find where f is flat and to compare a linear approximation with the exact value.
Differentiate f.

Solution

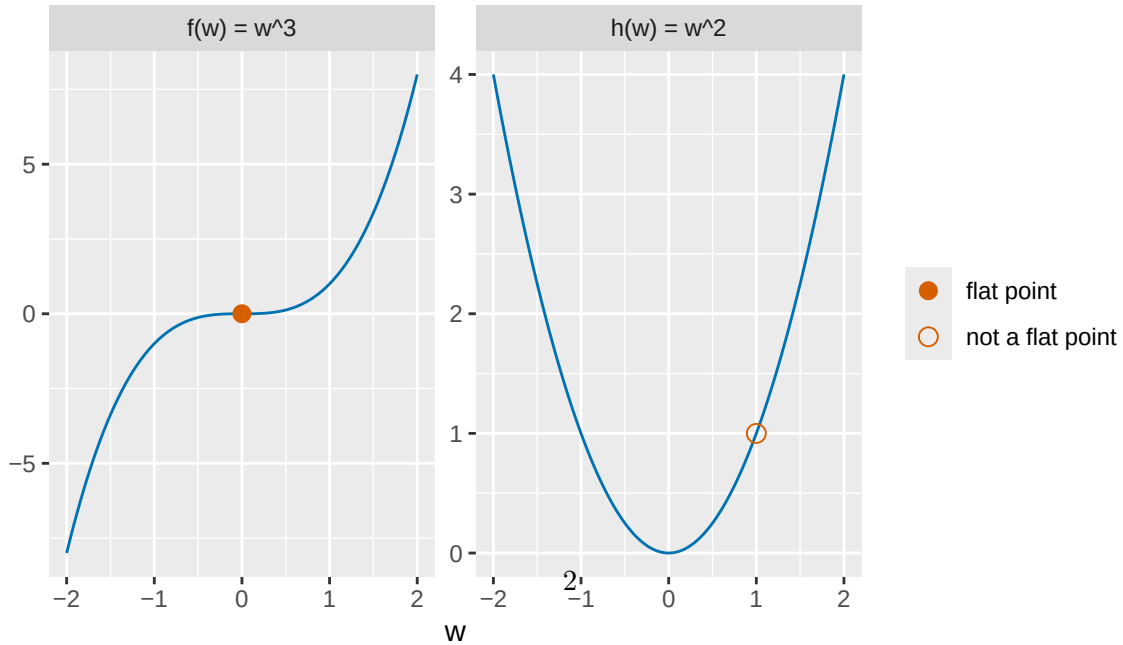
Solution 0.1. Differentiating each term:

$$\frac{df}{dw} = 2w - 4$$

Exercise 0.2 (Find the flat point)**.** In the setting of Exercise 0.1, find the flat point w of f.

R

```
fp_w_ff <- seq(-2, 2, length.out = 201)
fp_cube_ff <- tibble::tibble(
  w = fp_w_ff, value = fp_w_ff^3, panel = "f(w) = w^3"
)
fp_square_ff <- tibble::tibble(
  w = fp_w_ff, value = fp_w_ff^2, panel = "h(w) = w^2"
)
fp_curves_ff <- dplyr::bind_rows(fp_cube_ff, fp_square_ff)
fp_marks_w_ff <- c(0, 1)
fp_marks_panel_ff <- c("f(w) = w^3", "h(w) = w^2")
fp_marks_kind_ff <- c("flat point", "not a flat point")
fp_marks_ff <- tibble::tibble(
  w = fp_marks_w_ff, value = fp_marks_w_ff,
  panel = fp_marks_panel_ff, kind = fp_marks_kind_ff
)
fp_aes_ff <- ggplot2::aes(w, value)
fp_marks_aes_ff <- ggplot2::aes(shape = kind)
fp_shapes_ff <- c("flat point" = 16, "not a flat point" = 1)
ggplot2::ggplot(fp_curves_ff, fp_aes_ff) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(
    data = fp_marks_ff, mapping = fp_marks_aes_ff,
    colour = "#D55E00", size = 3
  ) +
  ggplot2::scale_shape_manual(values = fp_shapes_ff) +
  ggplot2::facet_wrap(~panel, scales = "free_y") +
  ggplot2::labs(x = "w", y = NULL, shape = NULL)
```



Python

```
import matplotlib.pyplot as plt
```

Solution

Solution 0.2. The derivative is zero where $2w - 4 = 0$, which gives $w = 2$.

Exercise 0.3 (Classify the flat point)**.** In the setting of Exercise 0.1, say whether $f(w)$ at the flat point is the minimum or the maximum of the values of f.

Solution

Solution 0.3. The flat point is $w = 2$, from Solution 0.2. Evaluating f there gives

$$\begin{aligned} f(2) &= 4 - 8 + 7 \\ &= 3. \end{aligned}$$

This value is the minimum of the values of f. Completing the square, $f(w) = (w-2)^2+3$, since

$$\begin{aligned} (w-2)^2 + 3 &= w^2 - 4w + 4 + 3 \\ &= w^2 - 4w + 7, \end{aligned}$$

and $(w-2)^2 \geq 0$, so $f(w) \geq 3 = f(2)$ for every w. Equivalently, the derivative is negative below $w = 2$ and positive above it, so the function falls into that point and rises out of it.

Exercise 0.4 (Slope at $w = 1$)**.** In the setting of Exercise 0.1, evaluate the derivative at $w = 1$.

Solution

Solution 0.4. At $w = 1$ the derivative is $2(1) - 4 = -2$, so f is falling there.

Exercise 0.5 (Predict a nearby value)**.** In the setting of Exercise 0.1, use the linear approximation and the derivative at $w = 1$ from Exercise 0.4 to predict $f(1.01)$.

Solution

Solution 0.5. With $\varepsilon = 0.01$, the linear approximation predicts a change of $(0.01)(-2) = -0.02$, from

$$\begin{aligned} f(1) &= 1 - 4 + 7 \\ &= 4 \end{aligned}$$

to 3.98.

Exercise 0.6 (Exact value)**.** In the setting of Exercise 0.1, find the exact value of $f(1.01)$.

Solution

Solution 0.6. The exact value is

$$\begin{aligned} f(1.01) &= (1.01)^2 - 4(1.01) + 7 \\ &= 1.0201 - 4.04 + 7 \\ &= 3.9801 \end{aligned}$$

a change of -0.0199 from $f(1) = 4$.

Exercise 0.7 (Check the prediction)**.** Compare the prediction in Exercise 0.5 with the exact value of $f(1.01)$ from Exercise 0.6.

Solution

Solution 0.7. The prediction 3.98 (Solution 0.5) and the exact value 3.9801 (Solution 0.6) differ by 0.0001, which is ε^2. For this f, the difference between $f(w+\varepsilon)$ and its linear approximation is, at any w, exactly

$$f(w+\varepsilon) - [f(w) + f'(w)\,\varepsilon] = \varepsilon^2,$$

so halving the step quarters that difference.
A step of gradient descent with step length s changes w by $\varepsilon = -sf'(w)$. The linear approximation predicts the change $-s\,f'(w)^2$ in f, and the difference between $f(w+\varepsilon)$ and its linear approximation is $\varepsilon^2 = s^2 f'(w)^2$. The actual change in f is therefore $-s\,f'(w)^2\,(1-s)$. This change is negative exactly when $0 < s < 1$, for $f'(w) \neq 0$, so f decreases only for those steps. For this f, the difference is smaller than the predicted decrease only when $s < 1$: the step must be short enough.

Definition 0.2 (Critical point)**.** Let f be a function defined on an open interval containing c. The point c is a **critical point** of f if either $f'(c) = 0$ or f is not differentiable at c.

Example 0.2 (Critical points that are and are not flat points)**.**

- $g(w) = |w|$ has no derivative at $w = 0$: its one-sided derivatives there are 1 and -1

(see the one-sided derivative example). So the difference quotient $\frac{|h|-0}{h}$ has no limit as $h \to 0$, because the one-sided limits from the left and from the right differ. So 0 is a critical point of g but not a flat point.

- $f(w) = w^3$ has

$$\begin{aligned} f'(0) &= 3 \cdot 0^2 \\ &= 0, \end{aligned}$$

so 0 is a flat point of f, and hence a critical point.

- $h(w) = w^2$ is differentiable everywhere, with $h'(w) = 2w$, which is 0 only at $w = 0$. So 0 is the only critical point of h. For example, $h'(1) = 2 \neq 0$, so 1 is not one.

Theorem 0.1 (Every flat point is a critical point)**.** *If c is a flat point of f (Definition 0.1), then c is a critical point of f (Definition 0.2).*

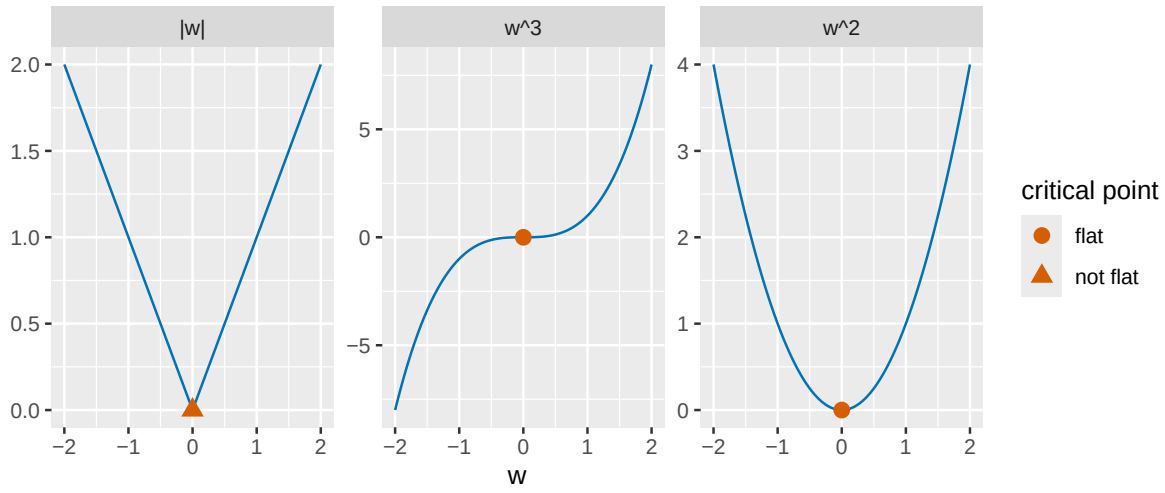
Proof

Proof. A flat point c has f differentiable at c and $f'(c) = 0$. The condition "$f'(c) = 0$" is the first alternative in Definition 0.2, so c is a critical point. □

Example 0.3 (The flat points of a cubic, and a critical point that is not flat)**.** For $f(w) = w^3 - 3w$, the derivative is $f'(w) = 3w^2 - 3$. The code differentiates f with `D()` and evaluates f' at several points. For $g(w) = |w|$, it computes the difference quotients at 0 from both sides.

R

```r
cp_w_fc <- seq(-2, 2, length.out = 201)
cp_abs_value_fc <- abs(cp_w_fc)
cp_abs_fc <- tibble::tibble(w = cp_w_fc, value = cp_abs_value_fc, panel = "|w|")
cp_cube_fc <- tibble::tibble(w = cp_w_fc, value = cp_w_fc^3, panel = "w^3")
cp_square_fc <- tibble::tibble(w = cp_w_fc, value = cp_w_fc^2, panel = "w^2")
cp_curves_fc <- dplyr::bind_rows(cp_abs_fc, cp_cube_fc, cp_square_fc)
cp_levels_fc <- c("|w|", "w^3", "w^2")
cp_curves_fc$panel <- factor(cp_curves_fc$panel, levels = cp_levels_fc)
cp_kinds_fc <- c("not flat", "flat", "flat")
cp_marks_fc <- tibble::tibble(
  w = 0, value = 0, panel = cp_levels_fc, kind = cp_kinds_fc
)
cp_marks_fc$panel <- factor(cp_marks_fc$panel, levels = cp_levels_fc)
cp_aes_fc <- ggplot2::aes(w, value)
cp_marks_aes_fc <- ggplot2::aes(shape = kind)
cp_shapes_fc <- c("flat" = 16, "not flat" = 17)
ggplot2::ggplot(cp_curves_fc, cp_aes_fc) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(
    data = cp_marks_fc, mapping = cp_marks_aes_fc,
    colour = "#D55E00", size = 3
  ) +
  ggplot2::scale_shape_manual(values = cp_shapes_fc) +
  ggplot2::facet_wrap(~panel, scales = "free_y") +
  ggplot2::labs(x = "w", y = NULL, shape = "critical point")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

cp_w_fc = np.linspace(-2, 2, 201)
cp_curves_fc = [("|w|", np.abs(cp_w_fc), "^"),
                ("w^3", cp_w_fc**3, "o"),
                ("w^2", cp_w_fc**2, "o")]
fig, axes = plt.subplots(1, 3, figsize=(7, 3), layout="constrained")
```

```r
f_prime <- D(quote(w^3 - 3 * w), "w")
points <- c(-2, -1, 0, 1, 2)
derivs <- data.frame(w = points, f_prime = eval(f_prime, list(w = points)))
derivs
#>    w f_prime
#> 1 -2       9
#> 2 -1       0
#> 3  0      -3
#> 4  1       0
#> 5  2       9
flat <- derivs$w[derivs$f_prime == 0]

h <- c(-1e-3, 1e-3)
quotients <- abs(h) / h
quotients
#> [1] -1  1
```

The derivative is 0 at 2 of the 5 points, namely $w = -1$ and $w = 1$. Both points are flat points of f, since the derivative is 0 there, so by Theorem 0.1 they are critical points. The difference quotients of g at 0 are -1 from the left and 1 from the right, so g has no derivative at 0. Thus 0 is a critical point of g (Definition 0.2), but not a flat point.

Theorem 0.2 (Second derivative test)**.** *Let c be a flat point of f, where the second derivative $f''(c)$ is defined.*

- *If $f''(c) > 0$, then $f(x) > f(c)$ for every $x \neq c$ in some open interval around c.*
- *If $f''(c) < 0$, then $f(x) < f(c)$ for every $x \neq c$ in some open interval around c.*

Proof

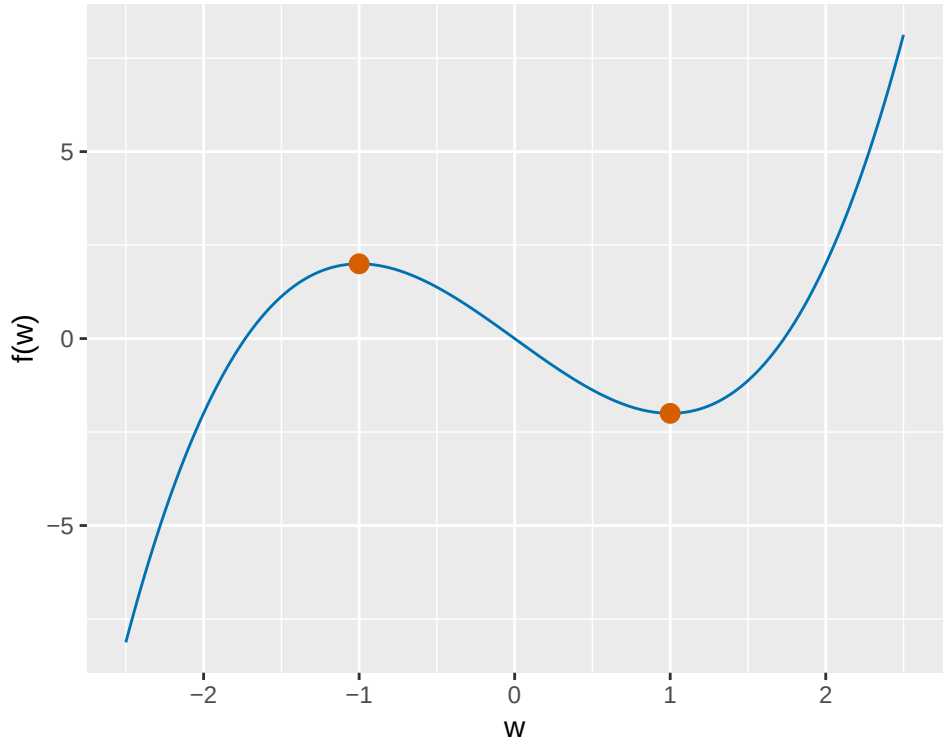
Proof. Suppose $f''(c) > 0$. Because $f'(c) = 0$, the derivative of f' at c is

$$f''(c) = \lim_{x \to c} \frac{f'(x)}{x - c}.$$

A limit that is positive forces the quotient $f'(x)/(x - c)$ to be positive for x near c. So $f'(x)$ has the same sign as $x - c$ on some open interval around c. Hence $f' < 0$ to the left of c and $f' > 0$ to the right of c, so f is decreasing to the left of c and increasing to the right of c. Therefore $f(x) > f(c)$ for every $x \neq c$ in that open interval.
For $f''(c) < 0$, apply the same reasoning to $-f$, since $(-f)''(c) = -f''(c) > 0$. Then $-f(x) > -f(c)$ near c, which says $f(x) < f(c)$ near c. □

R

```r
fpc_w_fk <- seq(-2.5, 2.5, length.out = 201)
fpc_curve_fk <- tibble::tibble(w = fpc_w_fk, value = fpc_w_fk^3 - 3 * fpc_w_fk)
fpc_flat_w_fk <- c(-1, 1)
fpc_flat_value_fk <- fpc_flat_w_fk^3 - 3 * fpc_flat_w_fk
fpc_flat_fk <- tibble::tibble(w = fpc_flat_w_fk, value = fpc_flat_value_fk)
fpc_aes_fk <- ggplot2::aes(w, value)
ggplot2::ggplot(fpc_curve_fk, fpc_aes_fk) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(data = fpc_flat_fk, colour = "#D55E00", size = 3) +
  ggplot2::labs(x = "w", y = "f(w)")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

fpc_w_fk = np.linspace(-2.5, 2.5, 201)
fpc_flat_fk = np.array([-1, 1])
fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
ax.plot(fpc_w_fk, fpc_w_fk**3 - 3 * fpc_w_fk, color="#0072B2")
ax.plot(fpc_flat_fk, fpc_flat_fk**3 - 3 * fpc_flat_fk, "o",
        color="#D55E00", markersize=8)
ax.set_xlabel("w")
ax.set_ylabel("f(w)")
plt.show()
```




Example 0.4 (Second derivative test for powers of x)**.** The function $f(x) = x^2$ has the flat point $c = 0$ and second derivative $f''(0) = 2 > 0$, so 0 is a local minimizer by Theorem 0.2.

Remark (The second derivative test can fail to decide)

Remark 0.1 (The second derivative test can fail to decide). If $f''(c) = 0$ at a flat point c, the second derivative test does not decide: f may be smallest at c among nearby points, largest there, or neither (Example 0.5).

Example 0.5 (Three functions with second derivative 0 at a flat point)**.** Take $f(x) = x^4$, $g(x) = -x^4$ and $h(x) = x^3$. Each has the derivative 0 at $x = 0$:

$$\begin{aligned} f'(x) &= 4x^3, & f'(0) &= 0, \\ g'(x) &= -4x^3, & g'(0) &= 0, \\ h'(x) &= 3x^2, & h'(0) &= 0. \end{aligned}$$

So 0 is a flat point of all three. Each has the second derivative 0 there:

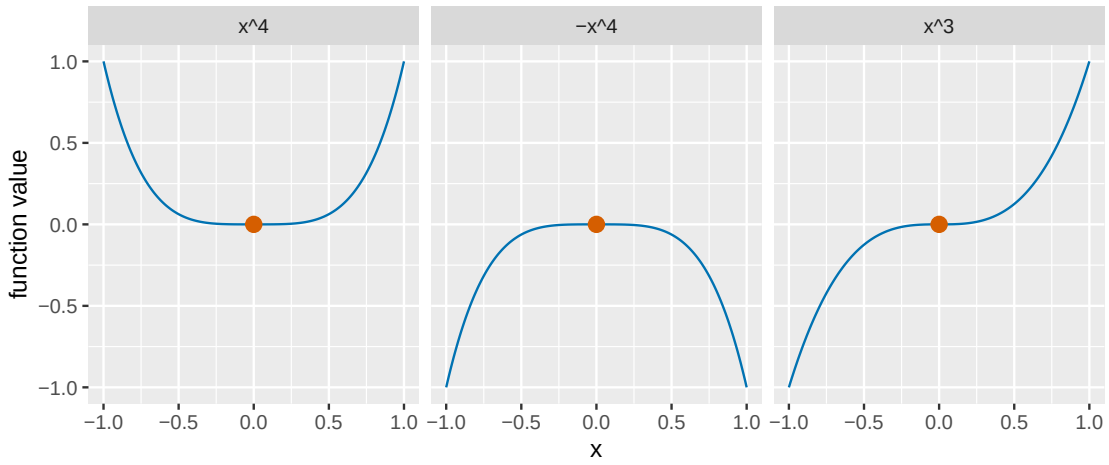
$$\begin{aligned} f''(x) &= 12x^2, & f''(0) &= 0, \\ g''(x) &= -12x^2, & g''(0) &= 0, \\ h''(x) &= 6x, & h''(0) &= 0. \end{aligned}$$

Because the second derivative is 0 in all three cases, Theorem 0.2 does not decide any of them. Yet the three behave differently near 0 (Figure 4):

- $f(x) = x^4 > 0 = f(0)$ for every $x \neq 0$, so 0 is a local minimizer.
- $g(x) = -x^4 < 0 = g(0)$ for every $x \neq 0$, so 0 is a local maximizer.
- $h(x) = x^3$ is negative for $x < 0$ and positive for $x > 0$, so $h(x) < h(0)$ to the left of 0 and $h(x) > h(0)$ to the right of 0. Thus 0 is neither a local minimizer nor a local maximizer.

R

```
sdz_x_fk <- seq(-1, 1, length.out = 201)
sdz_labels_fk <- c("x^4", "-x^4", "x^3")
sdz_values_fk <- c(sdz_x_fk^4, -sdz_x_fk^4, sdz_x_fk^3)
sdz_n_fk <- length(sdz_x_fk)
sdz_fn_fk <- rep(sdz_labels_fk, each = sdz_n_fk)
sdz_fn_fk <- factor(sdz_fn_fk, levels = sdz_labels_fk)
sdz_x_rep_fk <- rep(sdz_x_fk, 3)
sdz_curves_fk <- tibble::tibble(
  x = sdz_x_rep_fk, value = sdz_values_fk, fn = sdz_fn_fk
)
sdz_origin_fn_fk <- factor(sdz_labels_fk, levels = sdz_labels_fk)
sdz_origin_fk <- tibble::tibble(x = 0, value = 0, fn = sdz_origin_fn_fk)
sdz_aes_fk <- ggplot2::aes(x, value)
ggplot2::ggplot(sdz_curves_fk, sdz_aes_fk) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(data = sdz_origin_fk, colour = "#D55E00", size = 3) +
  ggplot2::facet_wrap(ggplot2::vars(fn)) +
  ggplot2::labs(x = "x", y = "function value")
```



Python

```
import matplotlib.pyplot as plt
import numpy as np

sdz_x_fk = np.linspace(-1, 1, 201)
sdz_curves_fk = [sdz_x_fk**4, -(sdz_x_fk**4), sdz_x_fk**3]
sdz_titles_fk = ["$x^4$", "$-x^4$", "$x^3$"]
fig, axes = plt.subplots(1, 3, figsize=(7, 3), layout="constrained")
for ax, curve, title in zip(axes, sdz_curves_fk, sdz_titles_fk):
    ax.plot(sdz_x_fk, curve, color="#0072B2")
    ax.plot(0, 0, "o", color="#D55E00", markersize=8)
    ax.set_title(title)
    ax.set_xlabel("x")
axes[0].set_ylabel("function value")
plt.show()
```

Definition 0.3 (Inflection point)**.** An **inflection point** of a function f is a point c such that, on some open interval around c, the second derivative f'' is positive on one side of c and negative on the other.

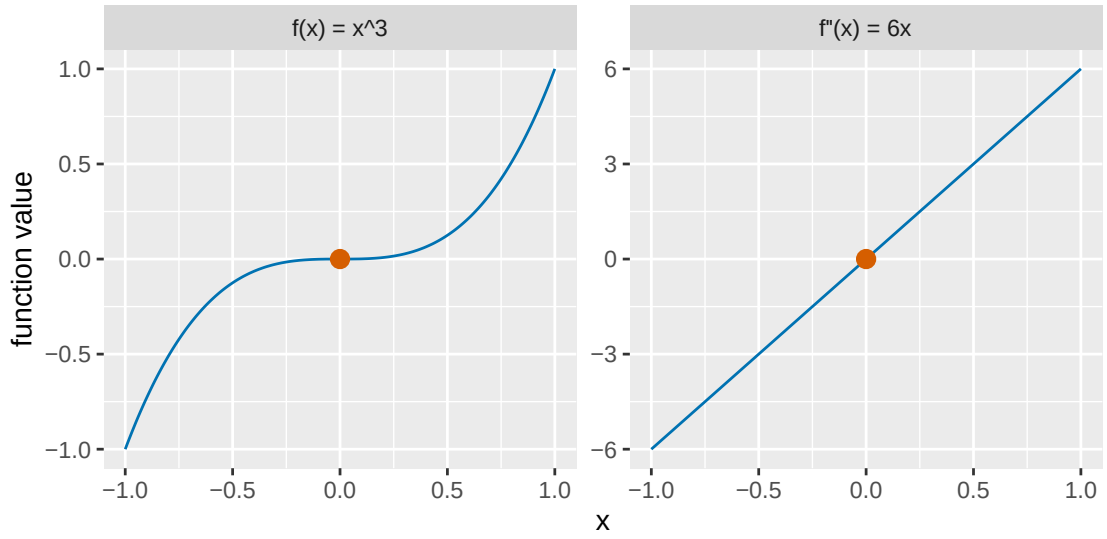
Remark (Inflection points change the curvature)

Remark 0.2 (Inflection points change the curvature). At an inflection point, the graph changes between curving up and curving down.

Example 0.6 (An inflection point of x^3)**.** For $f(x) = x^3$, the second derivative is $f''(x) = 6x$, which is negative for $x < 0$ and positive for $x > 0$. So 0 is an inflection point of f. Figure 5 shows f and f''.

R

```r
ip_x_ip <- seq(-1, 1, length.out = 201)
ip_labels_ip <- c("f(x) = x^3", "f''(x) = 6x")
ip_values_ip <- c(ip_x_ip^3, 6 * ip_x_ip)
ip_n_ip <- length(ip_x_ip)
ip_fn_ip <- rep(ip_labels_ip, each = ip_n_ip)
ip_fn_ip <- factor(ip_fn_ip, levels = ip_labels_ip)
ip_x_rep_ip <- rep(ip_x_ip, 2)
ip_curves_ip <- tibble::tibble(
  x = ip_x_rep_ip, value = ip_values_ip, fn = ip_fn_ip
)
ip_origin_fn_ip <- factor(ip_labels_ip, levels = ip_labels_ip)
ip_origin_ip <- tibble::tibble(x = 0, value = 0, fn = ip_origin_fn_ip)
ip_aes_ip <- ggplot2::aes(x, value)
ip_facet_ip <- ggplot2::vars(fn)
ip_curves_ip |>
  ggplot2::ggplot() +
  ip_aes_ip +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(data = ip_origin_ip, colour = "#D55E00", size = 3) +
  ggplot2::facet_wrap(ip_facet_ip, scales = "free_y") +
  ggplot2::labs(x = "x", y = "function value")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

ip_x_py = np.linspace(-1, 1, 201)
fig, axes = plt.subplots(1, 2, figsize=(6, 3), layout="constrained")
axes[0].plot(ip_x_py, ip_x_py**3, color="#0072B2")
axes[0].plot([0], [0], "o", color="#D55E00")
axes[0].set_title("f(x) = x^3")
axes[1].plot(ip_x_py, 6 * ip_x_py, color="#0072B2")
axes[1].plot([0], [0], "o", color="#D55E00")
```

Exercise 0.8 (Maximizing a gamma-family kernel)**.** Adapted from Miller [2], Question 1.1.10.
The gamma density with shape $a > 0$ has the kernel $x^{a-1}e^{-x}$ on $[0, \infty)$. A kernel has the same mode as its density, so maximizing the kernel finds the mode of the gamma density without computing the normalizing constant.
For shape $a = 5$, the kernel is

$$f(x) = x^4 e^{-x}.$$

Find the maximum of f on $[0, \infty)$.

Solution

Solution 0.8. Differentiate using the product rule:

$$\begin{aligned} f'(x) &= 4x^3 e^{-x} - x^4 e^{-x} \\ &= x^3 e^{-x}(4 - x) \end{aligned}$$

On $[0, \infty)$, $e^{-x} > 0$, so $f'(x) = 0$ only at $x = 0$ and $x = 4$.
Evaluate f at the boundary and critical points:

- At $x = 0$: $f(0) = 0$.
- At $x = 4$:

$$\begin{aligned} f(4) &= 4^4 e^{-4} \\ &= 256e^{-4} \\ &\approx 4.6888. \end{aligned}$$

- As $x \to \infty$: exponential decay dominates polynomial growth, so $\lim_{x\to\infty} x^4 e^{-x} = 0$.

Since $f(x) \geq 0$ for all $x \geq 0$ and $f(4) > 0$, the global maximum occurs at $x = 4$, with maximum value $256/e^4$.

Exercise 0.9 (Classifying critical points with the second derivative test)**.** Adapted from Miller [2], Question 1.1.11.
An optimization algorithm such as Newton's method or gradient descent can stop at a critical point, which may be a local maximum, a local minimum, or neither. The second derivative test tells these apart.
Consider the cubic

$$f(x) = 4x^3 - 3x^2.$$

Find its critical points, and decide whether each is a local maximum, a local minimum, or an inflection point.

Solution

Solution 0.9. Compute the first derivative:

$$\begin{aligned} f'(x) &= 12x^2 - 6x \\ &= 6x(2x - 1) \end{aligned}$$

Setting $f'(x) = 0$ gives critical points at $x = 0$ and $x = 1/2$.
Compute the second derivative:

$$f''(x) = 24x - 6$$

Apply the second derivative test at each critical point:

- At $x = 0$: $f''(0) = -6 < 0$, so $x = 0$ is a strict local maximum.
- At $x = 1/2$:

$$\begin{aligned} f''(1/2) &= 24(1/2) - 6 \\ &= 6 \\ &> 0, \end{aligned}$$

  so $x = 1/2$ is a strict local minimum.

Inflection points occur where the second derivative changes sign ($f''(x) = 0$ at $x = 1/4$). Because f'' is non-zero at both critical points, neither is an inflection point.

Optimality conditions in several variables

Source

This section is adapted from the multivariate calculus and optimality-condition parts of Zhou [3], used under the MIT License (see the license text in Linear Algebra). The source states the second-order Taylor approximation and the optimality conditions without proof; here they are proved from the one-variable Taylor theorem, and the approximation is made exact by evaluating the Hessian at a point between $\underset{\sim}{z}$ and $\underset{\sim}{z} + \underset{\sim}{h}$. The source calls points with zero gradient critical points; here they are stationary points, because Definition 0.2 gives "critical point" a wider meaning. The source states these

results for differentiable functions without saying how smooth; here the first-order results assume continuous first partial derivatives, and the second-order results continuous second partial derivatives. These parts of the source are not part of this section:

- its remark that maximizing f is minimizing $-f$, and its classification of constraints
- its figure of local, strict local and global minima
- its one-variable Taylor example and its plots
- matrix calculus
- convexity
- Lagrange multipliers
- Newton's method and gradient descent

A local minimizer of a differentiable function on an open interval is a flat point (Definition 0.1), and, at a flat point, a positive second derivative guarantees a strict local minimum. This section extends both facts to a function f of a $p \times 1$ vector $\underset{\sim}{x}$: the gradient $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$ takes the place of the first derivative, and a $p \times p$ matrix of second partial derivatives takes the place of the second; both facts are proved in this section, for every $p \geq 1$. Throughout, $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{z})$ means the gradient evaluated at $\underset{\sim}{x} = \underset{\sim}{z}$.

Theorem 0.1 (A minimizer has derivative zero)**.** *Let g be a real-valued function on an open interval containing c, and let g be differentiable at c. If c is a local minimizer of g, then $g'(c) = 0$. [4]*

Proof

Proof. Take $\delta > 0$ as in the definition of a local minimizer, shrunk if needed so that $(c-\delta, c+\delta)$ lies inside the interval; then $g(c) \leq g(c+h)$ whenever $|h| < \delta$. For $0 < h < \delta$, the difference quotient is

$$\frac{g(c+h) - g(c)}{h} \geq 0,$$

because its numerator is at least 0 and its denominator is positive. As $h \to 0$ from the right, these quotients tend to $g'(c)$ (the two-sided limit exists, so the one-sided limit equals it), and a limit of numbers that are all at least 0 is at least 0; so $g'(c) \geq 0$. For $-\delta < h < 0$, the numerator is still at least 0 but the denominator is negative, so the quotient is at most 0, and in the same way, with $h \to 0$ from the left, $g'(c) \leq 0$. Together, $g'(c) = 0$. □

Example 0.1 (Applying the test, and its limits)**.**

- $g(x) = (x-2)^2$ has its minimizer at $c = 2$, and

$$g'(2) = 2(2-2)$$
$$= 0.$$

- The converse fails: $g(x) = x^3$ has

$$g'(0) = 3 \cdot 0^2$$
$$= 0,$$

but 0 is not a local minimizer, since

$$g(-h) = -h^3$$
$$< 0$$
$$= g(0)$$

for every $h > 0$.

- Differentiability is needed: $g(x) = |x|$ has its minimizer at 0, but the difference quotient $|h|/h$ is 1 for $h > 0$ and -1 for $h < 0$, so it has no limit and $g'(0)$ does not exist.

Definition 0.1 (Stationary point)**.** Let f be a scalar-valued function of a $p \times 1$ vector $\underset{\sim}{x}$. A point $\underset{\sim}{z}$ at which the gradient exists and

$$\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{z}) = \underset{\sim}{0}_{p\times 1}$$

is a **stationary point** of f.

Example 0.2 (A stationary point, and a point that is not one)**.** Let $f(\underset{\sim}{x}) = x_1^2 + x_2^2 - 2x_1$. Its gradient is

$$\nabla f(\underset{\sim}{x}) = (2x_1 - 2,\ 2x_2)^\top,$$

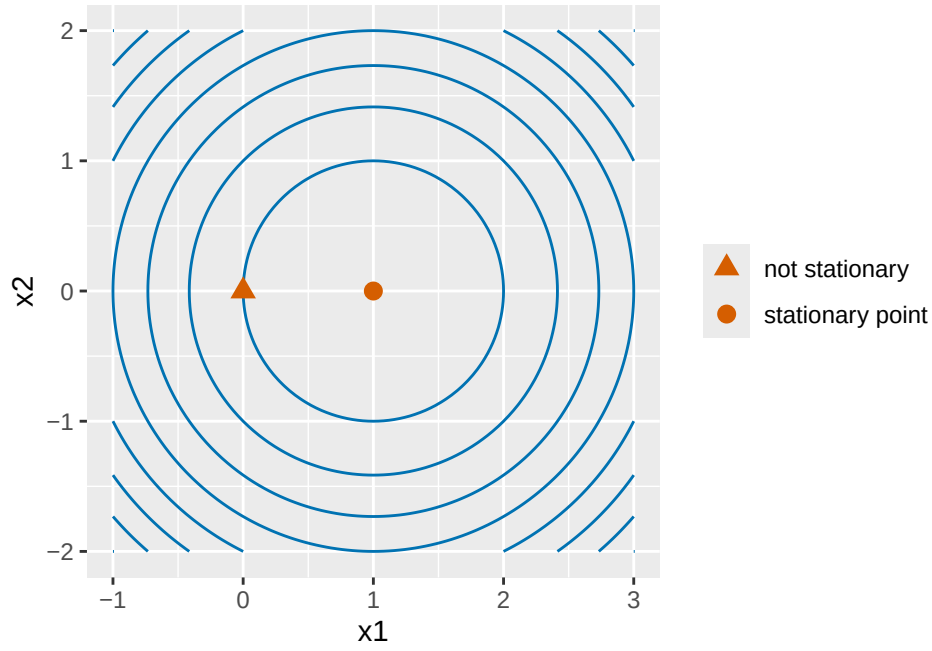
which is $\underset{\sim}{0}_{2\times 1}$ exactly when $x_1 = 1$ and $x_2 = 0$. So $(1, 0)^\top$ is its only stationary point, and $\underset{\sim}{0}$ is not one: $\nabla f(\underset{\sim}{0}_{2\times 1}) = (-2, 0)^\top$.

Remark (Stationary points in one variable)

Remark 0.1 (Stationary points in one variable). For $p = 1$, a stationary point is a flat point (Definition 0.1).

R

```r
sp_x1_fs <- seq(-1, 3, length.out = 101)
sp_x2_fs <- seq(-2, 2, length.out = 101)
sp_grid_fs <- expand.grid(x1 = sp_x1_fs, x2 = sp_x2_fs)
sp_grid_fs$value <- sp_grid_fs$x1^2 + sp_grid_fs$x2^2 - 2 * sp_grid_fs$x1
sp_marks_x1_fs <- c(1, 0)
sp_marks_kind_fs <- c("stationary point", "not stationary")
sp_marks_fs <- tibble::tibble(
  x1 = sp_marks_x1_fs, x2 = 0, kind = sp_marks_kind_fs
)
sp_aes_fs <- ggplot2::aes(x1, x2, z = value)
sp_marks_aes_fs <- ggplot2::aes(x1, x2, shape = kind, z = NULL)
sp_shapes_fs <- c("stationary point" = 16, "not stationary" = 17)
ggplot2::ggplot(sp_grid_fs, sp_aes_fs) +
  ggplot2::geom_contour(colour = "#0072B2") +
  ggplot2::geom_point(
    data = sp_marks_fs, mapping = sp_marks_aes_fs,
    colour = "#D55E00", size = 3, inherit.aes = FALSE
  ) +
  ggplot2::scale_shape_manual(values = sp_shapes_fs) +
  ggplot2::coord_fixed() +
  ggplot2::labs(x = "x1", y = "x2", shape = NULL)
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

sp_x1_fs = np.linspace(-1, 3, 101)
sp_x2_fs = np.linspace(-2, 2, 101)
```

R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

sp_x1_3d = np.linspace(-1, 3, 101)
sp_x2_3d = np.linspace(-2, 2, 101)
sp_x1_grid_3d, sp_x2_grid_3d = np.meshgrid(sp_x1_3d, sp_x2_3d)
sp_value_3d = sp_x1_grid_3d**2 + sp_x2_grid_3d**2 - 2 * sp_x1_grid_3d
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(sp_x1_grid_3d, sp_x2_grid_3d, sp_value_3d,
                color="#0072B2", alpha=0.6)
ax.scatter([1], [0], [-1], marker="o", color="#D55E00", s=50,
           label="stationary point")
ax.scatter([0], [0], [0], marker="D", color="#D55E00", s=50,
           label="not stationary")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f(x)")
ax.legend(loc="upper right")
plt.show();
```

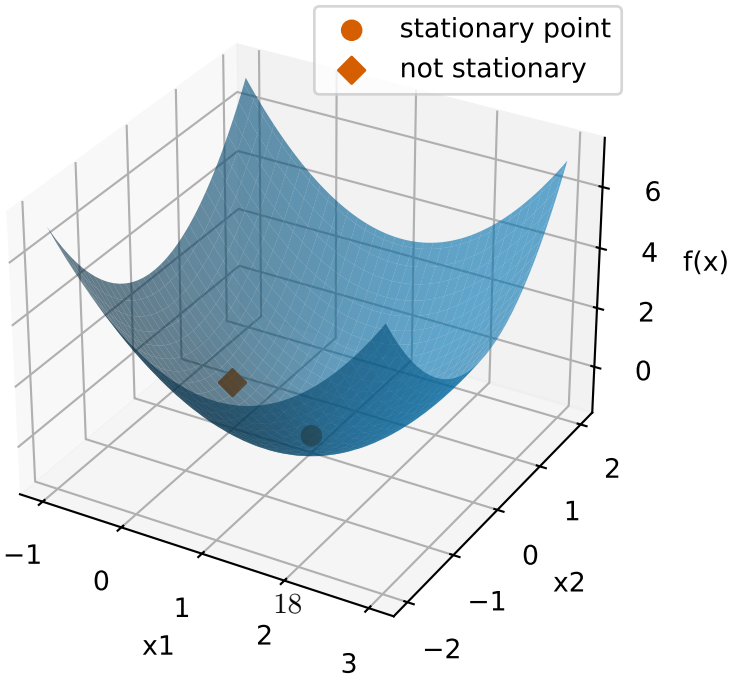


Figure 7: The graph of the function of Example 0.2, the surface whose level curves are in Figure 6. The surface is flat, with gradient 0, only at the bottom of the bowl

Theorem 0.2 (First-order necessary condition)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$. If $\underset{\sim}{x}^*$ is a local minimizer of f and the gradient of f exists at $\underset{\sim}{x}^*$, then $\underset{\sim}{x}^*$ is a stationary point of f (Definition 0.1).*

Proof

Proof. Take $\delta > 0$ as in the definition of a local minimizer. Let $i \in \{1, \dots, p\}$, let $\underset{\sim}{e}_i$ be the vector with 1 in entry i and 0 elsewhere, and let $g_i(t) \overset{\text{def}}{=} f(\underset{\sim}{x}^* + t\,\underset{\sim}{e}_i)$ for $t \in \mathbb{R}$. For $|t| < \delta$,

$$\begin{aligned} \|(\underset{\sim}{x}^* + t\,\underset{\sim}{e}_i) - \underset{\sim}{x}^*\| &= |t|\,\|\underset{\sim}{e}_i\| \\ &= |t| \\ &< \delta, \end{aligned}$$

so

$$\begin{aligned} g_i(0) &= f(\underset{\sim}{x}^*) \\ &\le f(\underset{\sim}{x}^* + t\,\underset{\sim}{e}_i) \\ &= g_i(t) : \end{aligned}$$

0 is a local minimizer of g_i. Moving $\underset{\sim}{x}$ from $\underset{\sim}{x}^*$ along $\underset{\sim}{e}_i$ changes only x_i, so $g_i'(0)$ is the partial derivative $\frac{\partial}{\partial x_i} f(\underset{\sim}{x}^*)$, which exists because the gradient does. By Theorem 0.1,

$$\begin{aligned} \frac{\partial}{\partial x_i} f(\underset{\sim}{x}^*) &= g_i'(0) \\ &= 0. \end{aligned}$$

The equation $g_i'(0) = 0$ holds for every i, so every entry of the gradient at $\underset{\sim}{x}^*$ is 0. □

Example 0.3 (Using the condition to locate, and to rule out, minimizers)**.**

- For $f(\underset{\sim}{x}) = x_1^2 + x_2^2 - 2x_1$ of Example 0.2, any local minimizer must be the stationary point $(1, 0)^\top$. It is one: completing the square,

$$\begin{aligned} f(\underset{\sim}{x}) &= (x_1 - 1)^2 + x_2^2 - 1 \\ &\ge -1 \\ &= f(1, 0). \end{aligned}$$

- For $f(\underset{\sim}{x}) = e^{2x_1 + x_2} - x_1$ of the Hessian example, the second entry of $\nabla f(\underset{\sim}{x})$ is $e^{2x_1 + x_2} > 0$, so f has no stationary point, and so no local minimizer.
- The converse fails: $f(\underset{\sim}{x}) = x_1^2 - x_2^2$ has gradient $(2x_1,\ -2x_2)^\top$, so $\underset{\sim}{0}$ is a stationary point, but

$$\begin{aligned} f(0,t) &= -t^2 \\ &< 0 \\ &= f(\underset{\sim}{0}) \end{aligned}$$

for every $t \neq 0$, so $\underset{\sim}{0}$ is not a local minimizer.

Example 0.4 (A stationary point of the sum of squared errors, on real data)**.** The Auto data record the horsepower and fuel economy (miles per gallon) of 392 car models sold between 1970 and 1982. They come from the StatLib library at Carnegie Mellon University and ship with the book *An Introduction to Statistical Learning*, in the Python package `ISLP` (`ISLP.load_data("Auto")`) and the R package `ISLR2` (`ISLR2::Auto`). This site has a copy at `data/auto.csv`, and both code tabs below read that file.
Let y_i be the miles per gallon and x_i the horsepower of car i. To predict miles per gallon from horsepower with a line, write $\beta = (\beta_0, \beta_1)^\top$ and let $\mathbf{X}$ be the matrix whose row i is $(1, x_i)$. The sum of squared errors is

$$f(\underset{\sim}{\beta}) = \|\underset{\sim}{y} - \mathbf{X}\underset{\sim}{\beta}\|^2,$$

a scalar-valued function of the 2×1 vector $\underset{\sim}{\beta}$. Its gradient is

$$\nabla f(\underset{\sim}{\beta}) = -2\,\mathbf{X}^\top(\underset{\sim}{y} - \mathbf{X}\underset{\sim}{\beta}).$$

By Theorem 0.2, a minimizer of f must make this gradient $\underset{\sim}{0}$, which is the equation $\mathbf{X}^\top\mathbf{X}\beta = \mathbf{X}^\top y$. The code below solves that equation and evaluates the gradient at the solution and at $\underset{\sim}{\beta} = \underset{\sim}{0}$.

Python

```python
import numpy as np
import pandas as pd

auto = pd.read_csv("data/auto.csv")
y = auto["mpg"].to_numpy(dtype=float)
X = np.column_stack([np.ones(len(auto)), auto["horsepower"].to_numpy(dtype=float)])

def grad(beta):
    return -2 * X.T @ (y - X @ beta)

beta_hat = np.linalg.solve(X.T @ X, X.T @ y)
grad_hat = grad(beta_hat)
grad_zero = grad(np.zeros(2))

pd.DataFrame(
    {"beta": [beta_hat, np.zeros(2)], "gradient": [grad_hat, grad_zero]},
    index=["solution", "zero vector"],
)
#>                                beta                gradient
#> solution     [39.935861, -0.157845]              [0.0, 0.0]
#> zero vector              [0.0, 0.0]  [-18381.6, -1737437.6]
```

A library fit gives the same coefficients:

```python
import numpy as np

# np.polyfit returns the slope first, so reverse to (intercept, slope).
beta_polyfit = np.polyfit(auto["horsepower"], auto["mpg"], deg=1)[::-1]
beta_polyfit
#> array([39.93586102, -0.15784473])
```

R

```r
auto <- read.csv("data/auto.csv")
y <- auto$mpg
X <- cbind(1, auto$horsepower)

grad <- function(beta) -2 * t(X) %*% (y - X %*% beta)

beta_hat <- solve(t(X) %*% X, t(X) %*% y)
grad_hat <- grad(beta_hat)
grad_zero <- grad(c(0, 0))

list(
  solution = list(beta = c(beta_hat), gradient = c(grad_hat)),
  zero_vector = list(beta = c(0, 0), gradient = c(grad_zero))
)
#> $solution
#> $solution$beta
#> [1] 39.9358610 -0.1578447
#>
#> $solution$gradient
#> [1] 1.071498e-11 1.673470e-10
#>
#>
#> $zero_vector
#> $zero_vector$beta
#> [1] 0 0
#>
#> $zero_vector$gradient
#> [1]   -18381.6 -1737437.6
```

A library fit gives the same coefficients:

```r
coef(lm(mpg ~ horsepower, data = auto))
#> (Intercept)  horsepower
#>  39.9358610  -0.1578447
```

The solution is an intercept of 39.9 miles per gallon and a slope of -0.158 miles per gallon per unit of horsepower. The largest entry of the gradient there is 1.7e-10, zero up to rounding error, against 1,737,438 at the zero vector. So the solution is a stationary point (Definition 0.1), and the zero vector is not. The library fits above agree with the solution of that equation.

Theorem 0.3 (Taylor's theorem with a second-order remainder)**.** *Let $a < b$, and let g be a real-valued function on $[a, b]$ whose derivative g' is continuous on $[a, b]$ and whose second derivative g'' exists at every point of (a, b). Then there is a $\tau \in (a, b)$ with [5]*

$$g(b) = g(a) + g'(a)\,(b - a) + \frac{1}{2}\,g''(\tau)\,(b - a)^2.$$

This is the case $n = 2$ of [6], Theorem 5.15, whose proof uses the mean value theorem, which these notes do not develop.

Example 0.5 (The remainder point for e^x)**.** Take $g(x) = e^x$ on $[0, 1]$, so

$$\begin{aligned} g' &= g'' \\ &= g. \end{aligned}$$

Theorem 0.3 says $e = 1 + 1 + \frac{1}{2}\,e^\tau$ for some $\tau \in (0, 1)$. Solving,

$$\begin{aligned} e^\tau &= 2(e - 2) \\ &\approx 2 \times 0.71828 \\ &= 1.43656, \end{aligned}$$

so $\tau = \log 1.43656 \approx 0.362$, where log is the natural logarithm. This τ is in $(0, 1)$.

Theorem 0.4 (Second-order Taylor theorem in several variables)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$ have first and second partial derivatives that are continuous on $\mathbb{R}^p$, and let $\underset{\sim}{z}, \underset{\sim}{h} \in \mathbb{R}^p$. Then there is a $\tau \in (0, 1)$ with [5]*

$$f(\underset{\sim}{z} + \underset{\sim}{h}) = f(\underset{\sim}{z}) + \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{z})\right)^{\top} \underset{\sim}{h} + \frac{1}{2}\,\underset{\sim}{h}^{\top}\,\mathbf{H}_f(\underset{\sim}{z} + \tau \underset{\sim}{h})\,\underset{\sim}{h}.$$

Proof

Proof. Let $g(t) \overset{\text{def}}{=} f(\underset{\sim}{z} + t\underset{\sim}{h})$ for $t \in \mathbb{R}$, the values of f along the line through $\underset{\sim}{z}$ in the direction $\underset{\sim}{h}$. The inner function $\underset{\sim}{y}(t) = \underset{\sim}{z} + t\underset{\sim}{h}$ has entries $y_j = z_j + t h_j$, so $\frac{\partial}{\partial t} y_j = h_j$, and its derivative is the $1 \times p$ matrix $\underset{\sim}{h}^{\top}$. By the vector chain rule applied with the argument t, the inner function $\underset{\sim}{y}(t) = \underset{\sim}{z} + t\underset{\sim}{h}$, and the outer function f,

$$\begin{aligned} g'(t) &= \underset{\sim}{h}^{\top} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{z} + t\underset{\sim}{h}) && \text{(vector chain rule)} \\ &= \sum_{j=1}^{p} h_j \frac{\partial}{\partial x_j} f(\underset{\sim}{z} + t\underset{\sim}{h}). && \text{(matrix product)} \end{aligned}$$

The second step writes out the matrix product. g' is continuous: $t \mapsto \underset{\sim}{z}+t\underset{\sim}{h}$ is continuous, since $\|(\underset{\sim}{z}+s\underset{\sim}{h})-(\underset{\sim}{z}+t\underset{\sim}{h})\| = |s-t|\,\|\underset{\sim}{h}\|$, each $\frac{\partial}{\partial x_j}f$ is continuous by assumption, and compositions, sums and constant multiples of continuous functions are continuous. Each $\frac{\partial}{\partial x_j}f$ has continuous partial derivatives, the second partial derivatives of f, so the same chain-rule computation applies to it: $\frac{\partial}{\partial t}\left[\frac{\partial}{\partial x_j}f(\underset{\sim}{z}+t\underset{\sim}{h})\right] = \sum_{i=1}^{p} h_i \frac{\partial}{\partial x_i}\left(\frac{\partial}{\partial x_j}f(\underset{\sim}{z}+t\underset{\sim}{h})\right)$. So

$$
\begin{aligned}
g''(t) &= \sum_{j=1}^{p} h_j \sum_{i=1}^{p} h_i \frac{\partial}{\partial x_i}\left(\frac{\partial}{\partial x_j}f(\underset{\sim}{z}+t\underset{\sim}{h})\right) && \text{(differentiate each term of } g'(t)\text{)} \\
&= \sum_{i=1}^{p}\sum_{j=1}^{p} h_i \left[\mathbf{H}_f(\underset{\sim}{z}+t\underset{\sim}{h})\right]_{ij} h_j && \text{(definition of the Hessian; reorder the finite sums)} \\
&= \underset{\sim}{h}^{\top}\mathbf{H}_f(\underset{\sim}{z}+t\underset{\sim}{h})\,\underset{\sim}{h}. && \text{(matrix product)}
\end{aligned}
$$

The last step is again the matrix product. Now apply Theorem 0.3 to g on $[0,1]$: there is a $\tau \in (0,1)$ with

$$
\begin{aligned}
f(\underset{\sim}{z}+\underset{\sim}{h}) &= g(1) && \text{(definition of } g\text{)} \\
&= g(0) + g'(0)\,(1-0) + \frac{1}{2}\,g''(\tau)\,(1-0)^2 && \text{(Theorem 0.3, with } a=0, b=1\text{)} \\
&= g(0) + g'(0) + \frac{1}{2}\,g''(\tau) && (1-0=1) \\
&= f(\underset{\sim}{z}) + \underset{\sim}{h}^{\top}\frac{\partial}{\partial \underset{\sim}{x}}f(\underset{\sim}{z}) + \frac{1}{2}\,\underset{\sim}{h}^{\top}\mathbf{H}_f(\underset{\sim}{z}+\tau\underset{\sim}{h})\,\underset{\sim}{h} && \text{(substitute } g(0), g'(0), g''(\tau)\text{)} \\
&= f(\underset{\sim}{z}) + \left(\frac{\partial}{\partial \underset{\sim}{x}}f(\underset{\sim}{z})\right)^{\top}\underset{\sim}{h} + \frac{1}{2}\,\underset{\sim}{h}^{\top}\mathbf{H}_f(\underset{\sim}{z}+\tau\underset{\sim}{h})\,\underset{\sim}{h}. && \text{(a } 1\times 1 \text{ matrix equals its transpose)}
\end{aligned}
$$

□

Example 0.6 (For a quadratic form the expansion is exact)**.** Let $f(\underset{\sim}{x}) = \underset{\sim}{x}^{\top}\mathbf{S}\underset{\sim}{x}$ with $\mathbf{S}$ symmetric and constant. Its gradient is $\nabla f(\underset{\sim}{x}) = 2\mathbf{S}\underset{\sim}{x}$ (quadratic form gradient) and its Hessian is $2\mathbf{S}$ at every point (quadratic form Hessian), so whatever τ is, Theorem 0.4 reads

$$
\begin{aligned}
f(\underset{\sim}{z}+\underset{\sim}{h}) &= \underset{\sim}{z}^{\top}\mathbf{S}\underset{\sim}{z} + (2\mathbf{S}\underset{\sim}{z})^{\top}\underset{\sim}{h} + \frac{1}{2}\,\underset{\sim}{h}^{\top}(2\mathbf{S})\underset{\sim}{h} \\
&= \underset{\sim}{z}^{\top}\mathbf{S}\underset{\sim}{z} + 2\,\underset{\sim}{z}^{\top}\mathbf{S}\underset{\sim}{h} + \underset{\sim}{h}^{\top}\mathbf{S}\underset{\sim}{h},
\end{aligned}
$$

using

$$
\begin{aligned}
(2\mathbf{S}\underset{\sim}{z})^\top &= 2\,\underset{\sim}{z}^\top \mathbf{S}^\top \\
&= 2\,\underset{\sim}{z}^\top \mathbf{S}.
\end{aligned}
$$

Multiplying out directly gives the same:

$$
\begin{aligned}
(\underset{\sim}{z}+\underset{\sim}{h})^\top \mathbf{S}(\underset{\sim}{z}+\underset{\sim}{h}) &= \underset{\sim}{z}^\top \mathbf{S}\underset{\sim}{z} + \underset{\sim}{z}^\top \mathbf{S}\underset{\sim}{h} + \underset{\sim}{h}^\top \mathbf{S}\underset{\sim}{z} + \underset{\sim}{h}^\top \mathbf{S}\underset{\sim}{h} && \text{(distribute)} \\
&= \underset{\sim}{z}^\top \mathbf{S}\underset{\sim}{z} + 2\,\underset{\sim}{z}^\top \mathbf{S}\underset{\sim}{h} + \underset{\sim}{h}^\top \mathbf{S}\underset{\sim}{h}. && (\underset{\sim}{h}^\top \mathbf{S}\underset{\sim}{z} = (\underset{\sim}{h}^\top \mathbf{S}\underset{\sim}{z})^\top, \text{ which equals } \underset{\sim}{z}^\top \mathbf{S}\underset{\sim}{h})
\end{aligned}
$$

Definition 0.2 (Strict local minimizer)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$. A point $\underset{\sim}{x}^*$ is a **strict local minimizer** of f if there is a number $\delta > 0$ such that $f(\underset{\sim}{x}^*) < f(\underset{\sim}{x})$ for every $\underset{\sim}{x} \neq \underset{\sim}{x}^*$ with $\|\underset{\sim}{x} - \underset{\sim}{x}^*\| < \delta$.

Example 0.7 (Strict and non-strict local minimizers)**.**

- $f(x) = (x-2)^2$ has

$$
\begin{aligned}
f(2) &= 0 \\
&< (x-2)^2 \\
&= f(x)
\end{aligned}
$$

  for every $x \neq 2$, so 2 is a strict local minimizer (any $\delta > 0$ works).

- A constant function $f(\underset{\sim}{x}) = 0$ has $f(\underset{\sim}{x}^*) \leq f(\underset{\sim}{x})$ for all $\underset{\sim}{x}^*$ and $\underset{\sim}{x}$, so every point is a local minimizer, but no point is a strict one: $f(\underset{\sim}{x}^*) < f(\underset{\sim}{x})$ never holds.

Lemma 0.1 (A quadratic form is at least the smallest eigenvalue times the squared length)**.** *Let* $\mathbf{A}$ *be a symmetric* $p \times p$ *matrix, and let* $\lambda_{\min}$ *be the smallest of its eigenvalues. Then* $\underset{\sim}{h}^\top \mathbf{A}\underset{\sim}{h} \geq \lambda_{\min} \|\underset{\sim}{h}\|^2$ *for every* $\underset{\sim}{h} \in \mathbb{R}^p$.

Proof

Proof. By the spectral theorem, write $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with $\mathbf{Q}$ an orthogonal matrix and $\mathbf{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_p)$, and let $\underset{\sim}{y} = \mathbf{Q}^\top \underset{\sim}{h}$. Then

$$
\begin{aligned}
\underset{\sim}{h}^\top \mathbf{A} \underset{\sim}{h} &= \underset{\sim}{h}^\top \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top \underset{\sim}{h} && \text{(substitute the eigendecomposition)} \\
&= \underset{\sim}{y}^\top \mathbf{\Lambda} \underset{\sim}{y} && (\underset{\sim}{h}^\top \mathbf{Q} = (\mathbf{Q}^\top \underset{\sim}{h})^\top, \text{ transpose of a product}) \\
&= \sum_{i=1}^{p} \lambda_i \, y_i^2 && (\mathbf{\Lambda} \text{ is diagonal}) \\
&\geq \sum_{i=1}^{p} \lambda_{\min} \, y_i^2 && (\lambda_i \geq \lambda_{\min} \text{ and } y_i^2 \geq 0) \\
&= \lambda_{\min} \, \|\underset{\sim}{y}\|^2 && \text{(the squared length is the sum of squares)} \\
&= \lambda_{\min} \, \|\underset{\sim}{h}\|^2. && \text{(orthogonal matrices preserve length)}
\end{aligned}
$$

The second step is the transpose of a product. The last step applies orthogonal matrices preserve length to $\mathbf{Q}^\top$. That theorem applies because $\mathbf{Q}^\top$ is an orthogonal matrix: since $\mathbf{Q}$ is square with $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_p$, also $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}_p$, so

$$(\mathbf{Q}^\top)^\top \mathbf{Q}^\top = \mathbf{Q}\mathbf{Q}^\top = \mathbf{I}_p.$$

□

Example 0.8 (Checking the bound)**.** $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ has eigenvalues 3 and 1, so $\lambda_{\min} = 1$. At $\underset{\sim}{h} = (1, 0)^\top$, $\underset{\sim}{h}^\top \mathbf{A} \underset{\sim}{h} = 2 \geq 1 \cdot 1$. At $\underset{\sim}{h} = (1, -1)^\top$,

$$
\begin{aligned}
\underset{\sim}{h}^\top \mathbf{A} \underset{\sim}{h} &= 2 - 1 - 1 + 2 \\
&= 2 \\
&= 1 \cdot 2 :
\end{aligned}
$$

the bound holds with equality, since this $\underset{\sim}{h}$ is an eigenvector for $\lambda_{\min}$.

Lemma 0.2 (A quadratic form with small entries is small)**.** *Let* $\mathbf{E}$ *be a* $p \times p$ *matrix whose entries all satisfy* $|e_{ij}| \leq m$*. Then* $|\underset{\sim}{h}^\top \mathbf{E} \underset{\sim}{h}| \leq p \, m \, \|\underset{\sim}{h}\|^2$ *for every* $\underset{\sim}{h} \in \mathbb{R}^p$*.*

Proof

Proof. Let $\underset{\sim}{a} = (|h_1|, \ldots, |h_p|)^\top$ and $\underset{\sim}{1} = (1, \ldots, 1)^\top$, both in $\mathbb{R}^p$. Then

$$
\begin{aligned}
\left|\underset{\sim}{h}^{\top}\mathbf{E}\underset{\sim}{h}\right| &= \left|\sum_{i=1}^{p}\sum_{j=1}^{p} h_i\, e_{ij}\, h_j\right| && \text{(matrix product)}\\
&\le \sum_{i=1}^{p}\sum_{j=1}^{p}|h_i|\,|e_{ij}|\,|h_j| && \text{(triangle inequality for numbers)}\\
&\le m\sum_{i=1}^{p}\sum_{j=1}^{p}|h_i|\,|h_j| && (|e_{ij}|\le m)\\
&= m\left(\sum_{i=1}^{p}|h_i|\right)^2 && \text{(the double sum factors)}\\
&= m\,(\underset{\sim}{1}\cdot\underset{\sim}{a})^2 && \text{(dot product)}\\
&\le m\,\|\underset{\sim}{1}\|^2\,\|\underset{\sim}{a}\|^2 && \text{(Cauchy-Schwarz, squared)}\\
&= m\,p\,\|\underset{\sim}{h}\|^2. && (\|\underset{\sim}{1}\|^2 = p \text{ and } \|\underset{\sim}{a}\|^2 = \textstyle\sum_i h_i^2, \text{ which equals } \|\underset{\sim}{h}\|^2)
\end{aligned}
$$

The steps use the matrix product, the dot product and the Cauchy-Schwarz inequality. □

Example 0.9 (Checking the bound)**.** Let $\mathbf{E} = \begin{bmatrix} 0.1 & -0.2 \\ -0.2 & 0.1 \end{bmatrix}$, so $m = 0.2$ works. Let $\underset{\sim}{h} = (1,1)^{\top}$, so $\|\underset{\sim}{h}\|^2 = 2$. Then

$$
\begin{aligned}
\underset{\sim}{h}^{\top}\mathbf{E}\underset{\sim}{h} &= 0.1 - 0.2 - 0.2 + 0.1\\
&= -0.2,
\end{aligned}
$$

and

$$
\begin{aligned}
|-0.2| &= 0.2\\
&\le 2\cdot 0.2\cdot 2\\
&= 0.8.
\end{aligned}
$$

Theorem 0.5 (Second-order sufficient condition)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$ have first and second partial derivatives that are continuous on $\mathbb{R}^p$. If $\underset{\sim}{x}^*$ is a stationary point of f (Definition 0.1) and $\mathbf{H}_f(\underset{\sim}{x}^*)$ is positive definite, then $\underset{\sim}{x}^*$ is a strict local minimizer of f (Definition 0.2).*

Proof

Proof. $\mathbf{H}_f(\underset{\sim}{x}^*)$ is positive definite, and so symmetric, so its smallest eigenvalue λ is positive, by the eigenvalue test for definiteness. Each of the p^2 entries of $\mathbf{H}_f$ is continuous

at $\underset{\sim}{x}^*$: for $\varepsilon = \lambda/(2p)$ there is a $\delta_{ij} > 0$ with $\left|\left[\mathbf{H}_f(\underset{\sim}{y})\right]_{ij} - \left[\mathbf{H}_f(\underset{\sim}{x}^*)\right]_{ij}\right| < \lambda/(2p)$ whenever $\|\underset{\sim}{y} - \underset{\sim}{x}^*\| < \delta_{ij}$. Let δ be the smallest of these p^2 numbers.
Now let $0 < \|\underset{\sim}{h}\| < \delta$. By Theorem 0.4 there is a $\tau \in (0,1)$ for which the first step of the next display holds; let $\underset{\sim}{y} \overset{\text{def}}{=} \underset{\sim}{x}^* + \tau\underset{\sim}{h}$ and $\mathbf{E} \overset{\text{def}}{=} \mathbf{H}_f(\underset{\sim}{y}) - \mathbf{H}_f(\underset{\sim}{x}^*)$. Since $\|\underset{\sim}{y} - \underset{\sim}{x}^*\| = \tau\|\underset{\sim}{h}\| < \delta$, every entry of $\mathbf{E}$ has absolute value less than $\lambda/(2p)$, so Lemma 0.2 applies with $m = \lambda/(2p)$:

$$
\begin{aligned}
f(\underset{\sim}{x}^* + \underset{\sim}{h}) - f(\underset{\sim}{x}^*) &= \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^*)\right)^{\top} \underset{\sim}{h} + \frac{1}{2}\,\underset{\sim}{h}^{\top}\,\mathbf{H}_f(\underset{\sim}{y})\,\underset{\sim}{h} && \text{(Theorem 0.4)} \\
&= \frac{1}{2}\,\underset{\sim}{h}^{\top}\,\mathbf{H}_f(\underset{\sim}{y})\,\underset{\sim}{h} && (\underset{\sim}{x}^* \text{ is stationary}) \\
&= \frac{1}{2}\,\underset{\sim}{h}^{\top}\,\mathbf{H}_f(\underset{\sim}{x}^*)\,\underset{\sim}{h} + \frac{1}{2}\,\underset{\sim}{h}^{\top}\,\mathbf{E}\,\underset{\sim}{h} && (\mathbf{H}_f(\underset{\sim}{y}) = \mathbf{H}_f(\underset{\sim}{x}^*) + \mathbf{E}\text{; distribute}) \\
&\geq \frac{1}{2}\,\lambda\,\|\underset{\sim}{h}\|^2 + \frac{1}{2}\,\underset{\sim}{h}^{\top}\,\mathbf{E}\,\underset{\sim}{h} && \text{(Lemma 0.1)} \\
&\geq \frac{1}{2}\,\lambda\,\|\underset{\sim}{h}\|^2 - \frac{1}{2}\left|\underset{\sim}{h}^{\top}\,\mathbf{E}\,\underset{\sim}{h}\right| && \text{(a number is at least minus its absolute value)} \\
&\geq \frac{1}{2}\,\lambda\,\|\underset{\sim}{h}\|^2 - \frac{1}{2}\,p\,\frac{\lambda}{2p}\,\|\underset{\sim}{h}\|^2 && \text{(Lemma 0.2, } m = \lambda/(2p)) \\
&= \frac{\lambda}{4}\,\|\underset{\sim}{h}\|^2 && \text{(arithmetic)} \\
&> 0. && (\lambda > 0 \text{ and } \underset{\sim}{h} \neq \underset{\sim}{0})
\end{aligned}
$$

So $f(\underset{\sim}{x}^*) < f(\underset{\sim}{x})$ whenever $0 < \|\underset{\sim}{x} - \underset{\sim}{x}^*\| < \delta$ (take $\underset{\sim}{h} = \underset{\sim}{x} - \underset{\sim}{x}^*$). □

Example 0.10 (Classifying stationary points)**.**

- $f(\underset{\sim}{x}) = x_1^2 + x_2^2 - 2x_1$ has the stationary point $(1,0)^{\top}$ (Example 0.2), and its Hessian is

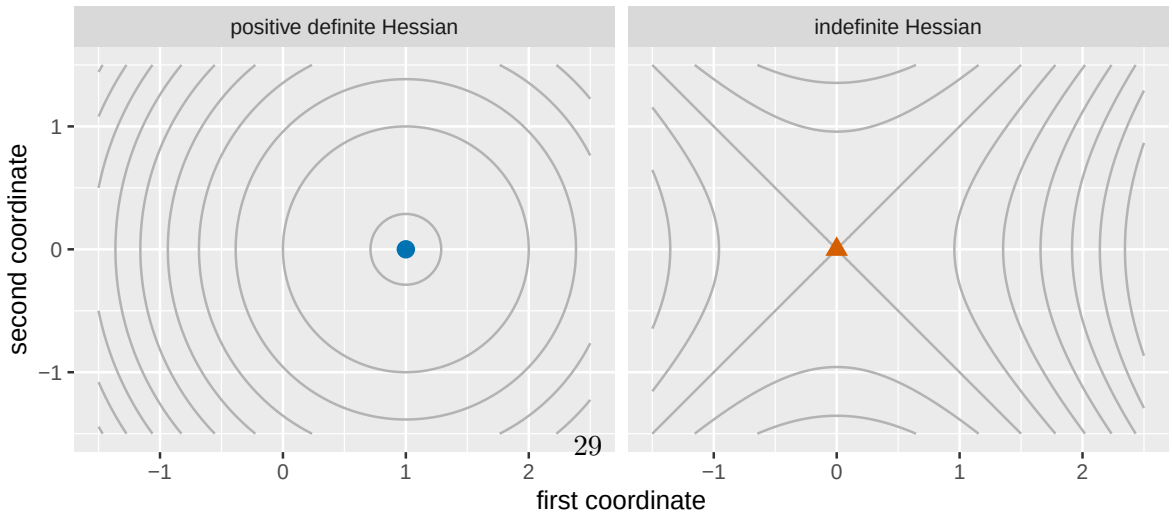
$$
\begin{aligned}
\mathbf{H}_f(\underset{\sim}{x}) &= \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} \\
&= 2\mathbf{I}_2,
\end{aligned}
$$

  which is positive definite, so $(1,0)^{\top}$ is a strict local minimizer.

- $f(\underset{\sim}{x}) = x_1^2 - x_2^2$ has the stationary point $\underset{\sim}{0}$ and Hessian $\begin{bmatrix} 2 & 0 \\ 0 & -2 \end{bmatrix}$, which is not positive definite ($(0,1)^{\top}$ gives -2). Indeed, $\underset{\sim}{0}$ is not a local minimizer (Example 0.3).

R

```
so_x_so <- seq(-1.5, 2.5, length.out = 101)
so_y_so <- seq(-1.5, 1.5, length.out = 101)
so_grid_so <- tidyr::expand_grid(x = so_x_so, y = so_y_so)
so_gx_so <- so_grid_so$x
so_gy_so <- so_grid_so$y
so_bowl_so <- so_gx_so^2 + so_gy_so^2 - 2 * so_gx_so
so_saddle_so <- so_gx_so^2 - so_gy_so^2
so_bowl_tbl_so <- dplyr::mutate(
  so_grid_so, z = so_bowl_so, panel = "positive definite Hessian"
)
so_saddle_tbl_so <- dplyr::mutate(
  so_grid_so, z = so_saddle_so, panel = "indefinite Hessian"
)
so_all_so <- dplyr::bind_rows(so_bowl_tbl_so, so_saddle_tbl_so)
so_order_so <- c("positive definite Hessian", "indefinite Hessian")
so_all_so$panel <- factor(so_all_so$panel, levels = so_order_so)
so_min_panel_so <- factor(so_order_so[1], levels = so_order_so)
so_sad_panel_so <- factor(so_order_so[2], levels = so_order_so)
so_min_so <- tibble::tibble(x = 1, y = 0, panel = so_min_panel_so)
so_sad_so <- tibble::tibble(x = 0, y = 0, panel = so_sad_panel_so)
so_aes_so <- ggplot2::aes(x, y)
ggplot2::ggplot(so_all_so, so_aes_so) +
  ggplot2::geom_contour(ggplot2::aes(z = z), colour = "grey70", bins = 12) +
  ggplot2::geom_point(data = so_min_so, colour = "#0072B2", size = 3) +
  ggplot2::geom_point(
    data = so_sad_so, colour = "#D55E00", size = 3, shape = 17
  ) +
  ggplot2::facet_wrap(~panel) +
  ggplot2::coord_equal() +
  ggplot2::labs(x = "first coordinate", y = "second coordinate")
```



Python

```
import matplotlib.pyplot as plt
```

R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

so_x_3d = np.linspace(-1.5, 2.5, 101)
so_y_3d = np.linspace(-1.5, 1.5, 101)
so_xx_3d, so_yy_3d = np.meshgrid(so_x_3d, so_y_3d)
so_bowl_3d = so_xx_3d**2 + so_yy_3d**2 - 2 * so_xx_3d
so_saddle_3d = so_xx_3d**2 - so_yy_3d**2
fig = plt.figure(figsize=(7, 3.5))
ax_bowl = fig.add_subplot(1, 2, 1, projection="3d")
ax_bowl.plot_surface(so_xx_3d, so_yy_3d, so_bowl_3d, color="#0072B2", alpha=0.6)
ax_bowl.scatter([1], [0], [-1], marker="o", color="#0072B2", s=50)
ax_bowl.set_title("positive definite Hessian")
ax_saddle = fig.add_subplot(1, 2, 2, projection="3d")
ax_saddle.plot_surface(so_xx_3d, so_yy_3d, so_saddle_3d, color="#0072B2",
                       alpha=0.6)
ax_saddle.scatter([0], [0], [0], marker="D", color="#D55E00", s=50)
ax_saddle.set_title("indefinite Hessian")
for ax in (ax_bowl, ax_saddle):
    ax.set_xlabel("x")
    ax.set_ylabel("y")
    ax.set_zlabel("f")
plt.show();
```

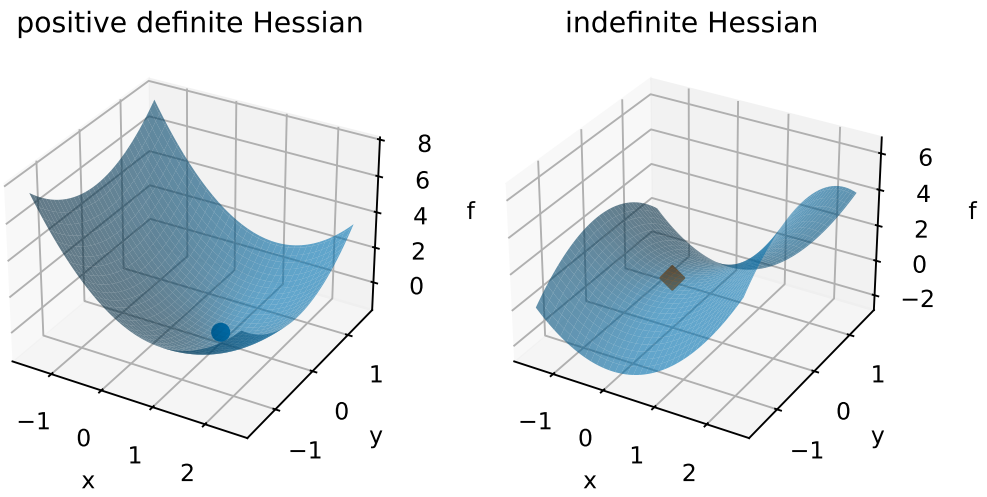


Figure 9: The graphs of the two functions in Example 0.10, the surfaces whose level curves are in Figure 8. The surface of $x_1^2 + x_2^2 - 2x_1$ is a bowl, and its stationary point $(1, 0)^\top$ (circle) is the bottom, a strict local minimizer. The surface of $x_1^2 - x_2^2$ is a saddle: it rises along the first coordinate axis and falls along the second, so the stationary point $\underset{\sim}{0}$ (diamond) is not a local minimizer.

Example 0.11 (The condition is sufficient, not necessary)**.** $f(\underset{\sim}{x}) = x_1^2 + x_2^4$ and $g(\underset{\sim}{x}) = x_1^2 - x_2^4$ both have gradient $\underset{\sim}{0}$ at $\underset{\sim}{0}$ and the same Hessian there, $\begin{bmatrix} 2 & 0 \\ 0 & 0 \end{bmatrix}$, since

$$\begin{aligned} \frac{\partial}{\partial x_2}\left(\frac{\partial}{\partial x_2}(\pm x_2^4)\right) &= \pm 12 x_2^2 \\ &= 0 \end{aligned}$$

at $x_2 = 0$. That Hessian is not positive definite ($(0,1)^\top$ gives 0), so Theorem 0.5 says nothing about either function. In fact $\underset{\sim}{0}$ is a strict local minimizer of f, since $f(\underset{\sim}{x}) > 0 = f(\underset{\sim}{0})$ for $\underset{\sim}{x} \neq \underset{\sim}{0}$, but not a local minimizer of g, since

$$\begin{aligned} g(0,t) &= -t^4 \\ &< 0 \\ &= g(\underset{\sim}{0}) \end{aligned}$$

for $t \neq 0$.

Definition 0.3 (Local maximizer)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$. A point $\underset{\sim}{x}^* \in \mathbb{R}^p$ is a **local maximizer** of f if there is a number $\delta > 0$ such that $f(\underset{\sim}{x}^*) \geq f(\underset{\sim}{x})$ for all $\underset{\sim}{x} \in \mathbb{R}^p$ with $\|\underset{\sim}{x} - \underset{\sim}{x}^*\| < \delta$.

Example 0.12 (A local maximizer)**.** Let $f(x) = 1 - x^2$. For every $x \in \mathbb{R}$,

$$\begin{aligned} f(0) - f(x) &= 1 - (1 - x^2) && \text{(substitute)} \\ &= x^2 && \text{(simplify)} \\ &\geq 0, && \text{(a square)} \end{aligned}$$

so $f(0) \geq f(x)$ for all x, and 0 is a local maximizer of f with any $\delta > 0$.

Definition 0.4 (Saddle point)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$. A stationary point of f that is neither a local minimizer nor a local maximizer of f is a **saddle point** of f [7].

Example 0.13 (A saddle point)**.** Let $f(\underset{\sim}{x}) = x_1^2 - x_2^2$. Its gradient is

$$\nabla f(\underset{\sim}{x}) = (2x_1,\ -2x_2)^\top.$$

At $\underset{\sim}{x} = \underset{\sim}{0}_{2\times 1}$, the gradient is $\nabla f(\underset{\sim}{0}_{2\times 1}) = \underset{\sim}{0}_{2\times 1}$, so $\underset{\sim}{0}_{2\times 1}$ is a stationary point.
For every $t \neq 0$,

$$
\begin{aligned}
f(t,0) &= t^2 && \text{(substitute)} \\
&> 0 && (t \neq 0) \\
&= f(\underset{\sim}{0}_{2\times 1}), && \text{(substitute)} \\
f(0,t) &= -t^2 && \text{(substitute)} \\
&< 0 && (t \neq 0) \\
&= f(\underset{\sim}{0}_{2\times 1}). && \text{(substitute)}
\end{aligned}
$$

So every ball around $\underset{\sim}{0}_{2\times 1}$ contains a point with a larger value of f and a point with a smaller one. Hence $\underset{\sim}{0}_{2\times 1}$ is neither a local minimizer nor a local maximizer, and it is a saddle point. Figure 10 shows the surface: the curve $t \mapsto f(t,0)$ rises from the origin along the x_1 axis, and the curve $t \mapsto f(0,t)$ falls from the origin along the x_2 axis.

R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

sp_t_3d = np.linspace(-1, 1, 61)
sp_x1_3d, sp_x2_3d = np.meshgrid(sp_t_3d, sp_t_3d)
sp_z_3d = sp_x1_3d**2 - sp_x2_3d**2
sp_zero_3d = np.zeros_like(sp_t_3d)
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(sp_x1_3d, sp_x2_3d, sp_z_3d, color="#0072B2", alpha=0.6)
ax.plot(sp_t_3d, sp_zero_3d, sp_t_3d**2, color="#D55E00",
        label="f(t, 0) = t^2")
ax.plot(sp_zero_3d, sp_t_3d, -sp_t_3d**2, "--", color="#D55E00",
        label="f(0, t) = -t^2")
ax.scatter([0], [0], [0], color="black", label="saddle point")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f(x)")
ax.legend(loc="upper right")
plt.show();
```

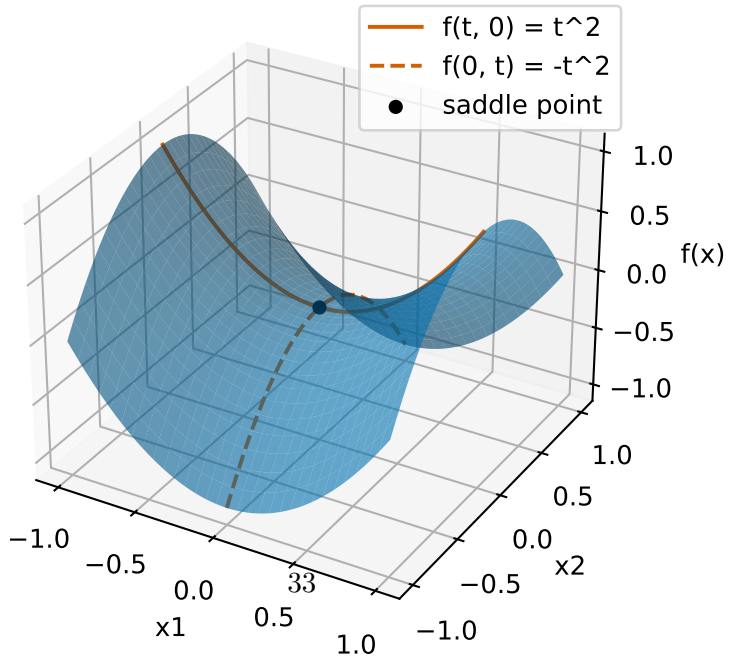


Figure 10: The graph of $f(x) = x_1^2 - x_2^2$. Along the x_1 axis (solid curve) the surface rises away from the origin, and along the x_2 axis (dashed curve) it falls away, so

Convexity in several variables

i Source

This section is adapted from the convexity part of Zhou [3], used under the MIT License (see the license text in Linear Algebra). The source states the first-order test (in one direction) and the second-derivative test for convexity without proof; here both directions of the first-order test are proved, and the second-derivative test is proved from the Taylor theorem of Section . The source defines both concave and strictly concave as "$-f$ is strictly convex"; here f is concave when $-f$ is convex. The source states that an intersection of convex sets is convex and that a maximum of convex functions is convex for any finite number of them; here each is stated for two and the general case follows by induction. These parts of the source are not part of this section:

- its catalogs of convex sets and convex functions, including the positive semidefinite matrices and $-\log\det$
- intersections of infinitely many convex sets, and suprema of infinitely many convex functions
- its examples, exercises and figures, including the multivariate normal maximum likelihood example

Definition 0.1 (Convex set)**.** A set $K \subseteq \mathbb{R}^p$ is **convex** if [8] for all $\underset{\sim}{x}, \underset{\sim}{y} \in K$ and all $t \in [0,1]$, the point $t\underset{\sim}{x} + (1-t)\underset{\sim}{y}$ is in K: K contains the whole line segment between any two of its points.

Example 0.1 (Convex sets)**.**

- $\mathbb{R}^p$ is convex, and so is any set with at most one point.
- The closed ball $B = \{\underset{\sim}{x} : \|\underset{\sim}{x}\| \le r\}$ is convex: for $\underset{\sim}{x}, \underset{\sim}{y} \in B$ and $t \in [0,1]$,

$$\begin{aligned} \|t\underset{\sim}{x} + (1-t)\underset{\sim}{y}\| &\le \|t\underset{\sim}{x}\| + \|(1-t)\underset{\sim}{y}\| && \text{(triangle inequality)} \\ &= t\,\|\underset{\sim}{x}\| + (1-t)\,\|\underset{\sim}{y}\| && \text{(norm of a multiple; } t, 1-t \ge 0) \\ &\le t\,r + (1-t)\,r && (\underset{\sim}{x}, \underset{\sim}{y} \in B) \\ &= r. && \text{(arithmetic)} \end{aligned}$$

  The first step is the triangle inequality, and the second uses the norm of a multiple.

Example 0.2 (A set that is not convex)**.** $K = \{x \in \mathbb{R} : |x| \geq 1\}$ is not convex: -1 and 1 are in K, but with $t = \frac{1}{2}$ the point $\frac{1}{2}(-1) + \frac{1}{2}(1) = 0$ is not.

Definition 0.2 (Half-space)**.** Let $\underset{\sim}{a} \in \mathbb{R}^p$ be a nonzero vector and c a real number. The set $\{\underset{\sim}{x} \in \mathbb{R}^p : \underset{\sim}{a} \cdot \underset{\sim}{x} \geq c\}$ is a (closed) **half-space**: the points on one side of the boundary $\{\underset{\sim}{x} : \underset{\sim}{a} \cdot \underset{\sim}{x} = c\}$, together with the boundary. In $\mathbb{R}^2$, a half-space is called a **half-plane**.

Example 0.3 (The right half-plane)**.** With $\underset{\sim}{a} = (1, 0)^\top$ and $c = 0$, the half-plane $\{\underset{\sim}{x} \in \mathbb{R}^2 : x_1 \geq 0\}$ is the set of points on or to the right of the x_2-axis. $(1, 5)^\top$ is in it, since $1 \geq 0$; $(0, -3)^\top$ is in it, on the boundary; and $(-1, 0)^\top$ is not, since $-1 < 0$.

Theorem 0.1 (An intersection of convex sets is convex)**.** *If K_1 and K_2 are convex subsets of $\mathbb{R}^p$ (Definition 0.1), then $K_1 \cap K_2$ is convex.*

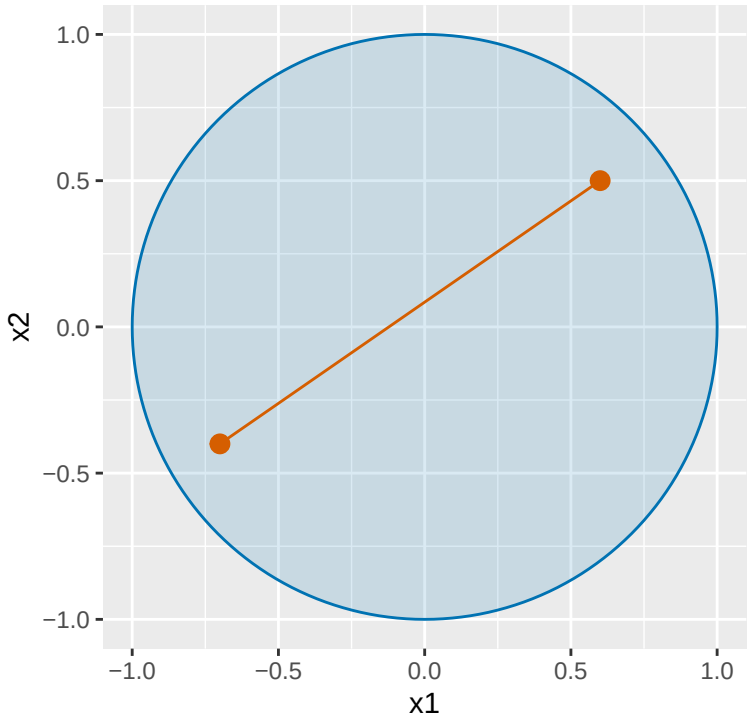
Proof

Proof. Let $\underset{\sim}{x}, \underset{\sim}{y} \in K_1 \cap K_2$ and $t \in [0, 1]$. Then $\underset{\sim}{x}, \underset{\sim}{y} \in K_1$, so $t\underset{\sim}{x} + (1-t)\underset{\sim}{y} \in K_1$, since K_1 is convex; in the same way $t\underset{\sim}{x} + (1-t)\underset{\sim}{y} \in K_2$. So $t\underset{\sim}{x} + (1-t)\underset{\sim}{y} \in K_1 \cap K_2$. □

Example 0.4 (Intersecting a ball with a half-plane)**.** The closed unit ball in $\mathbb{R}^2$ is convex (Example 0.1), and so is the half-plane (Definition 0.2) $\{\underset{\sim}{x} : x_1 \geq 0\}$: if $x_1 \geq 0$ and $y_1 \geq 0$, then $tx_1 + (1-t)y_1 \geq 0$ for $t \in [0, 1]$. By Theorem 0.1, the closed half-disk $\{\underset{\sim}{x} : \|\underset{\sim}{x}\| \leq 1,\ x_1 \geq 0\}$ is convex. A union need not be convex: $\{x \in \mathbb{R} : x \leq -1\}$ and $\{x \in \mathbb{R} : x \geq 1\}$ are convex, but their union is the set of Example 0.2.
Figure 12 shows the half-disk.

R

```r
cs_angle_fc <- seq(0, 2 * pi, length.out = 201)
cs_cos_fc <- cos(cs_angle_fc)
cs_sin_fc <- sin(cs_angle_fc)
cs_disk_fc <- tibble::tibble(x1 = cs_cos_fc, x2 = cs_sin_fc)
cs_points_x1_fc <- c(-0.7, 0.6)
cs_points_x2_fc <- c(-0.4, 0.5)
cs_points_fc <- tibble::tibble(x1 = cs_points_x1_fc, x2 = cs_points_x2_fc)
cs_segment_fc <- tibble::tibble(
  x1 = -0.7, x2 = -0.4, x1_end = 0.6, x2_end = 0.5
)
cs_aes_fc <- ggplot2::aes(x1, x2)
cs_seg_aes_fc <- ggplot2::aes(xend = x1_end, yend = x2_end)
ggplot2::ggplot(cs_disk_fc, cs_aes_fc) +
  ggplot2::geom_polygon(fill = "#0072B2", alpha = 0.15, colour = "#0072B2") +
  ggplot2::geom_segment(
    data = cs_segment_fc, mapping = cs_seg_aes_fc, colour = "#D55E00"
  ) +
  ggplot2::geom_point(data = cs_points_fc, colour = "#D55E00", size = 3) +
  ggplot2::coord_fixed() +
  ggplot2::labs(x = "x1", y = "x2")
```



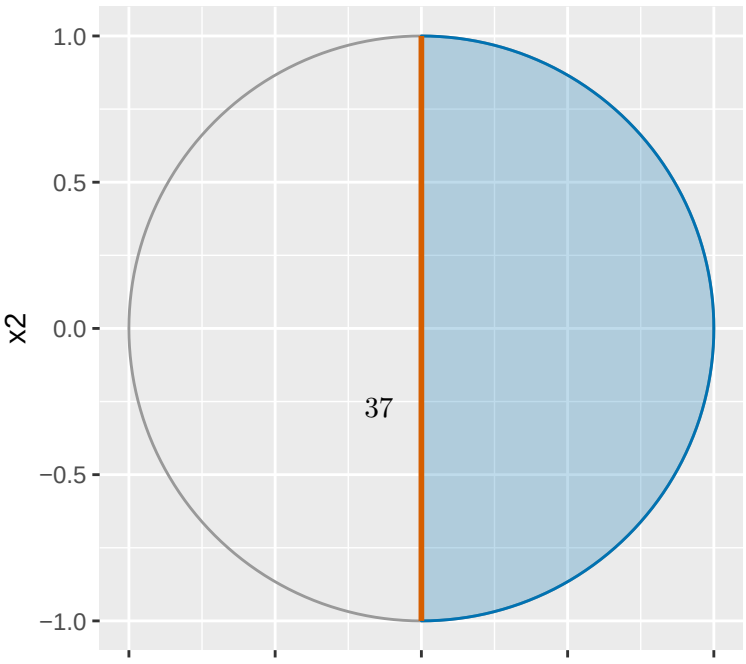
Python


```python
import matplotlib.pyplot as plt
import numpy as np

cs_angle_fc = np.linspace(0, 2 * np.pi, 201)
cs_x1_fc = np.array([-0.7, 0.6])
cs_x2_fc = np.array([-0.4, 0.5])
```

R

```
ci_angle_ci <- seq(-pi / 2, pi / 2, length.out = 101)
ci_half_x1_ci <- cos(ci_angle_ci)
ci_half_x2_ci <- sin(ci_angle_ci)
ci_half_ci <- tibble::tibble(x1 = ci_half_x1_ci, x2 = ci_half_x2_ci)
ci_full_angle_ci <- seq(0, 2 * pi, length.out = 201)
ci_full_x1_ci <- cos(ci_full_angle_ci)
ci_full_x2_ci <- sin(ci_full_angle_ci)
ci_full_ci <- tibble::tibble(x1 = ci_full_x1_ci, x2 = ci_full_x2_ci)
ci_edge_ci <- tibble::tibble(x1 = 0, x2 = -1, x1_end = 0, x2_end = 1)
ci_aes_ci <- ggplot2::aes(x1, x2)
ci_seg_aes_ci <- ggplot2::aes(xend = x1_end, yend = x2_end)
ci_full_ci |>
  ggplot2::ggplot() +
  ci_aes_ci +
  ggplot2::geom_path(colour = "grey60") +
  ggplot2::geom_polygon(
    data = ci_half_ci, fill = "#0072B2", alpha = 0.25, colour = "#0072B2"
  ) +
  ggplot2::geom_segment(
    data = ci_edge_ci, mapping = ci_seg_aes_ci,
    colour = "#D55E00", linewidth = 1
  ) +
  ggplot2::coord_fixed() +
  ggplot2::labs(x = "x1", y = "x2")
```



Definition 0.3 (Strictly convex function)**.** A function $f : \mathbb{R}^p \to \mathbb{R}$ is **strictly convex** if $f(t\underset{\sim}{x} + (1-t)\underset{\sim}{y}) < tf(\underset{\sim}{x}) + (1-t)f(\underset{\sim}{y})$ for all $\underset{\sim}{x} \neq \underset{\sim}{y}$ in $\mathbb{R}^p$ and all $t \in (0,1)$.

Definition 0.4 (Concave function)**.** A function $f : \mathbb{R}^p \to \mathbb{R}$ is **concave** if $-f$ is convex.

Definition 0.5 (Strictly concave function)**.** A function $f : \mathbb{R}^p \to \mathbb{R}$ is **strictly concave** if $-f$ is strictly convex (Definition 0.3).

Example 0.5 (Strictly convex, and convex but not strictly)**.**

- $f(x) = x^2$ is strictly convex:

$$\begin{aligned} tx^2 + (1-t)y^2 - (tx + (1-t)y)^2 &= tx^2 + (1-t)y^2 - t^2x^2 - 2t(1-t)xy - (1-t)^2y^2 && \text{(expand the squa} \\ &= t(1-t)x^2 - 2t(1-t)xy + t(1-t)y^2 && (t - t^2 = t(1-t) \\ &= t(1-t)(x-y)^2, && \text{(factor)} \end{aligned}$$

  which is positive when $x \neq y$ and $t \in (0,1)$. At $t \in \{0,1\}$ or $x = y$ it is 0, so x^2 is also convex.

- $f(x) = 2x + 1$ is convex but not strictly convex:

$$\begin{aligned} f(tx + (1-t)y) &= 2tx + 2(1-t)y + 1 \\ &= tf(x) + (1-t)f(y) \end{aligned}$$

  for every x, y, t, so the inequality holds, but never strictly. It is also concave.

Theorem 0.2 (The maximum of convex functions is convex)**.** *If f_1 and f_2 are convex functions on $\mathbb{R}^p$, then $f(\underset{\sim}{x}) \overset{def}{=} \max\{f_1(\underset{\sim}{x}), f_2(\underset{\sim}{x})\}$ is convex.*

Proof

Proof. Let $\underset{\sim}{x}, \underset{\sim}{y} \in \mathbb{R}^p$ and $t \in [0,1]$. For $i = 1, 2$,

$$\begin{aligned} f_i(t\underset{\sim}{x} + (1-t)\underset{\sim}{y}) &\leq tf_i(\underset{\sim}{x}) + (1-t)f_i(\underset{\sim}{y}) && (f_i \text{ is convex}) \\ &\leq tf(\underset{\sim}{x}) + (1-t)f(\underset{\sim}{y}). && (f_i \leq f, \text{ and } t, 1-t \geq 0) \end{aligned}$$

Both f_1 and f_2 at $t\underset{\sim}{x} + (1-t)\underset{\sim}{y}$ are at most $tf(\underset{\sim}{x}) + (1-t)f(\underset{\sim}{y})$, so their maximum $f(t\underset{\sim}{x} + (1-t)\underset{\sim}{y})$ is too. □

Example 0.6 (A maximum is convex; a minimum need not be)**.**

- $f_1(x) = x$ is convex, by the computation for $2x + 1$ in Example 0.5 with 2 and 1 replaced by 1 and 0, and $f_2(x) = x^2$ is convex by the same example, so $\max\{x, x^2\}$ is convex by Theorem 0.2.
- The minimum of two convex functions need not be convex: $g(x) = \min\{x^2, (x-2)^2\}$ has

$$\begin{aligned} g(0) &= g(2) \\ &= 0, \end{aligned}$$

but at the midpoint

$$\begin{aligned} g(1) &= \min\{1, 1\} \\ &= 1 \\ &> \tfrac{1}{2}g(0) + \tfrac{1}{2}g(2) \\ &= 0. \end{aligned}$$

Definition 0.6 (Epigraph)**.** The **epigraph** [9] of $f : \mathbb{R}^p \to \mathbb{R}$ is the set of points on or over its graph,

$$\operatorname{epi} f \stackrel{\text{def}}{=} \{(\underset{\sim}{x}, s) : \underset{\sim}{x} \in \mathbb{R}^p,\ s \in \mathbb{R},\ f(\underset{\sim}{x}) \le s\}.$$

A pair $(\underset{\sim}{x}, s)$ is treated as the $(p+1)$-vector with entries $x_1, \ldots, x_p, s$, so $\operatorname{epi} f \subseteq \mathbb{R}^{p+1}$, and sums and multiples of pairs are taken entry by entry.

Example 0.7 (The epigraph of x^2)**.** For $f(x) = x^2$, $\operatorname{epi} f = \{(x, s) : x^2 \le s\}$, the points (x, s) on the parabola $s = x^2$ or on the side of it where s is larger. $(1, 2)$ is in it, since $1^2 \le 2$; $(2, 1)$ is not, since $2^2 > 1$.

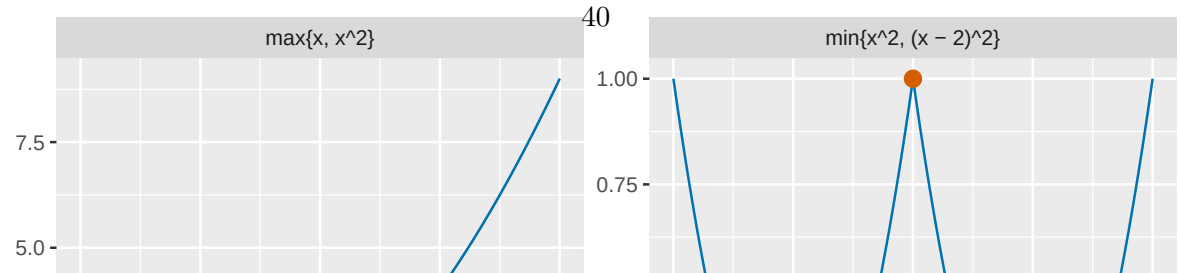
Theorem 0.3 (Convexity through the epigraph)**.** *$f : \mathbb{R}^p \to \mathbb{R}$ is convex if and only if its epigraph (Definition 0.6) is convex (Definition 0.1).*

Proof

Proof. **If.** Let $\underset{\sim}{x}, \underset{\sim}{y} \in \mathbb{R}^p$ and $t \in [0, 1]$. The points $(\underset{\sim}{x}, f(\underset{\sim}{x}))$ and $(\underset{\sim}{y}, f(\underset{\sim}{y}))$ are in $\operatorname{epi} f$, since $f(\underset{\sim}{x}) \le f(\underset{\sim}{x})$ and $f(\underset{\sim}{y}) \le f(\underset{\sim}{y})$. Since $\operatorname{epi} f$ is convex, it contains

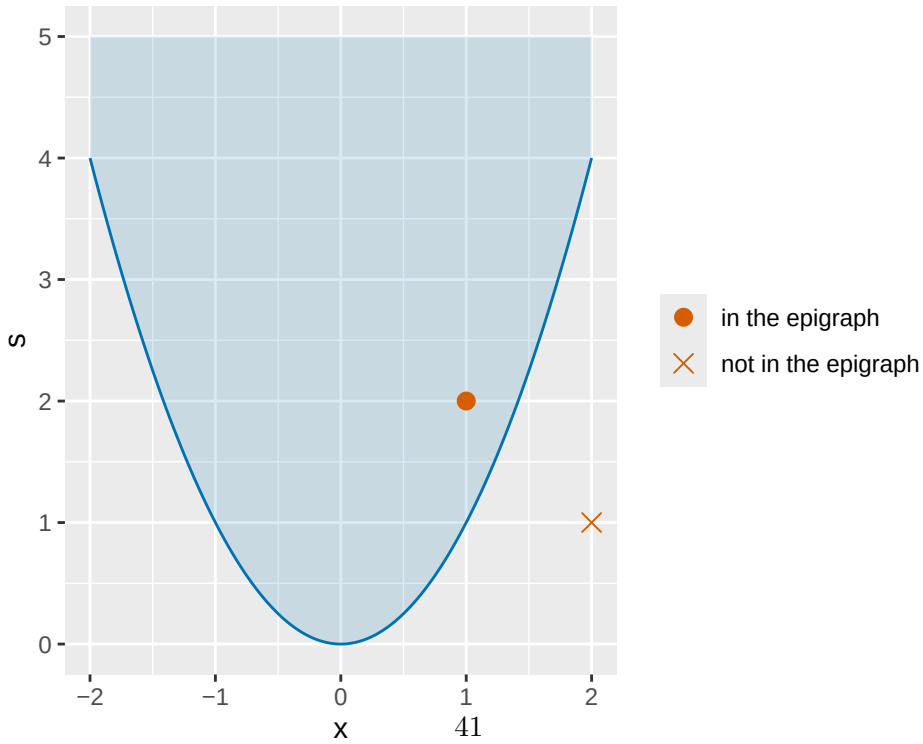
R

```
cm_x_fm <- seq(-1, 3, length.out = 201)
cm_levels_fm <- c("max{x, x^2}", "min{x^2, (x - 2)^2}")
cm_max_value_fm <- pmax(cm_x_fm, cm_x_fm^2)
cm_min_value_fm <- pmin(cm_x_fm^2, (cm_x_fm - 2)^2)
cm_max_fm <- tibble::tibble(
  x = cm_x_fm, value = cm_max_value_fm, panel = cm_levels_fm[1]
)
cm_min_fm <- tibble::tibble(
  x = cm_x_fm, value = cm_min_value_fm, panel = cm_levels_fm[2]
)
cm_main_fm <- dplyr::bind_rows(cm_max_fm, cm_min_fm)
cm_main_fm$panel <- factor(cm_main_fm$panel, levels = cm_levels_fm)
cm_square_value_fm <- cm_x_fm^2
cm_line_fm <- tibble::tibble(
  x = cm_x_fm, value = cm_x_fm, panel = cm_levels_fm[1], part = "x"
)
cm_square_fm <- tibble::tibble(
  x = cm_x_fm, value = cm_square_value_fm, panel = cm_levels_fm[1],
  part = "x^2"
)
cm_parts_fm <- dplyr::bind_rows(cm_line_fm, cm_square_fm)
cm_parts_fm$panel <- factor(cm_parts_fm$panel, levels = cm_levels_fm)
cm_chord_fm <- tibble::tibble(
  x = 0, xend = 2, value = 0, panel = cm_levels_fm[2]
)
cm_chord_fm$panel <- factor(cm_chord_fm$panel, levels = cm_levels_fm)
cm_mid_fm <- tibble::tibble(x = 1, value = 1, panel = cm_levels_fm[2])
cm_mid_fm$panel <- factor(cm_mid_fm$panel, levels = cm_levels_fm)
cm_aes_fm <- ggplot2::aes(x, value)
cm_parts_aes_fm <- ggplot2::aes(group = part)
cm_chord_aes_fm <- ggplot2::aes(xend = xend, yend = value)
ggplot2::ggplot(cm_main_fm, cm_aes_fm) +
  ggplot2::geom_line(
    data = cm_parts_fm, mapping = cm_parts_aes_fm,
    colour = "grey60", linewidth = 0.4
  ) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_segment(
    data = cm_chord_fm, mapping = cm_chord_aes_fm, colour = "#D55E00"
  ) +
  ggplot2::geom_point(data = cm_mid_fm, colour = "#D55E00", size = 3) +
  ggplot2::facet_wrap(~panel, scales = "free_y") +
  ggplot2::labs(x = "x", y = NULL)
```



R

```r
ep_x_fe <- seq(-2, 2, length.out = 201)
ep_curve_fe <- tibble::tibble(x = ep_x_fe, s = ep_x_fe^2, top = 5)
ep_marks_x_fe <- c(1, 2)
ep_marks_s_fe <- c(2, 1)
ep_marks_kind_fe <- c("in the epigraph", "not in the epigraph")
ep_marks_fe <- tibble::tibble(
  x = ep_marks_x_fe, s = ep_marks_s_fe, kind = ep_marks_kind_fe
)
ep_aes_fe <- ggplot2::aes(x, s)
ep_ribbon_aes_fe <- ggplot2::aes(ymin = s, ymax = top)
ep_marks_aes_fe <- ggplot2::aes(shape = kind)
ep_shapes_fe <- c("in the epigraph" = 16, "not in the epigraph" = 4)
ggplot2::ggplot(ep_curve_fe, ep_aes_fe) +
  ggplot2::geom_ribbon(
    mapping = ep_ribbon_aes_fe, fill = "#0072B2", alpha = 0.15
  ) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(
    data = ep_marks_fe, mapping = ep_marks_aes_fe,
    colour = "#D55E00", size = 3
  ) +
  ggplot2::scale_shape_manual(values = ep_shapes_fe) +
  ggplot2::labs(x = "x", y = "s", shape = NULL)
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np
```

$$t\,(\underset{\sim}{x}, f(\underset{\sim}{x})) + (1-t)\,(\underset{\sim}{y}, f(\underset{\sim}{y})) = (t\underset{\sim}{x} + (1-t)\underset{\sim}{y},\ tf(\underset{\sim}{x}) + (1-t)f(\underset{\sim}{y})),$$

and by Definition 0.6 that membership means $f(t\underset{\sim}{x} + (1-t)\underset{\sim}{y}) \le tf(\underset{\sim}{x}) + (1-t)f(\underset{\sim}{y})$.
Only if. Let $(\underset{\sim}{x}, s)$ and $(\underset{\sim}{y}, u)$ be in $\operatorname{epi} f$, so $f(\underset{\sim}{x}) \le s$ and $f(\underset{\sim}{y}) \le u$, and let $t \in [0,1]$. Then

$$\begin{aligned} f(t\underset{\sim}{x} + (1-t)\underset{\sim}{y}) &\le tf(\underset{\sim}{x}) + (1-t)f(\underset{\sim}{y}) && (f \text{ is convex}) \\ &\le ts + (1-t)u, && (f(\underset{\sim}{x}) \le s,\ f(\underset{\sim}{y}) \le u, \text{ and } t, 1-t \ge 0) \end{aligned}$$

so $t\,(\underset{\sim}{x}, s) + (1-t)\,(\underset{\sim}{y}, u) = (t\underset{\sim}{x} + (1-t)\underset{\sim}{y},\ ts + (1-t)u)$ is in $\operatorname{epi} f$. □

Example 0.8 (Two epigraphs)**.**

- The epigraph of $f(x) = |x|$ is $\{(x,s) : |x| \le s\}$, a wedge. It is the intersection of the half-planes $\{(x,s) : s - x \ge 0\}$ and $\{(x,s) : s + x \ge 0\}$. Each half-plane is convex: if $s_1 - x_1 \ge 0$ and $s_2 - x_2 \ge 0$, then $(ts_1 + (1-t)s_2) - (tx_1 + (1-t)x_2) = t(s_1 - x_1) + (1-t)(s_2 - x_2) \ge 0$, and the same with $+$ for the other. So the wedge is convex (Theorem 0.1), and $|x|$ is convex by Theorem 0.3.
- The epigraph of $g(x) = -x^2$ is not convex: $(-1,-1)$ and $(1,-1)$ are in it, since $g(\pm 1) = -1$, but their midpoint $(0,-1)$ is not, since $g(0) = 0 > -1$. So $-x^2$ is not convex.

Definition 0.7 (Tangent plane)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have a gradient at $\underset{\sim}{y}$. The **tangent plane** [10] to the graph of f at $\underset{\sim}{y}$ is the graph of the function

$$\underset{\sim}{x} \mapsto f(\underset{\sim}{y}) + \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y})\right)^{\top} (\underset{\sim}{x} - \underset{\sim}{y}),$$

which takes the value $f(\underset{\sim}{y})$ at $\underset{\sim}{x} = \underset{\sim}{y}$ and has the same gradient as f there.

Example 0.9 (A tangent line and a tangent plane)**.**

- For $f(x) = x^2$ at $y = 1$, $f(1) = 1$ and $f'(1) = 2$, so the tangent line is

$$\begin{aligned} s &= 1 + 2\,(x-1) && (\text{substitute } f(1) \text{ and } f'(1)) \\ &= 1 + 2x - 2 && (\text{distribute}) \\ &= 2x - 1. && (1 - 2 = -1) \end{aligned}$$

- For $f(\underset{\sim}{x}) = x_1^2 + x_2^2$ at $\underset{\sim}{y} = (1,1)^{\top}$, $f(\underset{\sim}{y}) = 2$ and the gradient is $\nabla f(\underset{\sim}{y}) = (2y_1,\ 2y_2)^{\top} = (2,2)^{\top}$, so the tangent plane is

$$
\begin{aligned}
s &= 2 + 2\,(x_1 - 1) + 2\,(x_2 - 1) && \text{(substitute } f(\underset{\sim}{y}) \text{ and the gradient)} \\
&= 2 + 2x_1 - 2 + 2x_2 - 2 && \text{(distribute)} \\
&= 2x_1 + 2x_2 - 2. && (2 - 2 - 2 = -2)
\end{aligned}
$$

Figure 15 shows the tangent plane of the second bullet.

R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

tp_x1_py, tp_x2_py = np.meshgrid(np.linspace(-1, 3, 41),
                                 np.linspace(-1, 3, 41))
tp_bowl_py = tp_x1_py**2 + tp_x2_py**2
tp_plane_py = 2 * tp_x1_py + 2 * tp_x2_py - 2
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(tp_x1_py, tp_x2_py, tp_bowl_py, color="#0072B2", alpha=0.6)
ax.plot_surface(tp_x1_py, tp_x2_py, tp_plane_py, color="#D55E00", alpha=0.4)
ax.scatter([1], [1], [2], color="black", label="point of tangency")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f(x)")
ax.legend(loc="upper right")
plt.show();
```

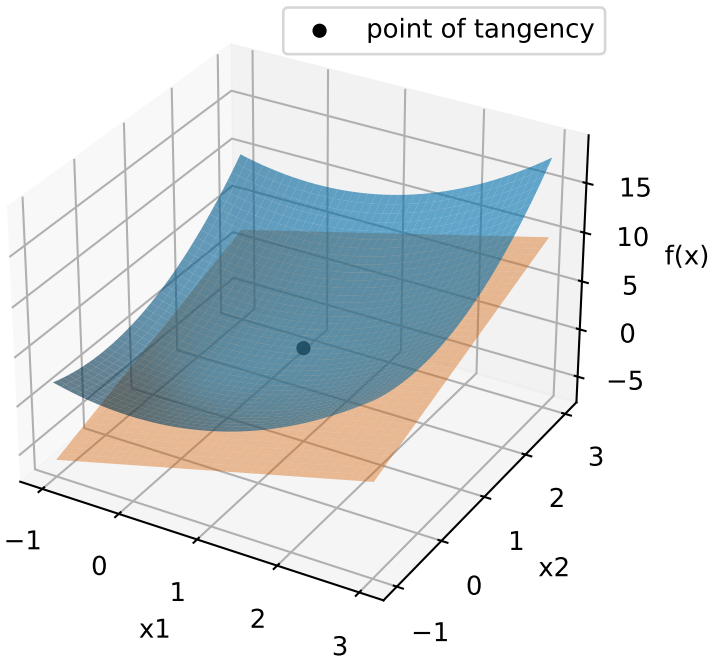


Figure 15: The tangent plane (vermilion) of the bowl $f(\underset{\sim}{x}) = x_1^2 + x_2^2$ of Example 0.9 touches it at $(1, 1)^\top$ and lies below it.

Remark (The tangent plane in one variable)

Remark 0.1 (The tangent plane in one variable). For $p = 1$ the tangent plane is the tangent line $s = f(y) + f'(y)\,(x - y)$, the line through $(y, f(y))$ with slope $f'(y)$.

Theorem 0.4 (First-order characterization of convexity)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives that are continuous on $\mathbb{R}^p$. Then f is convex if and only if*

$$f(\underset{\sim}{x}) \geq f(\underset{\sim}{y}) + \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y})\right)^{\top} (\underset{\sim}{x} - \underset{\sim}{y}) \qquad \text{for all } \underset{\sim}{x}, \underset{\sim}{y} \in \mathbb{R}^p : \tag{1}$$

every tangent plane (Definition 0.7) lies on or under the graph. If the inequality is strict whenever $\underset{\sim}{x} \neq \underset{\sim}{y}$, then f is strictly convex (Definition 0.3).

Proof

Proof. **Only if.** Let $\underset{\sim}{x}, \underset{\sim}{y} \in \mathbb{R}^p$, let $\underset{\sim}{d} \overset{\text{def}}{=} \underset{\sim}{x} - \underset{\sim}{y}$, and let $g(t) \overset{\text{def}}{=} f(\underset{\sim}{y} + t\underset{\sim}{d})$. The inner function $t \mapsto \underset{\sim}{y} + t\underset{\sim}{d}$ has derivative $\underset{\sim}{d}^{\top}$, so the vector chain rule gives

$$\begin{aligned} g'(0) &= \underset{\sim}{d}^{\top} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y}) && \text{(vector chain rule)} \\ &= \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y})\right)^{\top} \underset{\sim}{d}. && \text{(a } 1 \times 1 \text{ matrix equals its transpose)} \end{aligned}$$

For $t \in (0, 1]$, $\underset{\sim}{y} + t\underset{\sim}{d} = t\underset{\sim}{x} + (1 - t)\underset{\sim}{y}$, so

$$\begin{aligned} g(t) - g(0) &= f(t\underset{\sim}{x} + (1 - t)\underset{\sim}{y}) - f(\underset{\sim}{y}) && \text{(definition of } g) \\ &\leq t f(\underset{\sim}{x}) + (1 - t) f(\underset{\sim}{y}) - f(\underset{\sim}{y}) && (f \text{ is convex}) \\ &= t\,(f(\underset{\sim}{x}) - f(\underset{\sim}{y})), && \text{(collect terms)} \end{aligned}$$

and dividing by $t > 0$, $\dfrac{g(t) - g(0)}{t} \leq f(\underset{\sim}{x}) - f(\underset{\sim}{y})$. As $t \to 0$ from the right, the left side tends to $g'(0)$ (g is differentiable at 0, so the one-sided limit equals the two-sided derivative), and a limit of numbers that are all at most $f(\underset{\sim}{x}) - f(\underset{\sim}{y})$ is at most $f(\underset{\sim}{x}) - f(\underset{\sim}{y})$. So $\left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y})\right)^{\top} (\underset{\sim}{x} - \underset{\sim}{y}) \leq f(\underset{\sim}{x}) - f(\underset{\sim}{y})$, which rearranges to Equation 1.

If. Let $\underset{\sim}{x}, \underset{\sim}{y} \in \mathbb{R}^p$, $t \in [0, 1]$, $\underset{\sim}{z} \overset{\text{def}}{=} t\underset{\sim}{x} + (1 - t)\underset{\sim}{y}$, and $\underset{\sim}{g} \overset{\text{def}}{=} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{z})$. Applying Equation 1 at the point $\underset{\sim}{z}$, once toward $\underset{\sim}{x}$ and once toward $\underset{\sim}{y}$,

$$f(\underset{\sim}{x}) \geq f(\underset{\sim}{z}) + \underset{\sim}{g}^{\top} (\underset{\sim}{x} - \underset{\sim}{z}), \qquad f(\underset{\sim}{y}) \geq f(\underset{\sim}{z}) + \underset{\sim}{g}^{\top} (\underset{\sim}{y} - \underset{\sim}{z}).$$

Multiplying the first by $t \geq 0$ and the second by $1 - t \geq 0$ and adding,

$$
\begin{aligned}
t f(\underset{\sim}{x}) + (1-t) f(\underset{\sim}{y}) &\geq t(f(\underset{\sim}{z}) + \underset{\sim}{g}^\top(\underset{\sim}{x} - \underset{\sim}{z})) + (1-t)(f(\underset{\sim}{z}) + \underset{\sim}{g}^\top(\underset{\sim}{y} - \underset{\sim}{z})) && \text{(multiply by } t, 1-t \geq 0 \text{ and add)} \\
&= (t + (1-t))\, f(\underset{\sim}{z}) + t\, \underset{\sim}{g}^\top(\underset{\sim}{x} - \underset{\sim}{z}) + (1-t)\, \underset{\sim}{g}^\top(\underset{\sim}{y} - \underset{\sim}{z}) && \text{(distribute } t \text{ and } 1-t) \\
&= f(\underset{\sim}{z}) + t\, \underset{\sim}{g}^\top(\underset{\sim}{x} - \underset{\sim}{z}) + (1-t)\, \underset{\sim}{g}^\top(\underset{\sim}{y} - \underset{\sim}{z}) && (t + (1-t) = 1) \\
&= f(\underset{\sim}{z}) + \underset{\sim}{g}^\top(t(\underset{\sim}{x} - \underset{\sim}{z}) + (1-t)(\underset{\sim}{y} - \underset{\sim}{z})) && \text{(factor out } \underset{\sim}{g}^\top\text{; matrix products dis} \\
&= f(\underset{\sim}{z}) + \underset{\sim}{g}^\top(t\underset{\sim}{x} + (1-t)\underset{\sim}{y} - \underset{\sim}{z}) && \text{(collect terms)} \\
&= f(\underset{\sim}{z}). && \text{(definition of } \underset{\sim}{z})
\end{aligned}
$$

That inequality is the defining inequality of a convex function. If Equation 1 is strict for distinct points, and $\underset{\sim}{x} \neq \underset{\sim}{y}$ and $t \in (0,1)$, then:

- $\underset{\sim}{z}$ differs from both $\underset{\sim}{x}$ and $\underset{\sim}{y}$, since $\underset{\sim}{x} - \underset{\sim}{z} = (1-t)(\underset{\sim}{x} - \underset{\sim}{y}) \neq \underset{\sim}{0}$ and $\underset{\sim}{y} - \underset{\sim}{z} = t(\underset{\sim}{y} - \underset{\sim}{x}) \neq \underset{\sim}{0}$;
- so both inequalities at $\underset{\sim}{z}$ are strict;
- both multipliers t and $1-t$ are positive, so the first step of the display is strict, and f is strictly convex.

□

Example 0.10 (Tangent lines under a parabola)**.** For $f(x) = x^2$, the right side of Equation 1 is $y^2 + 2y(x-y) = 2xy - y^2$, and

$$
\begin{aligned}
f(x) - (2xy - y^2) &= x^2 - 2xy + y^2 \\
&= (x-y)^2 \\
&\geq 0,
\end{aligned}
$$

with equality only at $x = y$, confirming that x^2 is strictly convex. For $g(x) = -x^2$ the inequality fails: at $y = 0$ the tangent line is $s = 0$, and $g(1) = -1 < 0$.

Theorem 0.5 (Second-derivative test for convexity)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$ have first and second partial derivatives that are continuous on $\mathbb{R}^p$.*

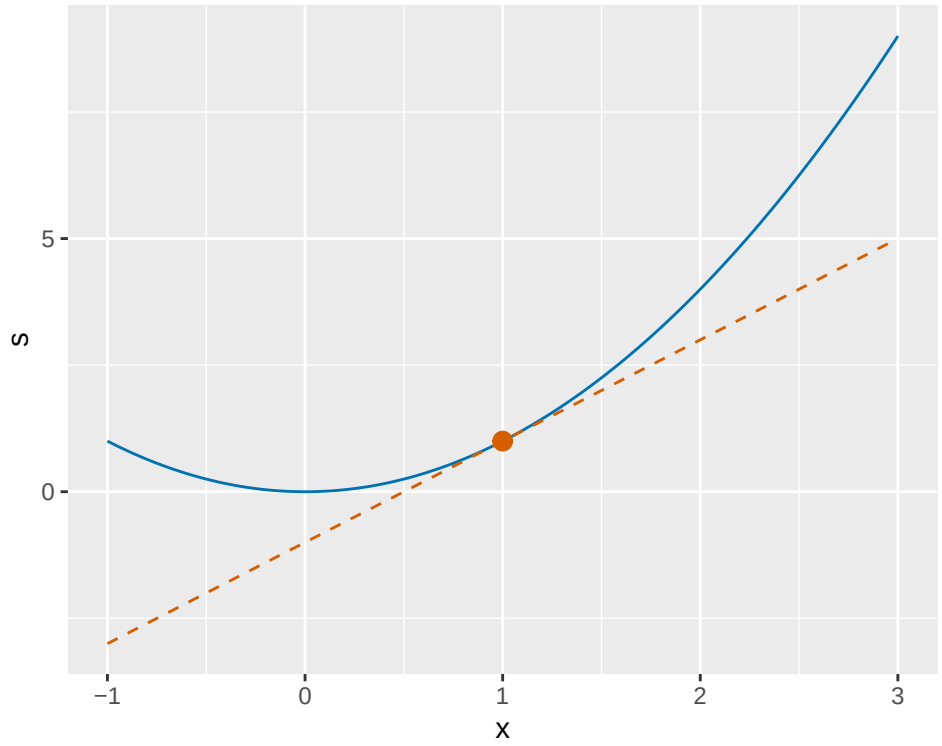
1. *If $\mathbf{H}_f(\underset{\sim}{x})$ is positive semidefinite for every $\underset{\sim}{x}$, then f is convex.*
2. *If $\mathbf{H}_f(\underset{\sim}{x})$ is positive definite for every $\underset{\sim}{x}$, then f is strictly convex (Definition 0.3).*

Proof

Proof. Let $\underset{\sim}{x}, \underset{\sim}{y} \in \mathbb{R}^p$. By Theorem 0.4 with $\underset{\sim}{z} = \underset{\sim}{y}$ and $\underset{\sim}{h} = \underset{\sim}{x} - \underset{\sim}{y}$, there is a $\tau \in (0,1)$ such that, with $\underset{\sim}{w} \overset{\text{def}}{=} \underset{\sim}{y} + \tau(\underset{\sim}{x} - \underset{\sim}{y})$,

R

```r
tl_x_ft <- seq(-1, 3, length.out = 201)
tl_curve_ft <- tibble::tibble(x = tl_x_ft, s = tl_x_ft^2)
tl_line_ft <- tibble::tibble(x = tl_x_ft, s = 2 * tl_x_ft - 1)
tl_point_ft <- tibble::tibble(x = 1, s = 1)
tl_aes_ft <- ggplot2::aes(x, s)
ggplot2::ggplot(tl_curve_ft, tl_aes_ft) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_line(
    data = tl_line_ft, colour = "#D55E00", linetype = "dashed"
  ) +
  ggplot2::geom_point(data = tl_point_ft, colour = "#D55E00", size = 3) +
  ggplot2::labs(x = "x", y = "s")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

tl_x_ft = np.linspace(-1, 3, 201)
fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
ax.plot(tl_x_ft, tl_x_ft**2, color="#0072B2", label="f(x) = x^2")
ax.plot(tl_x_ft, 2 * tl_x_ft - 1, "--", color="#D55E00",
        label="tangent line at 1")
ax.plot(1, 1, "o", color="#D55E00", markersize=8)
ax.set_xlabel("x")
ax.set_ylabel("s")
ax.legend(loc="upper center")
plt.show()
```

$$f(\underset{\sim}{x}) = f(\underset{\sim}{y}) + \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y})\right)^{\top} (\underset{\sim}{x} - \underset{\sim}{y}) + \frac{1}{2} (\underset{\sim}{x} - \underset{\sim}{y})^{\top} \mathbf{H}_f(\underset{\sim}{w}) (\underset{\sim}{x} - \underset{\sim}{y}) \quad \text{(Theorem 0.4)}$$

$$\geq f(\underset{\sim}{y}) + \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{y})\right)^{\top} (\underset{\sim}{x} - \underset{\sim}{y}). \quad (\mathbf{H}_f(\underset{\sim}{w}) \text{ is positive semidefinite})$$

This inequality is Equation 1, so f is convex by Theorem 0.4. Under the hypothesis of part 2 and with $\underset{\sim}{x} \neq \underset{\sim}{y}$, the quadratic-form term is positive, so the inequality is strict, and f is strictly convex by the last sentence of Theorem 0.4. □

Example 0.11 (Using the Hessian to show convexity)**.**

- For a symmetric positive semidefinite $\mathbf{A}$, $f(\underset{\sim}{x}) = \underset{\sim}{x}^{\top} \mathbf{A} \underset{\sim}{x}$ has Hessian $2\mathbf{A}$ (by the Hessian of a quadratic form), which is positive semidefinite because $\underset{\sim}{h}^{\top}(2\mathbf{A})\underset{\sim}{h} = 2\,\underset{\sim}{h}^{\top}\mathbf{A}\underset{\sim}{h} \geq 0$; so f is convex.
- $f(\underset{\sim}{x}) = x_1^2 + x_2^2 - 2x_1$ has Hessian $2\mathbf{I}_2$ (Example 0.10), which is positive definite, so f is strictly convex.
- $f(\underset{\sim}{x}) = e^{2x_1+x_2} - x_1$ has Hessian $e^u \begin{bmatrix} 4 & 2 \\ 2 & 1 \end{bmatrix}$ with $u = 2x_1 + x_2$ (the Hessian example), and

$$\begin{aligned} \underset{\sim}{h}^{\top} \mathbf{H}_f(\underset{\sim}{x})\, \underset{\sim}{h} &= e^u(4h_1^2 + 4h_1h_2 + h_2^2) \\ &= e^u(2h_1 + h_2)^2 \\ &\geq 0, \end{aligned}$$

  so f is convex. The Hessian is not positive definite ($\underset{\sim}{h} = (1, -2)^{\top}$ gives 0), so part 2 does not apply, and in fact f is not strictly convex: along $\underset{\sim}{x} = t\,(1, -2)^{\top}$,

$$\begin{aligned} u &= 2t - 2t \\ &= 0 \end{aligned}$$

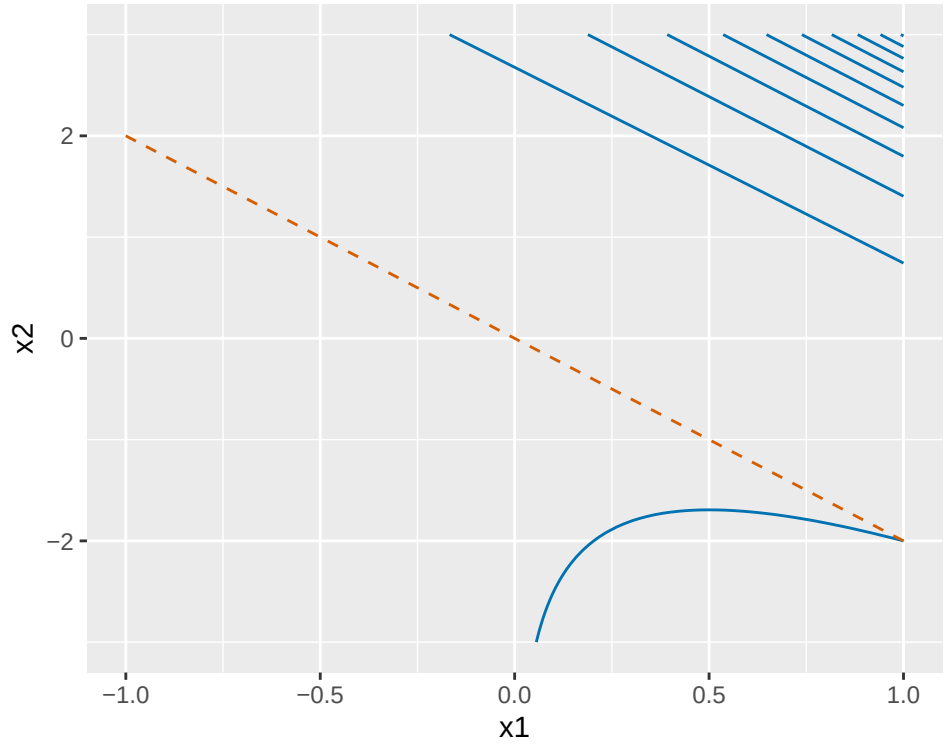
  and $f = 1 - t$ is linear in t, so the convexity inequality holds with equality there.

Example 0.12 (Strictly convex with a singular Hessian)**.** Part 2 is sufficient but not necessary. $f(x) = x^4$ has

$$\begin{aligned} f''(0) &= 12 \cdot 0^2 \\ &= 0, \end{aligned}$$

R

```r
ch_x1_ch <- seq(-1, 1, length.out = 101)
ch_x2_ch <- seq(-3, 3, length.out = 101)
ch_grid_ch <- tidyr::expand_grid(x1 = ch_x1_ch, x2 = ch_x2_ch)
ch_u_ch <- 2 * ch_grid_ch$x1 + ch_grid_ch$x2
ch_grid_ch$z <- exp(ch_u_ch) - ch_grid_ch$x1
ch_line_x1_ch <- c(-1, 1)
ch_line_x2_ch <- c(2, -2)
ch_line_ch <- tibble::tibble(x1 = ch_line_x1_ch, x2 = ch_line_x2_ch)
ch_aes_ch <- ggplot2::aes(x1, x2)
ch_z_aes_ch <- ggplot2::aes(z = z)
ggplot2::ggplot(ch_grid_ch, ch_aes_ch) +
  ggplot2::geom_contour(ch_z_aes_ch, colour = "#0072B2", bins = 12) +
  ggplot2::geom_line(
    data = ch_line_ch, colour = "#D55E00", linetype = "dashed"
  ) +
  ggplot2::labs(x = "x1", y = "x2")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np
```



```python
ch_x1_ax = np.linspace(-1, 1, 101)
ch_x2_ax = np.linspace(-3, 3, 101)
ch_xx_ax, ch_yy_ax = np.meshgrid(ch_x1_ax, ch_x2_ax)
ch_z_ax = np.exp(2 * ch_xx_ax + ch_yy_ax) - ch_xx_ax
fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
ax.contour(ch_xx_ax, ch_yy_ax, ch_z_ax, levels=12, colors="#0072B2")
ax.plot([-1, 1], [2, -2], "--", color="#D55E00")
```

R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

ch_x1_3d, ch_x2_3d = np.meshgrid(np.linspace(-1, 1, 101),
                                 np.linspace(-3, 3, 101))
ch_z_3d = np.exp(2 * ch_x1_3d + ch_x2_3d) - ch_x1_3d
ch_line_x1_3d = np.linspace(-1, 1, 21)
ch_line_x2_3d = -2 * ch_line_x1_3d
ch_line_z_3d = np.exp(2 * ch_line_x1_3d + ch_line_x2_3d) - ch_line_x1_3d
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(ch_x1_3d, ch_x2_3d, ch_z_3d, color="#0072B2", alpha=0.6)
ax.plot(ch_line_x1_3d, ch_line_x2_3d, ch_line_z_3d, "--", color="#D55E00",
        label="f is linear")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f(x)")
ax.legend(loc="upper right")
plt.show();
```

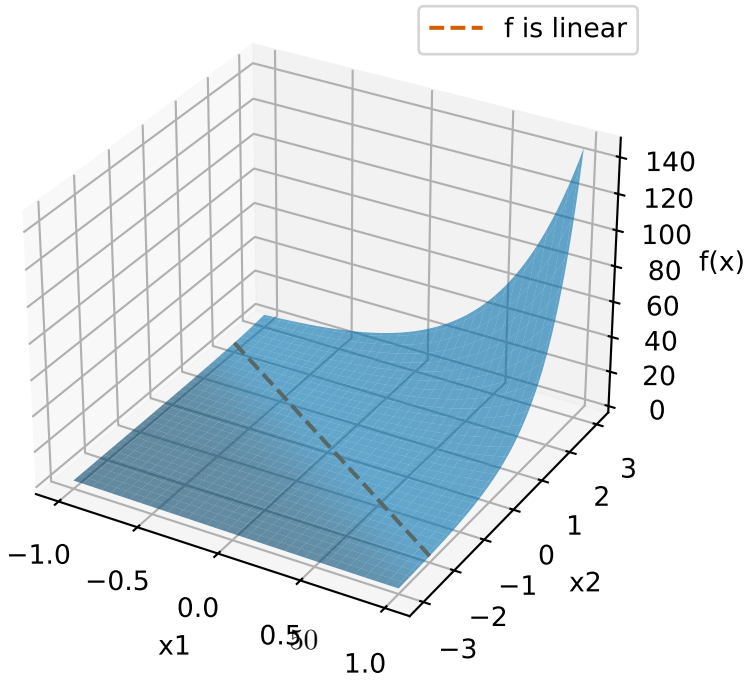


Figure 18: The graph of the third function in Example 0.11, the surface whose level curves are in Figure 17. Along the dashed line through the origin in the direction $(1, -2)^\top$ the surface is a straight line, so the function is linear there. The function is therefore

so $\mathbf{H}_f(0) = [0]$ is singular and not positive definite. Yet f is strictly convex, by the strict form of Equation 1: for $x \neq y$,

$$\begin{aligned} x^4 - y^4 - 4y^3(x-y) &= (x-y)(x^3 + x^2y + xy^2 + y^3) - 4y^3(x-y) && \text{(factor } x^4 - y^4\text{)} \\ &= (x-y)(x^3 + x^2y + xy^2 - 3y^3) && \text{(collect the } y^3 \text{ terms)} \\ &= (x-y)^2(x^2 + 2xy + 3y^2) && ((x-y)(x^2+2xy+3y^2) = x^3 + x^2y + x \\ &= (x-y)^2((x+y)^2 + 2y^2), && \text{(complete the square)} \end{aligned}$$

which is positive: $(x-y)^2 > 0$, and $(x+y)^2 + 2y^2 = 0$ would need $y = 0$ and then

$$\begin{aligned} x &= 0 \\ &= y. \end{aligned}$$

Theorem 0.6 (A convex function has a positive semidefinite Hessian)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$ have first and second partial derivatives that are continuous on $\mathbb{R}^p$. If f is convex, then the Hessian $\mathbf{H}_f(\underset{\sim}{x})$ is positive semidefinite for every $\underset{\sim}{x} \in \mathbb{R}^p$.*

Proof

Proof. Fix $\underset{\sim}{x}, \underset{\sim}{v} \in \mathbb{R}^p$. Let $h > 0$. By Theorem 0.4 applied with $\underset{\sim}{z} = \underset{\sim}{x}$ and $\underset{\sim}{h} = h\underset{\sim}{v}$, and again with $\underset{\sim}{h} = -h\underset{\sim}{v}$, there are $\tau_+, \tau_- \in (0,1)$ with

$$f(\underset{\sim}{x} + h\underset{\sim}{v}) = f(\underset{\sim}{x}) + h\left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})\right)^\top \underset{\sim}{v} + \frac{h^2}{2}\, \underset{\sim}{v}^\top \mathbf{H}_f(\underset{\sim}{w}_+)\, \underset{\sim}{v} \quad \text{(Theorem 0.4)}$$

$$f(\underset{\sim}{x} - h\underset{\sim}{v}) = f(\underset{\sim}{x}) - h\left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})\right)^\top \underset{\sim}{v} + \frac{h^2}{2}\, \underset{\sim}{v}^\top \mathbf{H}_f(\underset{\sim}{w}_-)\, \underset{\sim}{v}, \quad \text{(Theorem 0.4)}$$

where $\underset{\sim}{w}_\pm = \underset{\sim}{x} \pm \tau_\pm h\underset{\sim}{v}$. Adding the two equations cancels the first-order summands:

$$f(\underset{\sim}{x} + h\underset{\sim}{v}) + f(\underset{\sim}{x} - h\underset{\sim}{v}) - 2f(\underset{\sim}{x}) = \frac{h^2}{2}\, \underset{\sim}{v}^\top \big(\mathbf{H}_f(\underset{\sim}{w}_+) + \mathbf{H}_f(\underset{\sim}{w}_-)\big)\underset{\sim}{v}.$$

Since f is convex and $\underset{\sim}{x} = \frac{1}{2}(\underset{\sim}{x} + h\underset{\sim}{v}) + \frac{1}{2}(\underset{\sim}{x} - h\underset{\sim}{v})$,

$$f(\underset{\sim}{x}) \leq \frac{1}{2} f(\underset{\sim}{x} + h\underset{\sim}{v}) + \frac{1}{2} f(\underset{\sim}{x} - h\underset{\sim}{v}),$$

so the left side of the previous display is nonnegative. Dividing by $h^2/2$ gives

$$\underset{\sim}{v}^\top \big(\mathbf{H}_f(\underset{\sim}{w}_+) + \mathbf{H}_f(\underset{\sim}{w}_-)\big)\underset{\sim}{v} \geq 0 \qquad \text{for every } h > 0.$$

As $h \to 0$, both $\underset{\sim}{w}_+$ and $\underset{\sim}{w}_-$ approach $\underset{\sim}{x}$, and the entries of $\mathbf{H}_f$ are continuous, so the left side approaches $2\, \underset{\sim}{v}^\top \mathbf{H}_f(\underset{\sim}{x})\, \underset{\sim}{v}$. Therefore $\underset{\sim}{v}^\top \mathbf{H}_f(\underset{\sim}{x})\, \underset{\sim}{v} \geq 0$ for every $\underset{\sim}{v}$. □

Corollary 0.1 (Stationary points of a convex function are global minimizers)**.** *If f is as in Theorem 0.4 and convex, then every stationary point $\underset{\sim}{x}^*$ of f (Definition 0.1) is a global minimizer.*

Proof

Proof. For every $\underset{\sim}{x}$,

$$\begin{aligned} f(\underset{\sim}{x}) &\geq f(\underset{\sim}{x}^*) + \left(\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^*)\right)^\top (\underset{\sim}{x} - \underset{\sim}{x}^*) && \text{(Equation 1 with } \underset{\sim}{y} = \underset{\sim}{x}^*) \\ &= f(\underset{\sim}{x}^*) + \underset{\sim}{0}^\top (\underset{\sim}{x} - \underset{\sim}{x}^*) && (\underset{\sim}{x}^* \text{ is stationary}) \\ &= f(\underset{\sim}{x}^*). && (\underset{\sim}{0} \cdot \underset{\sim}{v} = 0) \end{aligned}$$

□

Example 0.13 (A convex function, and a non-convex one)**.**

- $f(\underset{\sim}{x}) = x_1^2 + x_2^2 - 2x_1$ is convex by Theorem 0.5, since its Hessian is $2\mathbf{I}_2$ (Example 0.10), so its stationary point $(1, 0)^\top$ (Example 0.2) is a global minimizer, as completing the square showed in Example 0.3.
- Without convexity, a stationary point need not be a minimizer at all: $x_1^2 - x_2^2$ has the stationary point $\underset{\sim}{0}$, which is not a minimizer (Example 0.3).
- The corollary says nothing about whether a minimizer exists: $e^{2x_1 + x_2} - x_1$ is convex (Example 0.11) but has no stationary point, and so no local minimizer (Example 0.3).

Theorem 0.7 (The exponential function lies above its tangent line at zero)**.** *For every real x,*

$$\exp\{x\} \geq 1 + x,$$

with equality exactly when $x = 0$.

Proof

Proof. Let $g(x) = \exp\{x\} - 1 - x$. The exponential function has second derivative $\exp\{x\} > 0$ at every x, so it is convex by Theorem 0.5. The function g differs from it by a linear function, so g is convex too. The derivative of g is $\frac{dg(x)}{dx} = \exp\{x\} - 1$, which is 0 at $x = 0$. By Corollary 0.1, 0 is a global minimizer of g, and $g(0) = 0$, so $g(x) \geq 0$ for every x.
Suppose $g(x) = 0$ for some x. Then x is also a global minimizer of g, and so a local

minimizer. Fermat's theorem gives $\frac{dg(x)}{dx} = \exp\{x\} - 1 = 0$. Then $\exp\{x\} = 1 = \exp\{0\}$, which gives $x = 0$ because the exponential function is injective. □

Corollary 0.2 (The natural logarithm lies below its tangent line at one)**.** *For every* $t > 0$*,*

$$\log\{t\} \leq t - 1,$$

with equality exactly when $t = 1$*.*

Proof

Proof. Put $x = \log\{t\}$, so that $\exp\{x\} = t$ by the inverse relation of the logarithm and the exponential. By Theorem 0.7, $t = \exp\{x\} \geq 1 + x = 1 + \log\{t\}$, which is $\log\{t\} \leq t - 1$. Equality holds exactly when $x = 0$, that is, when $\log\{t\} = 0$, that is, when $t = 1$. □

Example 0.14 (Two values of the bounds)**.**
At $x = 1$, $\exp\{1\} = \mathrm{e} \approx 2.718$, which is at least $1 + 1 = 2$. At $t = 2$, $\log\{2\} \approx 0.693$, which is at most $2 - 1 = 1$. Neither is an equality, because $x \neq 0$ and $t \neq 1$.

Minimizing a quadratic under linear constraints

Source

This section is adapted from the Lagrange-multiplier part of Zhou [3], used under the MIT License (see the license text in Linear Algebra). The source solves the stationarity equations of the Lagrangian and reports the solution; here the solution is also proved to be the unique minimizer, directly from the positive definiteness of the matrices involved. The source assumes only that $\mathbf{S}$ is positive definite; the added hypothesis that the columns of $\mathbf{A}$ are linearly independent makes $\mathbf{A}^{\top}\mathbf{S}^{-1}\mathbf{A}$ invertible. The source's example with general coefficients a_1, a_2, b is specialized here to $3, 4, 5$. These parts of the source are not part of this section:

- the Lagrange multiplier method for constraints that are not linear, or objectives that are not quadratic, including its multinomial maximum likelihood example
- the eigenvalues of the block matrix of the stationarity equations, and the convexity of the Lagrangian in x and its concavity in the multipliers
- its homework examples

Definition 0.1 (Feasible set)**.** In a problem that looks for a best point among some allowed points of $\mathbb{R}^p$, the set $F \subseteq \mathbb{R}^p$ of allowed points is the **feasible set**, [11] and its points are feasible.

Definition 0.2 (Constraints)**.** When the feasible set F (Definition 0.1) is the set of points that satisfy some equations or inequalities, those equations or inequalities are the **constraints**.

Example 0.1 (A line as a feasible set)**.** With the single constraint $3x_1 + 4x_2 = 5$, the feasible set is the line $\{\underset{\sim}{x} \in \mathbb{R}^2 : 3x_1 + 4x_2 = 5\}$. $(1, 0.5)^\top$ is feasible, since

$$\begin{aligned} 3 \cdot 1 + 4 \cdot 0.5 &= 3 + 2 \\ &= 5; \end{aligned}$$

$\underset{\sim}{0}$ is not feasible, since $3 \cdot 0 + 4 \cdot 0 = 0 \neq 5$.

Definition 0.3 (Objective function, objective)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ and let $F \subseteq \mathbb{R}^p$ be a feasible set (Definition 0.1). In the problem of minimizing f over F, f is the **objective function** (or **objective**).

Definition 0.4 (Minimizer over a feasible set)**.** With f and F as in Definition 0.3, a **minimizer of f over F** is a feasible $\underset{\sim}{x}^*$ with $f(\underset{\sim}{x}^*) \leq f(\underset{\sim}{x})$ for every feasible $\underset{\sim}{x}$: a global minimizer of f with its domain restricted to F.

Definition 0.5 (Minimization problem)**.** With f and F as in Definition 0.3, the **minimization problem** [12] is to find a minimizer of f over F (Definition 0.4). When F is given by constraints (Definition 0.2), the problem is to minimize f subject to them.

Example 0.2 (Minimizing over a line)**.** In the problem of minimizing $x_1^2 + x_2^2$ subject to $3x_1 + 4x_2 = 5$, the objective function is $f(\underset{\sim}{x}) = x_1^2 + x_2^2$, and the feasible set is the line of Example 0.1. The feasible point $(1, 0.5)^\top$ has

$$\begin{aligned} f(1, 0.5) &= 1 + 0.25 \\ &= 1.25. \end{aligned}$$

$\underset{\sim}{0}$ minimizes f over all of $\mathbb{R}^2$, with $f(\underset{\sim}{0}) = 0$, but it is not a minimizer over the feasible set, because it is not feasible.

Definition 0.6 (Optimization algorithm)**.** Let:

- $f : \mathbb{R}^p \to \mathbb{R}$: an objective function;
- $F \subseteq \mathbb{R}^p$: a feasible set.

An **optimization algorithm** [12] is a step-by-step procedure that maps the pair (f, F) to a feasible point $\underset{\sim}{x}^\dagger$:

$$(f, F) \mapsto \underset{\sim}{x}^\dagger \in F. \tag{2}$$

In words: given (f, F), the procedure returns a feasible point $\underset{\sim}{x}^\dagger$ intended to make $f(\underset{\sim}{x}^\dagger)$ small.

Example 0.3 (Exhaustive search over a finite feasible set)**.** Let $f(\underset{\sim}{x}) = x_1^2 + x_2^2$, with the feasible set the three points $\left\{(1, 0.5)^\top, (3, -1)^\top, (-1, 2)^\top\right\}$ of the line of Example 0.1. The procedure "evaluate f at every feasible point; return the point with the smallest value" is an optimization algorithm. Computing f at each point gives

$$\begin{aligned} f(1, 0.5) &= 1 + 0.25 = 1.25, \\ f(3, -1) &= 9 + 1 = 10, \\ f(-1, 2) &= 1 + 4 = 5, \end{aligned}$$

so it returns $(1, 0.5)^\top$, a minimizer of f over this feasible set.

Here the objective is a quadratic $f(\underset{\sim}{x}) = \frac{1}{2}\,\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x}$ with $\mathbf{S}$ positive definite, and the constraints (Definition 0.2) are m linear equations $\mathbf{A}^\top \underset{\sim}{x} = \underset{\sim}{b}$, where $\mathbf{A}$ is a $p \times m$ matrix and $\underset{\sim}{b} \in \mathbb{R}^m$. The only stationary point of the objective alone (Definition 0.1) is $\underset{\sim}{0}_{p\times 1}$ (its gradient is $\underset{\sim}{\nabla} f(\underset{\sim}{x}) = \mathbf{S}\underset{\sim}{x}$, and $\mathbf{S}$ is invertible), which may not satisfy the constraints. A Lagrange multiplier adds one unknown per constraint, and the minimizer over the feasible set (Definition 0.1) is then found among the stationary points of a new function of the extended set of unknowns.

Definition 0.7 (Lagrangian)**.** For the problem of minimizing $f(\underset{\sim}{x})$ over $\underset{\sim}{x} \in \mathbb{R}^p$ subject to $\mathbf{A}^\top \underset{\sim}{x} = \underset{\sim}{b}$, with $\mathbf{A}$ a $p \times m$ matrix and $\underset{\sim}{b} \in \mathbb{R}^m$, the **Lagrangian** [13] is the function of $\underset{\sim}{x} \in \mathbb{R}^p$ and $\underset{\sim}{\lambda} \in \mathbb{R}^m$

$$L(\underset{\sim}{x}, \underset{\sim}{\lambda}) \stackrel{\text{def}}{=} f(\underset{\sim}{x}) + \underset{\sim}{\lambda}^\top (\mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b}).$$

Definition 0.8 (Lagrange multipliers)**.** In the Lagrangian $L(\underset{\sim}{x}, \underset{\sim}{\lambda})$ (Definition 0.7), the entries of $\underset{\sim}{\lambda}$ are the **Lagrange multipliers** [13], one per constraint.

Example 0.4 (The gradients of the Lagrangian of a quadratic)**.** Let $f(\underset{\sim}{x}) = \frac{1}{2}\,\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x}$ with $\mathbf{S}$ symmetric and constant. By the transpose of a product, $\underset{\sim}{\lambda}^\top \mathbf{A}^\top = (\mathbf{A}\underset{\sim}{\lambda})^\top$, so

$$\begin{aligned} L(\underset{\sim}{x},\underset{\sim}{\lambda}) &= \tfrac{1}{2}\,\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x} + \underset{\sim}{\lambda}^\top \mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{\lambda}^\top \underset{\sim}{b} && \text{(distribute)} \\ &= \tfrac{1}{2}\,\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x} + (\mathbf{A}\underset{\sim}{\lambda})^\top \underset{\sim}{x} - \underset{\sim}{\lambda}^\top \underset{\sim}{b}, && \text{(transpose of a product)} \end{aligned}$$

and, since $\underset{\sim}{\lambda}^\top(\mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b})$ is 1×1 and so equals its transpose,

$$L(\underset{\sim}{x},\underset{\sim}{\lambda}) = \tfrac{1}{2}\,\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x} + (\mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b})^\top \underset{\sim}{\lambda}.$$

The partial derivatives of a sum are the sums of the partial derivatives, entry by entry. Using the first of these forms, the derivative of a quadratic form applies to the first term, the derivative of a linear combination to the second ($\mathbf{A}\underset{\sim}{\lambda}$ is constant in $\underset{\sim}{x}$), and the third is constant with respect to a vector,

$$\begin{aligned} \frac{\partial}{\partial \underset{\sim}{x}} L(\underset{\sim}{x},\underset{\sim}{\lambda}) &= \tfrac{1}{2}\,\frac{\partial}{\partial \underset{\sim}{x}}(\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x}) + \frac{\partial}{\partial \underset{\sim}{x}}\big((\mathbf{A}\underset{\sim}{\lambda})^\top \underset{\sim}{x}\big) - \frac{\partial}{\partial \underset{\sim}{x}}(\underset{\sim}{\lambda}^\top \underset{\sim}{b}) && \text{(sum and constant multiple)} \\ &= \tfrac{1}{2}(2\mathbf{S}\underset{\sim}{x}) + \mathbf{A}\underset{\sim}{\lambda} - \underset{\sim}{0}_{p\times 1} && \text{(the three derivatives)} \\ &= \mathbf{S}\underset{\sim}{x} + \mathbf{A}\underset{\sim}{\lambda}. && \text{(simplify)} \end{aligned}$$

Using the second form, in which $f(\underset{\sim}{x})$ and $\mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b}$ are constant in $\underset{\sim}{\lambda}$, and the derivative of a linear combination again,

$$\begin{aligned} \frac{\partial}{\partial \underset{\sim}{\lambda}} L(\underset{\sim}{x},\underset{\sim}{\lambda}) &= \frac{\partial}{\partial \underset{\sim}{\lambda}} f(\underset{\sim}{x}) + \frac{\partial}{\partial \underset{\sim}{\lambda}}\big((\mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b})^\top \underset{\sim}{\lambda}\big) && \text{(sum)} \\ &= \underset{\sim}{0}_{m\times 1} + (\mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b}) && \text{(the two derivatives)} \\ &= \mathbf{A}^\top \underset{\sim}{x} - \underset{\sim}{b}. && \text{(simplify)} \end{aligned}$$

So $(\underset{\sim}{x},\underset{\sim}{\lambda})$ makes the gradients equal $\underset{\sim}{0}_{p\times 1}$ and $\underset{\sim}{0}_{m\times 1}$ exactly when

$$\mathbf{S}\underset{\sim}{x} + \mathbf{A}\underset{\sim}{\lambda} = \underset{\sim}{0}_{p\times 1} \quad \text{and} \quad \mathbf{A}^\top \underset{\sim}{x} = \underset{\sim}{b}, \qquad \text{that is,} \qquad \underbrace{\begin{bmatrix} \mathbf{S} & \mathbf{A} \\ \mathbf{A}^\top & \underset{\sim}{0}_{m\times m} \end{bmatrix}}_{(p+m)\times(p+m)} \underbrace{\begin{bmatrix} \underset{\sim}{x} \\ \underset{\sim}{\lambda} \end{bmatrix}}_{(p+m)\times 1} = \underbrace{\begin{bmatrix} \underset{\sim}{0}_{p\times 1} \\ \underset{\sim}{b} \end{bmatrix}}_{(p+m)\times 1}, \tag{3}$$

the second equation being the constraint itself.

Theorem 0.1 (Minimizing a positive definite quadratic under linear constraints)**.** *Let* $\mathbf{S}$ *be a* $p \times p$ *positive definite matrix,* $\mathbf{A}$ *a* $p \times m$ *matrix with linearly independent columns, and* $\underset{\sim}{b} \in \mathbb{R}^m$*. Then the* $m \times m$ *matrix* $\mathbf{M} \overset{def}{=} \mathbf{A}^\top \mathbf{S}^{-1}\mathbf{A}$ *is invertible, and* $f(\underset{\sim}{x}) = \frac{1}{2}\,\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x}$ *has exactly one minimizer over the* $\underset{\sim}{x} \in \mathbb{R}^p$ *with* $\mathbf{A}^\top \underset{\sim}{x} = \underset{\sim}{b}$*, namely*

$$\underset{\sim}{x}^* = \mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b}.$$

With $\underset{\sim}{\lambda}^* \overset{def}{=} -\mathbf{M}^{-1}\underset{\sim}{b}$*, the pair* $(\underset{\sim}{x}^*, \underset{\sim}{\lambda}^*)$ *solves Equation 3, and the minimum value is* $f(\underset{\sim}{x}^*) = \frac{1}{2}\,\underset{\sim}{b}^\top\mathbf{M}^{-1}\underset{\sim}{b}$.

Proof

Proof. The steps marked "transpose of a product" use the transpose of a product, and $\mathbf{S}$ is symmetric because it is positive definite.

$\mathbf{M}$ is invertible. $\mathbf{S}^{-1}$ is positive definite, by the positive definite inverse theorem, so $\mathbf{M} = \mathbf{A}^\top\mathbf{S}^{-1}\mathbf{A}$ is positive definite, by operations that preserve definiteness (part 1), and so invertible, again by the positive definite inverse. $\mathbf{S}^{-1}$ and $\mathbf{M}^{-1}$ are symmetric, by the inverse of a symmetric matrix, because $\mathbf{S}$ and $\mathbf{M}$ are.

$\underset{\sim}{x}^*$ satisfies the constraint, and the pair solves Equation 3.

$$\begin{aligned}
\mathbf{A}^\top\underset{\sim}{x}^* &= \mathbf{A}^\top\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(substitute } \underset{\sim}{x}^*\text{)}\\
&= \mathbf{M}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(definition of } \mathbf{M}\text{)}\\
&= \underset{\sim}{b}, && (\mathbf{M}\mathbf{M}^{-1} = \mathbf{I}_m)\\
\mathbf{S}\underset{\sim}{x}^* + \mathbf{A}\underset{\sim}{\lambda}^* &= \mathbf{S}\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} - \mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(substitute } \underset{\sim}{x}^* \text{ and } \underset{\sim}{\lambda}^*\text{)}\\
&= \mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} - \mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && (\mathbf{S}\mathbf{S}^{-1} = \mathbf{I}_p)\\
&= \underset{\sim}{0}_{p\times 1}. && \text{(subtract)}
\end{aligned}$$

$\underset{\sim}{x}^*$ is the unique minimizer. Let $\underset{\sim}{x}$ satisfy $\mathbf{A}^\top\underset{\sim}{x} = \underset{\sim}{b}$, and let $\underset{\sim}{d} \overset{\text{def}}{=} \underset{\sim}{x} - \underset{\sim}{x}^*$, so

$$\begin{aligned}
\mathbf{A}^\top\underset{\sim}{d} &= \mathbf{A}^\top\underset{\sim}{x} - \mathbf{A}^\top\underset{\sim}{x}^*\\
&= \underset{\sim}{b} - \underset{\sim}{b}\\
&= \underset{\sim}{0}_{m\times 1}
\end{aligned}$$

(matrix products distribute; both points satisfy the constraint). First, the cross term vanishes:

$$\begin{aligned}
\underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{x}^* &= \underset{\sim}{d}^\top\mathbf{S}\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(substitute } \underset{\sim}{x}^*\text{)}\\
&= \underset{\sim}{d}^\top\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && (\mathbf{S}\mathbf{S}^{-1} = \mathbf{I}_p)\\
&= (\mathbf{A}^\top\underset{\sim}{d})^\top\mathbf{M}^{-1}\underset{\sim}{b} && \text{(transpose of a product; } (\mathbf{A}^\top)^\top = \mathbf{A}\text{)}\\
&= \underset{\sim}{0}_{m\times 1}{}^\top\mathbf{M}^{-1}\underset{\sim}{b} && (\mathbf{A}^\top\underset{\sim}{d} = \underset{\sim}{0}_{m\times 1})\\
&= 0. && \text{(a product with a zero factor is zero)}
\end{aligned}$$

Then

$$
\begin{aligned}
f(\underset{\sim}{x}) &= \tfrac{1}{2}\,(\underset{\sim}{x}^* + \underset{\sim}{d})^\top \mathbf{S}(\underset{\sim}{x}^* + \underset{\sim}{d}) && (\underset{\sim}{x} = \underset{\sim}{x}^* + \underset{\sim}{d}) \\
&= \tfrac{1}{2}(\underset{\sim}{x}^{*\top} + \underset{\sim}{d}^\top)\mathbf{S}(\underset{\sim}{x}^* + \underset{\sim}{d}) && \text{(the transpose of a sum is the sum of the transposes)} \\
&= \tfrac{1}{2}(\underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{x}^* + \underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{d} + \underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{x}^* + \underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{d}) && \text{(distribute)} \\
&= \tfrac{1}{2}(\underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{x}^* + \underset{\sim}{d}^\top\mathbf{S}^\top\underset{\sim}{x}^* + \underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{x}^* + \underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{d}) && (1\times 1\text{: } \underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{d} = (\underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{d})^\top\text{; transpose of a product}) \\
&= \tfrac{1}{2}(\underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{x}^* + 2\,\underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{x}^* + \underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{d}) && (\mathbf{S}^\top = \mathbf{S}\text{; combine}) \\
&= \tfrac{1}{2}\,\underset{\sim}{x}^{*\top}\mathbf{S}\underset{\sim}{x}^* + \underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{x}^* + \tfrac{1}{2}\,\underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{d} && (\text{distribute } \tfrac{1}{2}) \\
&= f(\underset{\sim}{x}^*) + \tfrac{1}{2}\,\underset{\sim}{d}^\top\mathbf{S}\underset{\sim}{d} && \text{(the cross term is 0)} \\
&\geq f(\underset{\sim}{x}^*), && (\mathbf{S}\text{ is positive definite})
\end{aligned}
$$

with equality only when $\underset{\sim}{d} = \underset{\sim}{0}_{p\times 1}$, that is, $\underset{\sim}{x} = \underset{\sim}{x}^*$.
The minimum value.

$$
\begin{aligned}
f(\underset{\sim}{x}^*) &= \tfrac{1}{2}\,(\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b})^\top\mathbf{S}\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(substitute } \underset{\sim}{x}^*) \\
&= \tfrac{1}{2}\,\underset{\sim}{b}^\top(\mathbf{M}^{-1})^\top\mathbf{A}^\top(\mathbf{S}^{-1})^\top\mathbf{S}\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(transpose of a product, applied repeatedly)} \\
&= \tfrac{1}{2}\,\underset{\sim}{b}^\top\mathbf{M}^{-1}\mathbf{A}^\top\mathbf{S}^{-1}\mathbf{S}\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && (\mathbf{S}^{-1}\text{ and }\mathbf{M}^{-1}\text{ are symmetric}) \\
&= \tfrac{1}{2}\,\underset{\sim}{b}^\top\mathbf{M}^{-1}\mathbf{A}^\top\mathbf{S}^{-1}\mathbf{A}\mathbf{M}^{-1}\underset{\sim}{b} && (\mathbf{S}^{-1}\mathbf{S} = \mathbf{I}_p) \\
&= \tfrac{1}{2}\,\underset{\sim}{b}^\top\mathbf{M}^{-1}\mathbf{M}\mathbf{M}^{-1}\underset{\sim}{b} && \text{(definition of } \mathbf{M}) \\
&= \tfrac{1}{2}\,\underset{\sim}{b}^\top\mathbf{M}^{-1}\underset{\sim}{b}. && (\mathbf{M}^{-1}\mathbf{M} = \mathbf{I}_m)
\end{aligned}
$$

□

Example 0.5 (The closest point on a line to the origin)**.** Minimize $x_1^2 + x_2^2$ subject to $3x_1 + 4x_2 = 5$. In the form of Theorem 0.1, $x_1^2 + x_2^2 = \frac{1}{2}\,\underset{\sim}{x}^\top(2\mathbf{I}_2)\underset{\sim}{x}$, so $\mathbf{S} = 2\mathbf{I}_2$, $\mathbf{A} = (3,4)^\top$ ($p = 2$, $m = 1$) and $\underset{\sim}{b} = [5]$. The hypotheses hold: $2\mathbf{I}_2$ is positive definite, and the single column $(3,4)^\top$ is nonzero, so it is linearly independent. Also $\mathbf{S}^{-1} = \frac{1}{2}\mathbf{I}_2$. Then, from the formulas of Theorem 0.1,

$$
\begin{aligned}
\mathbf{M} &= \mathbf{A}^\top \tfrac{1}{2}\mathbf{I}_2\, \mathbf{A} \\
&= \tfrac{1}{2}\,(3^2 + 4^2) \\
&= \tfrac{25}{2}, \\
\underset{\sim}{x}^* &= \tfrac{1}{2}\mathbf{I}_2 \begin{bmatrix} 3 \\ 4 \end{bmatrix} (\tfrac{2}{25})(5) \\
&= \tfrac{5}{25} \begin{bmatrix} 3 \\ 4 \end{bmatrix} \\
&= \begin{bmatrix} 0.6 \\ 0.8 \end{bmatrix}, \\
\underset{\sim}{\lambda}^* &= -(\tfrac{2}{25})(5) \\
&= -0.4, \\
f(\underset{\sim}{x}^*) &= \tfrac{1}{2}(5)(\tfrac{2}{25})(5) \\
&= 1.
\end{aligned}
$$

Checks:

- the constraint:

$$
\begin{aligned}
3(0.6) + 4(0.8) &= 1.8 + 3.2 \\
&= 5;
\end{aligned}
$$

- the first block of Equation 3:

$$
\begin{aligned}
\mathbf{S}\underset{\sim}{x}^* + \mathbf{A}\underset{\sim}{\lambda}^* &= (1.2, 1.6)^\top + (3, 4)^\top (-0.4) \\
&= (1.2, 1.6)^\top - (1.2, 1.6)^\top \\
&= \underset{\sim}{0}_{2\times 1};
\end{aligned}
$$

- the value:

$$
\begin{aligned}
0.6^2 + 0.8^2 &= 0.36 + 0.64 \\
&= 1.
\end{aligned}
$$

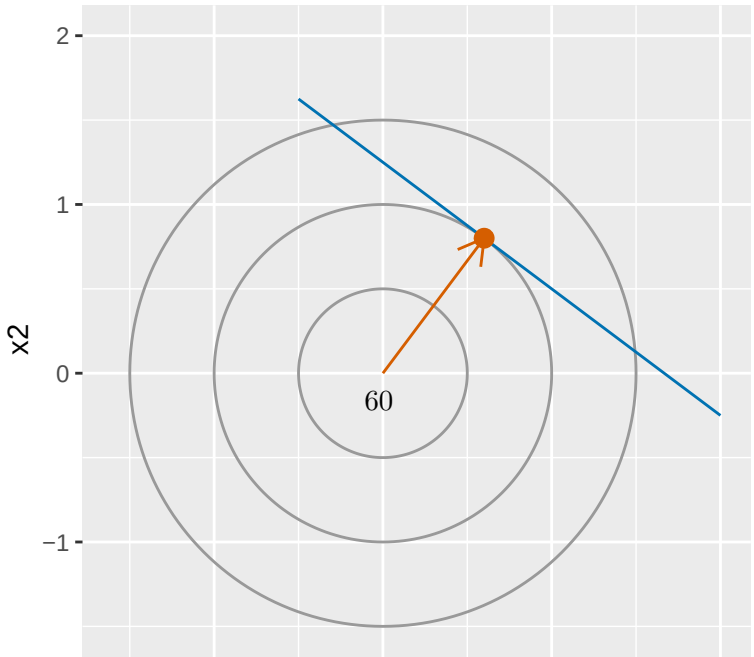
Geometrically, $\underset{\sim}{x}^*$ is the point of the line nearest the origin, at distance 1.

Example 0.6 (The hypotheses are needed)**.**

- If $\mathbf{S}$ is not positive definite there may be no minimizer: with $\mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ and the

R

```
ll_theta_ll <- seq(0, 2 * pi, length.out = 201)
ll_radii_ll <- c(0.5, 1, 1.5)
ll_circles_ll <- expand.grid(theta = ll_theta_ll, radius = ll_radii_ll)
ll_circles_ll$x1 <- ll_circles_ll$radius * cos(ll_circles_ll$theta)
ll_circles_ll$x2 <- ll_circles_ll$radius * sin(ll_circles_ll$theta)
ll_line_x1_ll <- c(-0.5, 2)
ll_line_x2_ll <- (5 - 3 * ll_line_x1_ll) / 4
ll_line_ll <- tibble::tibble(x1 = ll_line_x1_ll, x2 = ll_line_x2_ll)
ll_point_ll <- tibble::tibble(x1 = 0.6, x2 = 0.8)
ll_arrow_ll <- tibble::tibble(x = 0, y = 0, xend = 0.6, yend = 0.8)
ll_aes_ll <- ggplot2::aes(x1, x2)
ll_aes_circle_ll <- ggplot2::aes(x1, x2, group = radius)
ll_aes_arrow_ll <- ggplot2::aes(x = x, y = y, xend = xend, yend = yend)
ll_head_length_ll <- grid::unit(0.15, "inches")
ll_head_ll <- grid::arrow(length = ll_head_length_ll)
ll_lim_ll <- c(-1.6, 2)
ggplot2::ggplot() +
  ggplot2::geom_path(
    data = ll_circles_ll, mapping = ll_aes_circle_ll, colour = "grey60"
  ) +
  ggplot2::geom_line(
    data = ll_line_ll, mapping = ll_aes_ll, colour = "#0072B2"
  ) +
  ggplot2::geom_segment(
    data = ll_arrow_ll, mapping = ll_aes_arrow_ll,
    colour = "#D55E00", arrow = ll_head_ll
  ) +
  ggplot2::geom_point(
    data = ll_point_ll, mapping = ll_aes_ll, colour = "#D55E00", size = 3
  ) +
  ggplot2::coord_fixed(xlim = ll_lim_ll, ylim = ll_lim_ll) +
  ggplot2::labs(x = "x1", y = "x2")
```



R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

ll_x1_3d, ll_x2_3d = np.meshgrid(np.linspace(-1.6, 2, 101),
                                 np.linspace(-1.6, 2, 101))
ll_line_x1_3d = np.linspace(-0.5, 2, 101)
ll_line_x2_3d = (5 - 3 * ll_line_x1_3d) / 4
ll_line_z_3d = ll_line_x1_3d**2 + ll_line_x2_3d**2
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(ll_x1_3d, ll_x2_3d, ll_x1_3d**2 + ll_x2_3d**2,
                color="grey", alpha=0.4)
ax.plot(ll_line_x1_3d, ll_line_x2_3d, ll_line_z_3d, color="#0072B2",
        label="constraint")
ax.scatter([0.6], [0.8], [1], marker="o", color="#D55E00", s=50,
           label="minimizer")
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_zlabel("f(x)")
ax.legend(loc="upper right")
plt.show();
```

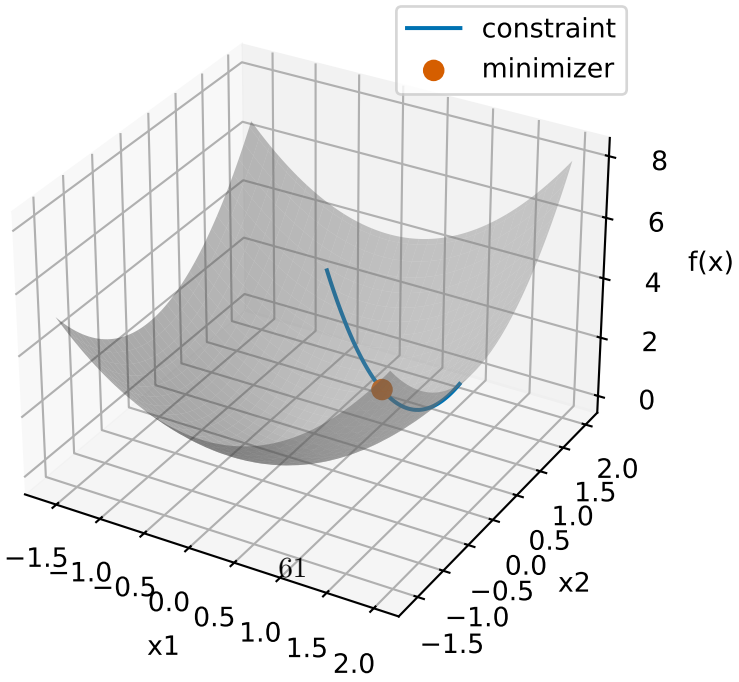


Figure 20: The graph of the objective $x_1^2 + x_2^2$, the surface whose level curves (circles) are in

constraint $x_1 = 1$ ($\mathbf{A} = (1,0)^\top$, $\underset{\sim}{b} = [1]$), $f(1, x_2) = \frac{1}{2}(1 - x_2^2)$ decreases without bound as x_2 grows.

- If the columns of $\mathbf{A}$ are dependent, $\mathbf{M}$ is not invertible: with $\mathbf{S} = \mathbf{I}_2$ and $\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$,

$$\begin{aligned} \mathbf{M} &= \mathbf{A}^\top \mathbf{A} \\ &= \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix}, \end{aligned}$$

whose columns are equal. So $\mathbf{M}(1,-1)^\top = \underset{\sim}{0}_{2\times 1}$, and $\mathbf{M}$ is not invertible, since a square matrix is invertible exactly when its null space is zero. The two constraints both read $x_1 + x_2 = b_i$, so they have no solution at all when $b_1 \neq b_2$. When $b_1 = b_2$ a unique minimizer still exists,

$$\begin{aligned} x_1 &= x_2 \\ &= b_1/2, \end{aligned}$$

but the formula of Theorem 0.1 cannot produce it.

Definition 0.9 (Sensitivity)**.** Suppose a minimization problem (Definition 0.5) depends on a vector $\underset{\sim}{b} \in \mathbb{R}^m$, and let $f^*(\underset{\sim}{b})$ be its minimum value, as a function of $\underset{\sim}{b}$. Where f^* has a gradient $\frac{\partial}{\partial \underset{\sim}{b}} f^*(\underset{\sim}{b})$, that gradient is the **sensitivity** of the minimum value to $\underset{\sim}{b}$: by the definition of a partial derivative, its entry i is the limit as a change in b_i alone shrinks to 0, of the resulting change in the minimum value divided by the change in b_i.

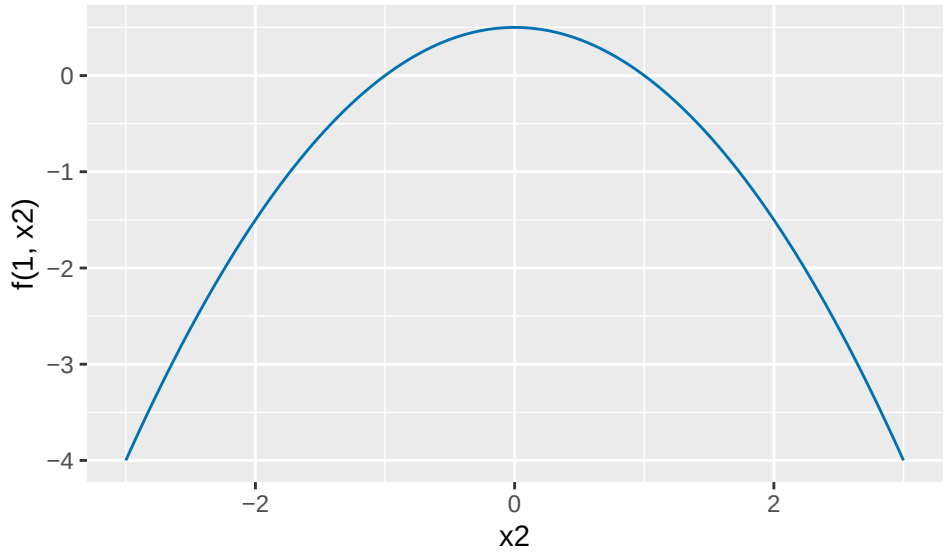
Example 0.7 (The sensitivity of a one-point problem)**.** Minimize x^2 subject to $x = b$, for a number b. The feasible set (Definition 0.1) is $\{b\}$, so the minimum value is $f^*(b) = b^2$, and its sensitivity to b is $\frac{d}{db} f^*(b) = 2b$. At $b = 3$ the sensitivity is 6: moving b from 3 to 3.1 raises the minimum value from 9 to 9.61, an increase of 0.61, close to $6 \times 0.1 = 0.6$.

Corollary 0.1 (The multiplier is the sensitivity of the minimum value)**.** *In Theorem 0.1, write the minimum value as a function of* $\underset{\sim}{b}$*,* $f^*(\underset{\sim}{b}) = \frac{1}{2}\, \underset{\sim}{b}^\top \mathbf{M}^{-1} \underset{\sim}{b}$*. Its sensitivity to* $\underset{\sim}{b}$ *(Definition 0.9) is*

$$\begin{aligned} \frac{\partial}{\partial \underset{\sim}{b}} f^*(\underset{\sim}{b}) &= \mathbf{M}^{-1} \underset{\sim}{b} \\ &= -\underset{\sim}{\lambda}^*. \end{aligned}$$

R

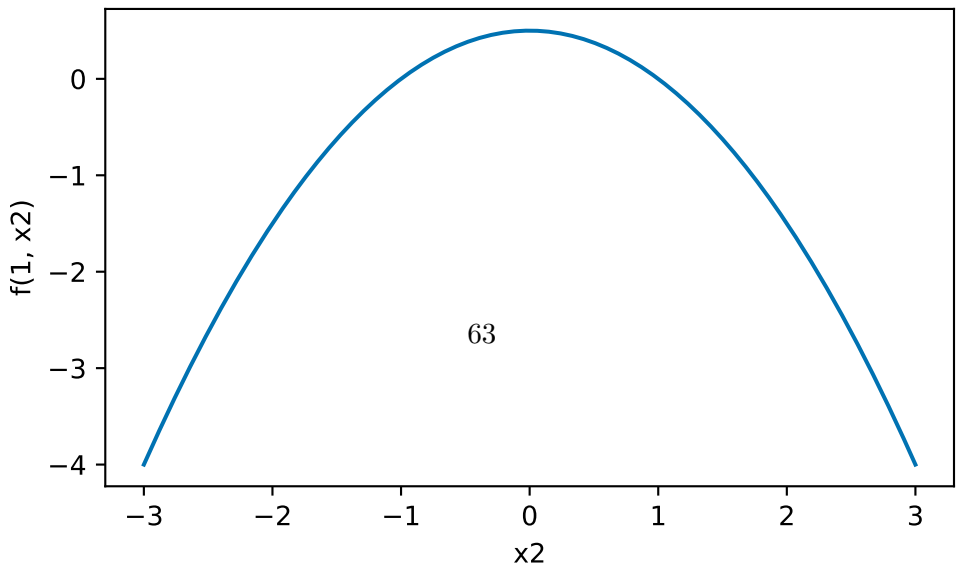
```r
qf_x2_qf <- seq(-3, 3, length.out = 201)
qf_curve_qf <- tibble::tibble(x2 = qf_x2_qf, f = 0.5 * (1 - qf_x2_qf^2))
ggplot2::ggplot(qf_curve_qf, ggplot2::aes(x2, f)) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::labs(x = "x2", y = "f(1, x2)")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

qf_x2_ax = np.linspace(-3, 3, 201)
fig, ax = plt.subplots(figsize=(5, 3), layout="constrained")
ax.plot(qf_x2_ax, 0.5 * (1 - qf_x2_ax**2), color="#0072B2")
ax.set_xlabel("x2")
ax.set_ylabel("f(1, x2)")
plt.show();
```



Proof

Proof. $\mathbf{M}^{-1}$ is symmetric and does not depend on $\underset{\sim}{b}$, so derivative of a quadratic form applies with $\mathbf{M}^{-1}$ in place of $\mathbf{S}$:

$$\begin{aligned}
\frac{\partial}{\partial \underset{\sim}{b}} f^*(\underset{\sim}{b}) &= \tfrac{1}{2}\,\frac{\partial}{\partial \underset{\sim}{b}}\big(\underset{\sim}{b}^{\top}\mathbf{M}^{-1}\underset{\sim}{b}\big) && \text{(constant multiple)}\\
&= \tfrac{1}{2}\big(2\mathbf{M}^{-1}\underset{\sim}{b}\big) && \text{(derivative of a quadratic form)}\\
&= \mathbf{M}^{-1}\underset{\sim}{b} && \text{(simplify)}\\
&= -\underset{\sim}{\lambda}^{*}. && \text{(definition of } \underset{\sim}{\lambda}^{*}\text{)}
\end{aligned}$$

□

Example 0.8 (Moving the line)**.** In Example 0.5 with a general right side b in place of 5, $\mathbf{M}^{-1} = \frac{2}{25}$ and

$$\begin{aligned}
f^*(b) &= \tfrac{1}{2}\big(\tfrac{2}{25}\big)\,b^2\\
&= \tfrac{b^2}{25},
\end{aligned}$$

so $\frac{d}{db} f^*(b) = \frac{2b}{25}$. At $b = 5$ this derivative is

$$\begin{aligned}
\tfrac{10}{25} &= 0.4\\
&= -\lambda^* :
\end{aligned}$$

moving the line $3x_1 + 4x_2 = b$ outward by a small step ε in b raises the minimum value by about $0.4\,\varepsilon$.

Newton's method

Source

This section is adapted from the Newton-Raphson and gradient descent parts of Zhou [3], used under the MIT License (see the license text in Linear Algebra). The source motivates the Newton step by minimizing a quadratic approximation; here that step is proved to be the exact minimizer of the quadratic model. In the source's zigzag example the starting value of the objective is $\frac{1}{2}(b^2 + b)$, not $\frac{1}{2}(1 + b^2)$ as the source writes; the ratio of successive values is unaffected. In the same example the source's formula for $\underset{\sim}{x}^{(1)}$ has $\underset{\sim}{x}^{(1)}$ on its right side where $\underset{\sim}{x}^{(0)}$ is meant. These parts of the source are not part of this section:

- the quadratic convergence theorem for Newton's method in general, and its remedies

for instability (a positive definite substitute for the Hessian, and shortening the Newton step by a search along its direction)
- its remark that Hessians are expensive to compute
- the convergence-rate theorem for strongly convex functions
- gradient descent with momentum, and its optimal parameters
- Nesterov acceleration, and stochastic gradient descent
- its code and animations

Definition 0.1 (Iterative method)**.** An **iterative method** [14] starts from a point $\underset{\sim}{x}^{(0)}$ and computes $\underset{\sim}{x}^{(t+1)}$ from $\underset{\sim}{x}^{(t)}$ by a fixed rule, for $t = 0, 1, 2, \ldots$.

Definition 0.2 (Iterate)**.** In an iterative method (Definition 0.1), the points $\underset{\sim}{x}^{(t)}$ are its **iterates**.

Definition 0.3 (Error of an iterate)**.** For a point $\underset{\sim}{x}^*$ that an iterative method (Definition 0.1) is meant to reach, $\left\|\underset{\sim}{x}^{(t)} - \underset{\sim}{x}^*\right\|$ is the **error** of the iterate $\underset{\sim}{x}^{(t)}$ (Definition 0.2).

Definition 0.4 (Convergence of a sequence of vectors)**.** A sequence $\underset{\sim}{x}^{(0)}, \underset{\sim}{x}^{(1)}, \ldots$ in $\mathbb{R}^p$ **converges** to $x^* \in \mathbb{R}^p$ if the sequence of distances $\left\|\underset{\sim}{x}^{(t)} - \underset{\sim}{x}^*\right\|$ converges to 0. For $p = 1$, $\left\|x^{(t)} - x^*\right\| = \left|x^{(t)} - x^*\right|$, and this is the convergence of a sequence of numbers to x^*.

Example 0.1 (Halving a vector)**.** The iterative method $\underset{\sim}{x}^{(t+1)} = \frac{1}{2}\,\underset{\sim}{x}^{(t)}$ from $\underset{\sim}{x}^{(0)} = (4, 2)^\top$ has iterates $(4, 2)^\top, (2, 1)^\top, (1, 0.5)^\top, \ldots$, that is, $\underset{\sim}{x}^{(t)} = 2^{-t}\,(4, 2)^\top$. With $\underset{\sim}{x}^* = \underset{\sim}{0}$, the error of $\underset{\sim}{x}^{(t)}$ is

$$\begin{aligned} \left\|2^{-t}\,(4, 2)^\top - \underset{\sim}{0}\right\| &= 2^{-t}\left\|(4, 2)^\top\right\| && \text{(norm of a multiple)} \\ &= 2^{-t}\sqrt{20}, && (4^2 + 2^2 = 20) \end{aligned}$$

which converges to 0, so the iterates converge to $\underset{\sim}{0}$.

Definition 0.5 (Newton's method for minimization)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first and second partial derivatives, and let $\underset{\sim}{x}^{(0)}$ be a starting point. **Newton's method** [15] computes, for $t = 0, 1, 2, \ldots$, as long as $\mathbf{H}_f(\underset{\sim}{x}^{(t)})$ is invertible,

$$\underset{\sim}{x}^{(t+1)} \overset{\text{def}}{=} \underset{\sim}{x}^{(t)} - \left[\mathbf{H}_f(\underset{\sim}{x}^{(t)})\right]^{-1} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}).$$

Definition 0.6 (Newton step)**.** In Newton's method (Definition 0.5), the vector $-\left[\mathbf{H}_f(\underset{\sim}{x}^{(t)})\right]^{-1}\frac{\partial}{\partial \underset{\sim}{x}}f(\underset{\sim}{x}^{(t)})$ is the **Newton step** at $\underset{\sim}{x}^{(t)}$.

Example 0.2 (Newton's method for a cubic)**.** Let $f(x)=\frac{1}{3}x^3-4x$, so $f'(x)=x^2-4$ and $f''(x)=2x$. For $x^{(t)}\neq 0$, one Newton step gives

$$\begin{aligned} x^{(t+1)} &= x^{(t)}-\frac{(x^{(t)})^2-4}{2x^{(t)}} && \text{(Newton's method with } p=1\text{)}\\ &= \frac{2(x^{(t)})^2-(x^{(t)})^2+4}{2x^{(t)}} && \text{(common denominator)}\\ &= \frac{1}{2}\left(x^{(t)}+\frac{4}{x^{(t)}}\right). && \text{(simplify)} \end{aligned}$$

From $x^{(0)}=2.5$:

- $$\begin{aligned} x^{(1)} &= \tfrac{1}{2}(2.5+1.6)\\ &= 2.05; \end{aligned}$$
- $$\begin{aligned} x^{(2)} &= \tfrac{1}{2}(2.05+1.951219\ldots)\\ &= 2.000610 \end{aligned}$$

  (to six decimals);
- $x^{(3)}=2.0000001$ (to seven decimals).

The iterates (Definition 0.2) approach 2, the local minimizer of f (a stationary point, since $f'(2)=0$, with $f''(2)=4>0$; Theorem 0.5).

Definition 0.7 (Quadratic model, second-order Taylor polynomial)**.** Let f be as in Definition 0.5 and let $\underset{\sim}{x}$ be a point, with $g\overset{\text{def}}{=}\frac{\partial}{\partial \underset{\sim}{x}}f(\underset{\sim}{x})$ and $\mathbf{H}\overset{\text{def}}{=}\mathbf{H}_f(\underset{\sim}{x})$. The **quadratic model** of f at $\underset{\sim}{x}$ is the function of a step $\underset{\sim}{\delta}\in\mathbb{R}^p$

$$q(\underset{\sim}{\delta})\overset{\text{def}}{=}f(\underset{\sim}{x})+\underset{\sim}{g}\cdot\underset{\sim}{\delta}+\tfrac{1}{2}\,\underset{\sim}{\delta}^{\top}\mathbf{H}\underset{\sim}{\delta},$$

also called the **second-order Taylor polynomial** of f at $\underset{\sim}{x}$.

Example 0.3 (The model at $x=2.5$)**.** For $f(x)=\frac{1}{3}x^3-4x$ at $x=2.5$ (Example 0.2),

$$g = 2.5^2 - 4$$
$$= 2.25$$

and

$$H = [2 \cdot 2.5]$$
$$= [5],$$

so $q(\delta) = f(2.5) + 2.25\,\delta + 2.5\,\delta^2$. For $f(\underset{\sim}{x}) = \frac{1}{2}(x_1^2 + 3x_2^2)$, whose Hessian is $\begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$ at every point, the model at any $\underset{\sim}{x}$ equals $f(\underset{\sim}{x} + \underset{\sim}{\delta})$ exactly (Example 0.6 with $\mathbf{S} = \begin{bmatrix} 1/2 & 0 \\ 0 & 3/2 \end{bmatrix}$).

Remark (The quadratic model and Taylor's theorem)

Remark 0.1 (The quadratic model and Taylor's theorem). When the second partial derivatives of f are continuous, Theorem 0.4 says that $f(\underset{\sim}{x} + \underset{\sim}{\delta})$ equals the expression for $q(\underset{\sim}{\delta})$ in Definition 0.7 with the Hessian evaluated at a point between $\underset{\sim}{x}$ and $\underset{\sim}{x} + \underset{\sim}{\delta}$.

Theorem 0.1 (The Newton step minimizes the quadratic model)**.** *Let q be the quadratic model of f at $\underset{\sim}{x}$ (Definition 0.7), with $\mathbf{H} = \mathbf{H}_f(\underset{\sim}{x})$ positive definite. Its unique minimizer is the Newton step $\underset{\sim}{\delta}^* = -\mathbf{H}^{-1}\underset{\sim}{g}$.*

Proof

Proof. $\mathbf{H}$ is symmetric, by the definition of positive definite, and invertible, so $\underset{\sim}{\delta}^*$ exists and $\mathbf{H}\underset{\sim}{\delta}^* = -\underset{\sim}{g}$. For any $\underset{\sim}{d} \in \mathbb{R}^p$, first

$$\begin{aligned} \underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{\delta}^* &= -\underset{\sim}{d} \cdot \underset{\sim}{g} && (\mathbf{H}\underset{\sim}{\delta}^* = -\underset{\sim}{g}) \\ &= -\underset{\sim}{g} \cdot \underset{\sim}{d}, && (\text{a } 1 \times 1 \text{ matrix equals its transpose}) \end{aligned}$$

and $\underset{\sim}{\delta}^{*\top} \mathbf{H} \underset{\sim}{d} = \underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{\delta}^*$ in the same way, using $\mathbf{H}^\top = \mathbf{H}$. Then

$$\begin{aligned} q(\underset{\sim}{\delta}^* + \underset{\sim}{d}) &= f(\underset{\sim}{x}) + \underset{\sim}{g} \cdot (\underset{\sim}{\delta}^* + \underset{\sim}{d}) + \tfrac{1}{2}\,(\underset{\sim}{\delta}^* + \underset{\sim}{d})^\top \mathbf{H} (\underset{\sim}{\delta}^* + \underset{\sim}{d}) && (\text{definition of } q) \\ &= f(\underset{\sim}{x}) + \underset{\sim}{g} \cdot \underset{\sim}{\delta}^* + \underset{\sim}{g} \cdot \underset{\sim}{d} + \tfrac{1}{2}\left(\underset{\sim}{\delta}^{*\top} \mathbf{H} \underset{\sim}{\delta}^* + \underset{\sim}{\delta}^{*\top} \mathbf{H} \underset{\sim}{d} + \underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{\delta}^* + \underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{d}\right) && (\text{distribute}) \\ &= f(\underset{\sim}{x}) + \underset{\sim}{g} \cdot \underset{\sim}{\delta}^* + \underset{\sim}{g} \cdot \underset{\sim}{d} + \tfrac{1}{2}\,\underset{\sim}{\delta}^{*\top} \mathbf{H} \underset{\sim}{\delta}^* - \underset{\sim}{g} \cdot \underset{\sim}{d} + \tfrac{1}{2}\,\underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{d} && (\text{both cross terms equal } -\underset{\sim}{g} \cdot \underset{\sim}{d}) \\ &= f(\underset{\sim}{x}) + \underset{\sim}{g} \cdot \underset{\sim}{\delta}^* + \tfrac{1}{2}\,\underset{\sim}{\delta}^{*\top} \mathbf{H} \underset{\sim}{\delta}^* + \tfrac{1}{2}\,\underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{d} && (\text{cancel } \underset{\sim}{g} \cdot \underset{\sim}{d}) \\ &= q(\underset{\sim}{\delta}^*) + \tfrac{1}{2}\,\underset{\sim}{d}^\top \mathbf{H} \underset{\sim}{d}, && (\text{definition of } q) \end{aligned}$$

which is greater than $q(\underset{\sim}{\delta}^*)$ for every $\underset{\sim}{d} \neq \underset{\sim}{0}_{p\times 1}$, since $\mathbf{H}$ is positive definite. □

Example 0.4 (The Newton step at $x = 2.5$)**.** For the model $q(\delta) = f(2.5) + 2.25\,\delta + 2.5\,\delta^2$ of Example 0.3, $H = [5]$ is positive definite, so by Theorem 0.1 its minimizer is

$$\begin{aligned} \delta^* &= -H^{-1} g \\ &= -2.25/5 \\ &= -0.45. \end{aligned}$$

Setting the derivative $2.25 + 5\delta$ to 0 gives the same δ^*, and

$$\begin{aligned} 2.5 - 0.45 &= 2.05 \\ &= x^{(1)} \end{aligned}$$

(Example 0.2).

Definition 0.8 (Linear approximation of the gradient)**.** Let f be as in Newton's method and let $\underset{\sim}{x}$ be a point. The **linear approximation of the gradient** of f at $\underset{\sim}{x}$ is the $p \times 1$ vector-valued function $\underset{\sim}{\ell}$ of a step $\underset{\sim}{\delta} \in \mathbb{R}^p$ whose jth entry, for $j = 1, \ldots, p$, is the function whose graph is the tangent plane to the graph of the partial derivative $\frac{\partial}{\partial x_j} f$ at $\underset{\sim}{x}$, evaluated at $\underset{\sim}{x} + \underset{\sim}{\delta}$:

$$\left[\underset{\sim}{\ell}(\underset{\sim}{\delta})\right]_j \stackrel{\text{def}}{=} \frac{\partial}{\partial x_j} f(\underset{\sim}{x}) + \left(\frac{\partial}{\partial \underset{\sim}{x}} \left(\frac{\partial}{\partial x_j} f(\underset{\sim}{x}) \right) \right)^{\top} \underset{\sim}{\delta}. \tag{4}$$

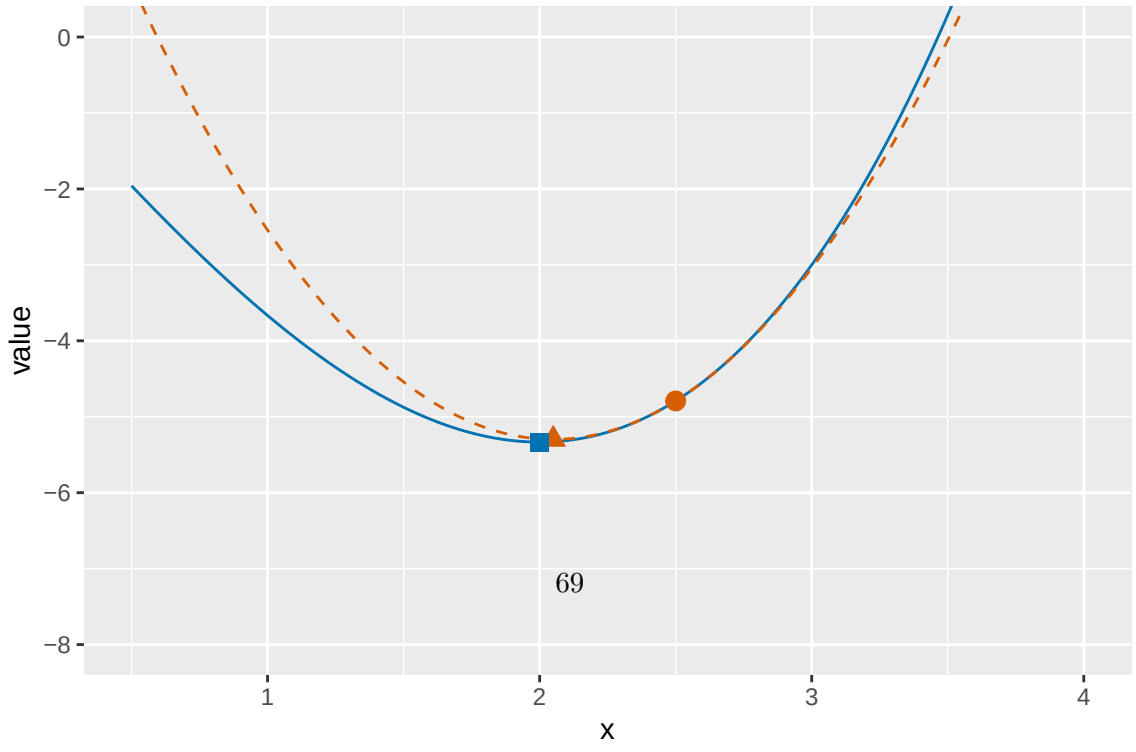
Example 0.5 (The linear approximation of the derivative of a cubic at $x = -2.5$)**.** For $f(x) = \frac{1}{3}x^3 - 4x$, with $f'(x) = x^2 - 4$ and $f''(x) = 2x$ (Example 0.2), $p = 1$, so the linear approximation of the gradient at $x = -2.5$ is the tangent line to f' there:

$$\begin{aligned} \ell(\delta) &= f'(-2.5) + f''(-2.5)\,\delta && \text{(Equation 4 with } p = 1) \\ &= ((-2.5)^2 - 4) + 2(-2.5)\,\delta && \text{(substitute } f' \text{ and } f'') \\ &= 2.25 - 5\delta. && \text{(arithmetic)} \end{aligned} \tag{5}$$

Figure 23 shows f' and ℓ.

R

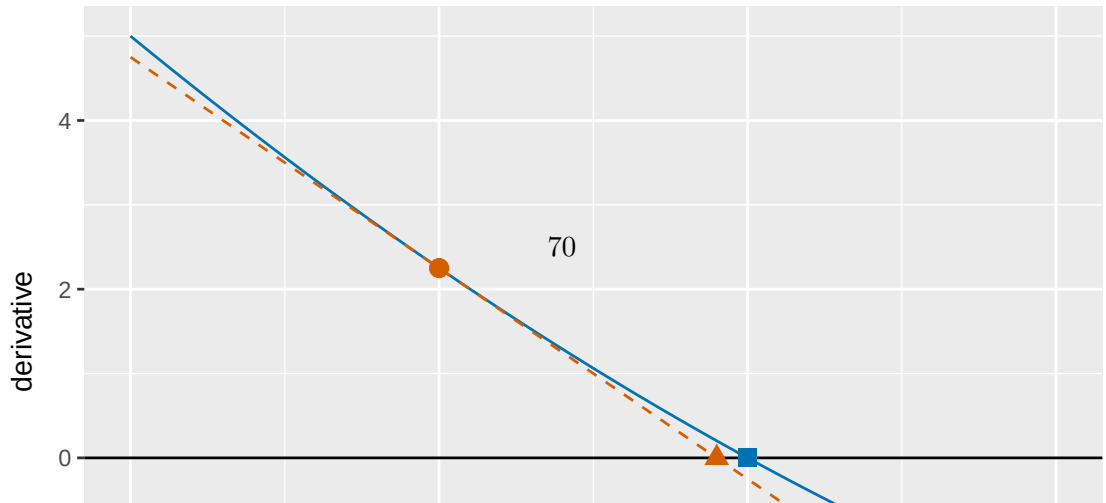
```r
nm_x_nm <- seq(0.5, 4, length.out = 301)
nm_f_nm <- nm_x_nm^3 / 3 - 4 * nm_x_nm
nm_f0_nm <- 2.5^3 / 3 - 4 * 2.5
nm_q_nm <- nm_f0_nm + 2.25 * (nm_x_nm - 2.5) + 2.5 * (nm_x_nm - 2.5)^2
nm_q_step_nm <- nm_f0_nm + 2.25 * (2.05 - 2.5) + 2.5 * (2.05 - 2.5)^2
nm_f_min_nm <- 2^3 / 3 - 4 * 2
nm_curve_nm <- tibble::tibble(x = nm_x_nm, y = nm_f_nm)
nm_model_nm <- tibble::tibble(x = nm_x_nm, y = nm_q_nm)
nm_start_nm <- tibble::tibble(x = 2.5, y = nm_f0_nm)
nm_step_nm <- tibble::tibble(x = 2.05, y = nm_q_step_nm)
nm_min_nm <- tibble::tibble(x = 2, y = nm_f_min_nm)
nm_aes_nm <- ggplot2::aes(x, y)
nm_ylim_nm <- c(-8, 0)
ggplot2::ggplot(nm_curve_nm, nm_aes_nm) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_line(
    data = nm_model_nm, colour = "#D55E00", linetype = "dashed"
  ) +
  ggplot2::geom_point(data = nm_start_nm, colour = "#D55E00", size = 3) +
  ggplot2::geom_point(
    data = nm_step_nm, colour = "#D55E00", size = 3, shape = 17
  ) +
  ggplot2::geom_point(
    data = nm_min_nm, colour = "#0072B2", size = 3, shape = 15
  ) +
  ggplot2::coord_cartesian(ylim = nm_ylim_nm) +
  ggplot2::labs(x = "x", y = "value")
```



Python

R

```
gla_x <- seq(-3, -1.5, length.out = 301)
gla_deriv <- gla_x^2 - 4
gla_line <- 2.25 - 5 * (gla_x + 2.5)
gla_curve <- tibble::tibble(x = gla_x, y = gla_deriv)
gla_tangent <- tibble::tibble(x = gla_x, y = gla_line)
gla_start <- tibble::tibble(x = -2.5, y = 2.25)
gla_step <- tibble::tibble(x = -2.05, y = 0)
gla_root <- tibble::tibble(x = -2, y = 0)
gla_ylim <- c(-2, 5)
gla_curve |>
  ggplot2::ggplot() +
  ggplot2::aes(x = x, y = y) +
  ggplot2::geom_hline(yintercept = 0) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_line(
    data = gla_tangent, colour = "#D55E00", linetype = "dashed"
  ) +
  ggplot2::geom_point(data = gla_start, colour = "#D55E00", size = 3) +
  ggplot2::geom_point(
    data = gla_step, colour = "#D55E00", size = 3, shape = 17
  ) +
  ggplot2::geom_point(
    data = gla_root, colour = "#0072B2", size = 3, shape = 15
  ) +
  ggplot2::coord_cartesian(ylim = gla_ylim) +
  ggplot2::labs(x = "x", y = "derivative")
```



Example 0.6 (The tangent line to f is not the linear approximation of the gradient)**.** For the cubic $f(x) = \frac{1}{3}x^3 - 4x$ at $x = -2.5$ (Example 0.5), the tangent line to f itself is

$$\begin{aligned} f(-2.5) + f'(-2.5)\,\delta &= \left(\tfrac{1}{3}(-2.5)^3 - 4(-2.5)\right) + \left((-2.5)^2 - 4\right)\delta && \text{(substitute } f \text{ and } f'\text{)} \\ &= \tfrac{115}{24} + 2.25\,\delta. && \text{(arithmetic)} \end{aligned} \tag{6}$$

It is not the linear approximation of the gradient: it approximates f, not f'. Its root, $\delta = -\frac{115}{54}$, is a step toward a zero of f, not toward a stationary point of f.

Lemma 0.1 (The linear approximation of the gradient uses the Hessian)**.** *Let f, $\underset{\sim}{x}$, the gradient $\underset{\sim}{g}$ and the Hessian $\mathbf{H}$ be as in the quadratic model, and let the second partial derivatives of f be continuous at every point of an open ball around $\underset{\sim}{x}$. Then the linear approximation of the gradient of f at $\underset{\sim}{x}$ is the gradient plus the Hessian times the step:*

$$\underset{\sim}{\ell}(\underset{\sim}{\delta}) = \underset{\sim}{g} + \mathbf{H}\underset{\sim}{\delta}. \tag{7}$$

Proof

Proof. For each i and j, the ith entry of $\frac{\partial}{\partial \underset{\sim}{x}}\left(\frac{\partial}{\partial x_j} f(\underset{\sim}{x})\right)$ is $\frac{\partial}{\partial x_i}\left(\frac{\partial}{\partial x_j} f(\underset{\sim}{x})\right)$, which is $[\mathbf{H}]_{ij}$ by the definitions of the Hessian and of the derivative of a vector-valued function. So, for each j,

$$\begin{aligned} [\underset{\sim}{\ell}(\underset{\sim}{\delta})]_j &= \frac{\partial}{\partial x_j} f(\underset{\sim}{x}) + \sum_{i=1}^{p} [\mathbf{H}]_{ij}\,\delta_i && \text{(Equation 4, as a sum)} \\ &= [\underset{\sim}{g}]_j + \sum_{i=1}^{p} [\mathbf{H}]_{ij}\,\delta_i && \text{(definition of } \underset{\sim}{g}\text{)} \\ &= [\underset{\sim}{g}]_j + \sum_{i=1}^{p} [\mathbf{H}]_{ji}\,\delta_i && (\mathbf{H} \text{ is symmetric}) \\ &= [\underset{\sim}{g}]_j + [\mathbf{H}\underset{\sim}{\delta}]_j && \text{(matrix-vector product)} \\ &= [\underset{\sim}{g} + \mathbf{H}\underset{\sim}{\delta}]_j. && \text{(vector addition)} \end{aligned} \tag{8}$$

The third line uses the symmetry of the Hessian, whose hypothesis the lemma assumes. □

Theorem 0.2 (The Newton step is the root of the linear approximation of the gradient)**.** *Let f, $\underset{\sim}{x}$, g and the Hessian $\mathbf{H}$ be as in Lemma 0.1, with $\mathbf{H}$ invertible. The only step $\underset{\sim}{\delta}$ at which the linear approximation of the gradient is zero is the Newton step:*

$$\underset{\sim}{\ell}(\underset{\sim}{\delta}) = \underset{\sim}{0}_{p\times 1} \iff \underset{\sim}{\delta} = -\mathbf{H}^{-1}\underset{\sim}{g}. \tag{9}$$

Proof

Proof. By Lemma 0.1, $\underset{\sim}{\ell}(\underset{\sim}{\delta}) = \underset{\sim}{0}_{p\times 1}$ exactly when

$$\begin{aligned} \underset{\sim}{0}_{p\times 1} &= \underset{\sim}{g} + \mathbf{H}\underset{\sim}{\delta} && \text{(Equation 7)} \\ -\underset{\sim}{g} &= \mathbf{H}\underset{\sim}{\delta} && \text{(subtract } \underset{\sim}{g} \text{ from both sides)} \\ -\mathbf{H}^{-1}\underset{\sim}{g} &= \underset{\sim}{\delta}. && \text{(multiply both sides by } \mathbf{H}^{-1} \text{ on the left)} \end{aligned} \tag{10}$$

Each line implies the next and, because $\mathbf{H}$ is invertible, the one before, so $\underset{\sim}{\delta} = -\mathbf{H}^{-1}\underset{\sim}{g}$ is the only solution. □

Example 0.7 (The Newton step at $x = -2.5$ as a root)**.** At $x = -2.5$, the linear approximation of the gradient of the cubic $f(x) = \frac{1}{3}x^3 - 4x$ is $\ell(\delta) = 2.25 - 5\delta$ (Example 0.5). Setting it to 0,

$$\begin{aligned} 0 &= 2.25 - 5\delta && \text{(Equation 5)} \\ 5\delta &= 2.25 && \text{(add } 5\delta \text{ to both sides)} \\ \delta &= 0.45, && \text{(divide both sides by 5)} \end{aligned} \tag{11}$$

so the next iterate is $-2.5 + 0.45 = -2.05$, the point where the tangent line in Figure 23 crosses 0. The Newton's method iteration $x^{(t+1)} = \frac{1}{2}\left(x^{(t)} + 4/x^{(t)}\right)$ of Example 0.2 gives the same iterate. Here $H = [-5]$ is not positive definite, so Theorem 0.1 does not describe this step, but Theorem 0.2 does.

Remark (Two ways to read the Newton step)

Remark 0.2 (Two ways to read the Newton step). Theorem 0.1 reads the Newton step as the minimizer of the quadratic model, which needs the Hessian $\mathbf{H}$ to be positive definite. Theorem 0.2 reads it as the root of the linear approximation of the gradient, which needs only $\mathbf{H}$ invertible. Newton's method solves $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) = \underset{\sim}{0}_{p\times 1}$ by replacing the gradient with its linear approximation at each iterate, so it looks for a stationary point of any kind, not only a minimizer.

Definition 0.9 (Quadratic convergence)**.** A sequence $\underset{\sim}{x}^{(0)}, \underset{\sim}{x}^{(1)}, \ldots$ in $\mathbb{R}^p$ that converges to $\underset{\sim}{x}^*$ **converges quadratically** [16] if there is a constant C with

$$\left\|\underset{\sim}{x}^{(t+1)} - \underset{\sim}{x}^*\right\| \le C\left\|\underset{\sim}{x}^{(t)} - \underset{\sim}{x}^*\right\|^2 \qquad \text{for every } t.$$

Example 0.8 (Halving the error is not quadratic convergence)**.** $x^{(t)} = 2^{-t}$ converges to 0, but it does not converge quadratically: the bound would need $2^{-(t+1)} \le C\,(2^{-t})^2 = C\,2^{-2t}$, that is, $2^{t-1} \le C$, which fails once $t > 1 + \log_2 C$. Each step only halves the error.

Example 0.9 (The error squares at each step)**.** For the iteration $x^{(t+1)} = \frac{1}{2}(x^{(t)} + 4/x^{(t)})$ of Example 0.2,

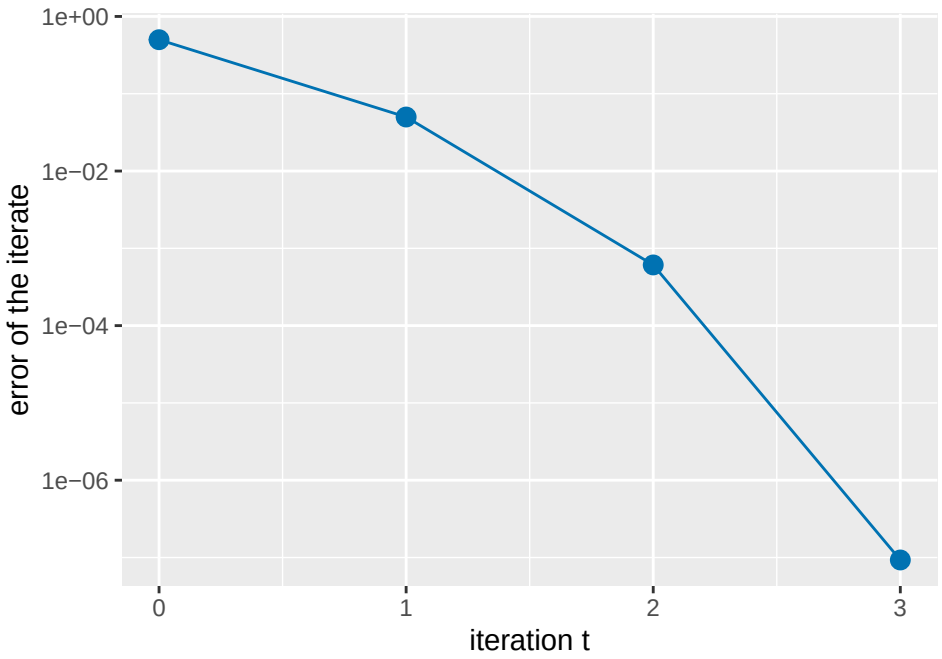
$$\begin{aligned} x^{(t+1)} - 2 &= \frac{1}{2}\left(x^{(t)} + \frac{4}{x^{(t)}}\right) - 2 && \text{(the iteration)} \\ &= \frac{(x^{(t)})^2 + 4 - 4x^{(t)}}{2x^{(t)}} && \text{(common denominator)} \\ &= \frac{(x^{(t)} - 2)^2}{2x^{(t)}}. && \text{(the numerator is a perfect square)} \end{aligned}$$

So near 2 the error is roughly squared and divided by 4 at each step: the errors are 0.05, then 0.00061, then 0.000000093, and $0.05^2/(2 \cdot 2.05) \approx 0.00061$. The iterates converge quadratically (Definition 0.9): $C = \frac{1}{2}$ works for every $x^{(t)} \ge 1$, since then $\frac{1}{2x^{(t)}} \le \frac{1}{2}$, and every iterate from 2.5 is at least 2, since the identity makes $x^{(t+1)} - 2 \ge 0$ whenever $x^{(t)} > 0$. Zhou [3] states such a bound for Newton's method in general; it holds from starting points close enough to a minimizer where the Hessian is positive definite, when near that minimizer the Hessian changes by at most a constant times the distance moved, which is not proved here.

Example 0.10 (Newton's method can find a maximum)**.** From $x^{(0)} = -2.5$ the same iteration gives -2.05, -2.000610, -2.0000001, approaching -2. But -2 is not a minimizer of f: it is a stationary point of $-f$, whose second derivative there is $-f''(-2) = 4 > 0$, so it is a strict local minimizer of $-f$ (Theorem 0.5), that is, f is strictly larger at -2 than at all nearby points. Newton's method looks only for a stationary point, and here the Hessian $f''(x) = 2x$ is negative along the way, so the quadratic model has no minimizer (with $H < 0$, $q(\delta) = f + g\,\delta + \frac{1}{2}H\delta^2$ decreases without bound) and Theorem 0.1 does not apply: the positive definite hypothesis there is needed.
Figure 25 shows the cubic and its model.

R

```
nc_x_nc <- numeric(4)
nc_x_nc[1] <- 2.5
for (nc_t_nc in 1:3) {
  nc_prev_nc <- nc_x_nc[nc_t_nc]
  nc_x_nc[nc_t_nc + 1] <- 0.5 * (nc_prev_nc + 4 / nc_prev_nc)
}
nc_steps_nc <- tibble::tibble(t = 0:3, error = nc_x_nc - 2)
ggplot2::ggplot(nc_steps_nc, ggplot2::aes(t, error)) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(colour = "#0072B2", size = 3) +
  ggplot2::scale_y_log10() +
  ggplot2::labs(x = "iteration t", y = "error of the iterate")
```



Python

```
import matplotlib.pyplot as plt

nc_x_ax = [2.5]
for _ in range(3):
    nc_x_ax.append(0.5 * (nc_x_ax[-1] + 4 / nc_x_ax[-1]))
nc_err_ax = [x - 2 for x in nc_x_ax]
fig, ax = plt.subplots(figsize=(5, 3.5), layout="constrained")
ax.semilogy(range(4), nc_err_ax, "o-", color="#0072B2")
#> [<matplotlib.lines.Line2D object at 0x7fa978117610>]
ax.set_xlabel("iteration t")
#> Text(0.5, 0, 'iteration t')
ax.set_ylabel("error of the iterate")
#> Text(0, 0.5, 'error of the iterate')
ax.set_xticks(range(4))
#> [<matplotlib.axis.XTick object at 0x7fa9784d5450>, <matplotlib.axis.XTick object at 0x7fa9
plt.show();
```

R

```
nx_x_nx <- seq(-4, -0.5, length.out = 301)
nx_f_nx <- nx_x_nx^3 / 3 - 4 * nx_x_nx
nx_f0_nx <- (-2.5)^3 / 3 - 4 * (-2.5)
nx_dx_nx <- nx_x_nx + 2.5
nx_q_nx <- nx_f0_nx + 2.25 * nx_dx_nx - 2.5 * nx_dx_nx^2
nx_dstep_nx <- -2.05 + 2.5
nx_q_step_nx <- nx_f0_nx + 2.25 * nx_dstep_nx - 2.5 * nx_dstep_nx^2
nx_f_max_nx <- (-2)^3 / 3 - 4 * (-2)
nx_curve_nx <- tibble::tibble(x = nx_x_nx, y = nx_f_nx)
nx_model_nx <- tibble::tibble(x = nx_x_nx, y = nx_q_nx)
nx_start_nx <- tibble::tibble(x = -2.5, y = nx_f0_nx)
nx_step_nx <- tibble::tibble(x = -2.05, y = nx_q_step_nx)
nx_max_nx <- tibble::tibble(x = -2, y = nx_f_max_nx)
nx_aes_nx <- ggplot2::aes(x, y)
nx_ylim_nx <- c(-2, 6.5)
nx_curve_nx |>
  ggplot2::ggplot() +
  nx_aes_nx +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_line(
    data = nx_model_nx, colour = "#D55E00", linetype = "dashed"
  ) +
  ggplot2::geom_point(data = nx_start_nx, colour = "#D55E00", size = 3) +
  ggplot2::geom_point(
```

Theorem 0.3 (Newton's method minimizes a quadratic in one step)**.** *Let* $\mathbf{S}$ *be a* $p \times p$ *positive definite matrix,* $\underset{\sim}{c} \in \mathbb{R}^p$*, and* $f(\underset{\sim}{x}) = \frac{1}{2}\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x} - \underset{\sim}{c} \cdot \underset{\sim}{x}$*. From any* $\underset{\sim}{x}^{(0)}$*, Newton's method (Definition 0.5) gives* $\underset{\sim}{x}^{(1)} = \mathbf{S}^{-1}\underset{\sim}{c}$*, the unique minimizer of* f*.*

Proof

Proof. Partial derivatives of a sum or of a constant multiple are the sum or multiple of the partial derivatives, so the derivative of a quadratic form applies to $\frac{1}{2}\underset{\sim}{x}^\top \mathbf{S}\underset{\sim}{x}$ and the derivative of a linear combination to $\underset{\sim}{c} \cdot \underset{\sim}{x}$:

$$\begin{aligned}\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) &= \tfrac{1}{2}(2\mathbf{S}\underset{\sim}{x}) - \underset{\sim}{c} \\ &= \mathbf{S}\underset{\sim}{x} - \underset{\sim}{c};\end{aligned}$$

by the Hessian of a quadratic form with $\frac{1}{2}\mathbf{S}$ in place of $\mathbf{S}$, and because the gradient $-\underset{\sim}{c}$ of the linear term is constant, $\mathbf{H}_f(\underset{\sim}{x}) = \mathbf{S}$, which is invertible. So

$$\begin{aligned}\underset{\sim}{x}^{(1)} &= \underset{\sim}{x}^{(0)} - \mathbf{S}^{-1}(\mathbf{S}\underset{\sim}{x}^{(0)} - \underset{\sim}{c}) && \text{(definition of Newton's method)} \\ &= \underset{\sim}{x}^{(0)} - \mathbf{S}^{-1}\mathbf{S}\underset{\sim}{x}^{(0)} + \mathbf{S}^{-1}\underset{\sim}{c} && \text{(distribute)} \\ &= \underset{\sim}{x}^{(0)} - \underset{\sim}{x}^{(0)} + \mathbf{S}^{-1}\underset{\sim}{c} && (\mathbf{S}^{-1}\mathbf{S} = \mathbf{I}_p) \\ &= \mathbf{S}^{-1}\underset{\sim}{c}. && \text{(cancel)}\end{aligned}$$

The gradient at $\mathbf{S}^{-1}\underset{\sim}{c}$ is $\mathbf{S}\mathbf{S}^{-1}\underset{\sim}{c} - \underset{\sim}{c} = \underset{\sim}{0}_{p\times 1}$, and f is strictly convex (Theorem 0.5, part 2), so $\mathbf{S}^{-1}\underset{\sim}{c}$ is a global minimizer (Corollary 0.1). It is the only one: a strictly convex function has at most one global minimizer, since the midpoint of two distinct global minimizers, which have equal values, would have a strictly smaller value (Definition 0.3 with $t = \frac{1}{2}$). □

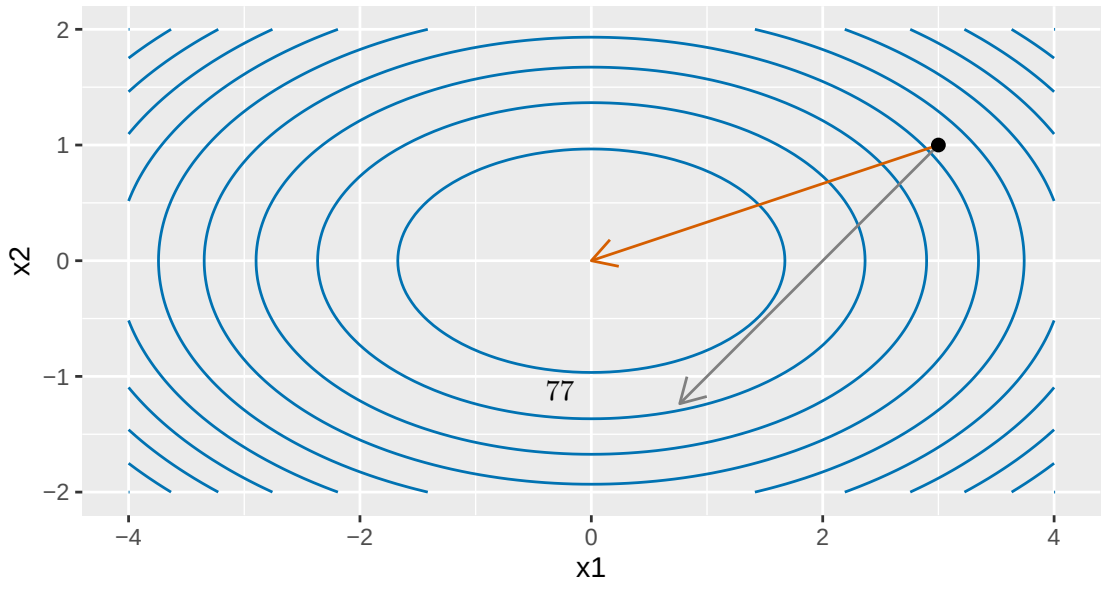
Example 0.11 (One step to the minimizer)**.** Let $f(\underset{\sim}{x}) = \frac{1}{2}(x_1^2 + 3x_2^2)$, so $\mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$ and $\underset{\sim}{c} = \underset{\sim}{0}_{2\times 1}$. From $\underset{\sim}{x}^{(0)} = (3,1)^\top$ the gradient is $\nabla f(\underset{\sim}{x}^{(0)}) = \mathbf{S}\underset{\sim}{x}^{(0)} = (3,3)^\top$, and $\mathbf{S}^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1/3 \end{bmatrix}$, so the Newton step is $-\mathbf{S}^{-1}(3,3)^\top = -(3,1)^\top$ and

$$\begin{aligned}\underset{\sim}{x}^{(1)} &= (3,1)^\top - (3,1)^\top \\ &= \underset{\sim}{0}_{2\times 1},\end{aligned}$$

the minimizer.
Figure 26 shows the step on the level curves of f.

R

```
nq_x1_nq <- seq(-4, 4, length.out = 101)
nq_x2_nq <- seq(-2, 2, length.out = 101)
nq_grid_nq <- tidyr::expand_grid(x1 = nq_x1_nq, x2 = nq_x2_nq)
nq_grid_nq$z <- (nq_grid_nq$x1^2 + 3 * nq_grid_nq$x2^2) / 2
nq_newton_nq <- tibble::tibble(x = 3, y = 1, xend = 0, yend = 0)
nq_grad_nq <- tibble::tibble(x = 3, y = 1, xend = 0.764, yend = -1.236)
nq_start_nq <- tibble::tibble(x1 = 3, x2 = 1)
nq_head_nq <- grid::unit(0.15, "inches")
nq_arrow_nq <- grid::arrow(length = nq_head_nq)
nq_aes_nq <- ggplot2::aes(x1, x2)
nq_z_aes_nq <- ggplot2::aes(z = z)
nq_seg_aes_nq <- ggplot2::aes(x = x, y = y, xend = xend, yend = yend)
nq_grid_nq |>
  ggplot2::ggplot() +
  nq_aes_nq +
  ggplot2::geom_contour(nq_z_aes_nq, colour = "#0072B2", bins = 10) +
  ggplot2::geom_segment(
    nq_seg_aes_nq,
    data = nq_grad_nq,
    colour = "grey50",
    arrow = nq_arrow_nq
  ) +
  ggplot2::geom_segment(
    nq_seg_aes_nq,
    data = nq_newton_nq,
    colour = "#D55E00",
    arrow = nq_arrow_nq
  ) +
  ggplot2::geom_point(data = nq_start_nq, size = 2) +
  ggplot2::coord_fixed() +
  ggplot2::labs(x = "x1", y = "x2")
```



> Figure 27 shows the same function as a surface.

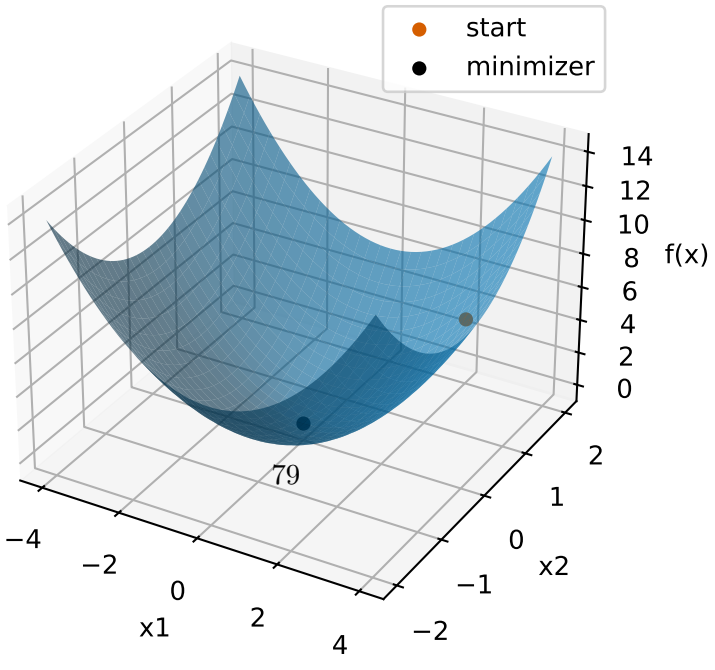
R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

nq_x1_3d, nq_x2_3d = np.meshgrid(np.linspace(-4, 4, 61),
                                 np.linspace(-2, 2, 61))
nq_z_3d = (nq_x1_3d**2 + 3 * nq_x2_3d**2) / 2
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(nq_x1_3d, nq_x2_3d, nq_z_3d, color="#0072B2", alpha=0.6)
#> <mpl_toolkits.mplot3d.art3d.Poly3DCollection object at 0x7fa977fbd590>
ax.scatter([3], [1], [6], color="#D55E00", label="start")
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa977fbd6d0>
ax.scatter([0], [0], [0], color="black", label="minimizer")
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa977fbd810>
ax.set_xlabel("x1")
#> Text(0.5, 0, 'x1')
ax.set_ylabel("x2")
#> Text(0.5, 0.5, 'x2')
ax.set_zlabel("f(x)")
#> Text(0.5, 0, 'f(x)')
ax.legend(loc="upper right")
#> <matplotlib.legend.Legend object at 0x7fa977fbd950>
plt.show();
```



Gradient descent

Definition 0.1 (Gradient descent)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x}^{(0)}$ be a starting point, and let $s^{(0)}, s^{(1)}, \ldots$ be positive **step lengths**. **Gradient descent** [17] computes, for $t = 0, 1, 2, \ldots$,

$$\underset{\sim}{x}^{(t+1)} \stackrel{\text{def}}{=} \underset{\sim}{x}^{(t)} - s^{(t)} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}).$$

Example 0.1 (Fixed steps on a parabola)**.** For $f(x) = x^2$, $f'(x) = 2x$, so gradient descent with fixed step length s is

$$\begin{aligned} x^{(t+1)} &= x^{(t)} - 2s\, x^{(t)} \\ &= (1 - 2s)\, x^{(t)}, \end{aligned}$$

and $x^{(t)} = (1 - 2s)^t x^{(0)}$.

- With $s = 0.25$, $x^{(t)} = 0.5^t x^{(0)}$, which tends to the minimizer 0.
- With $s = 0.5$, the first step lands on 0, since $1 - 2s = 0$.
- With $s = 1.5$, $x^{(t)} = (-2)^t x^{(0)}$, which moves away from 0 for any $x^{(0)} \neq 0$: too long a step can make gradient descent fail.

Remark (Gradient descent as Newton's method with a simpler Hessian)

Remark 0.1 (Gradient descent as Newton's method with a simpler Hessian). Gradient descent (Definition 0.1) is Newton's method (Definition 0.5) with the Hessian replaced by $\frac{1}{s^{(t)}} \mathbf{I}_p$, so it needs no second derivatives.

Definition 0.2 (Gradient ascent)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x}^{(0)}$ be a starting point, and let $s^{(0)}, s^{(1)}, \ldots$ be positive step lengths. **Gradient ascent** [17] repeats, for $t = 0, 1, 2, \ldots$, the update

$$\underset{\sim}{x}^{(t+1)} \stackrel{\text{def}}{=} \underset{\sim}{x}^{(t)} + s^{(t)} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}).$$

Example 0.2 (Gradient ascent is gradient descent on the negative function)**.** Gradient ascent on f takes the same steps as gradient descent on $-f$, since $\frac{\partial}{\partial \underset{\sim}{x}}(-f) = -\frac{\partial}{\partial \underset{\sim}{x}} f$. In one variable, for $f(x) = -x^2$ the derivative is $f'(x) = -2x$, so gradient ascent with fixed step length s is $x^{(t+1)} = (1 - 2s)\, x^{(t)}$, the iteration of Example 0.1, which tends to the maximizer 0 when $0 < s < 1$.

Definition 0.3 (Line search)**.** In gradient descent (Definition 0.1), a **line search** [18] is a rule that chooses the step length $s^{(t)}$ by examining the values of f along the line $\{\underset{\sim}{x}^{(t)} - s\,\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}) : s > 0\}$.

Definition 0.4 (Exact line search)**. Exact line search** is the line search (Definition 0.3) that chooses $s^{(t)}$ to minimize $f(\underset{\sim}{x}^{(t)} - s\,\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}))$ over $s > 0$.

Example 0.3 (Exact line search on a parabola)**.** For $f(x) = x^2$ and $x^{(t)} = 2$, the derivative is $f'(2) = 4$, so the line is $\{2 - 4s : s > 0\}$ and the values along it are

$$\begin{aligned} \phi(s) &= f(2 - 4s) && \text{(the value at step length } s\text{)} \\ &= (2 - 4s)^2 && (f(x) = x^2) \end{aligned}$$

$\phi(s) \geq 0$ for every s, and

$$\begin{aligned} \phi(0.5) &= (2 - 2)^2 \\ &= 0, \end{aligned}$$

so exact line search chooses $s^{(t)} = 0.5$, which lands on the minimizer $x^{(t+1)} = 0$ in one step.

Definition 0.5 (Descent direction)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have a gradient $\underset{\sim}{g} \stackrel{\text{def}}{=} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$ at $\underset{\sim}{x}$. A direction $\underset{\sim}{d}$ is a **descent direction** for f at $\underset{\sim}{x}$ if $\underset{\sim}{g} \cdot \underset{\sim}{d} < 0$. When $\underset{\sim}{g} \neq \underset{\sim}{0}_{p\times 1}$, $-\underset{\sim}{g}$ is a descent direction, since $\underset{\sim}{g} \cdot (-\underset{\sim}{g}) = -\|\underset{\sim}{g}\|^2 < 0$.

Example 0.4 (Descent directions for $x_1^2 + x_2^2$ at $(1,0)^\top$)**.** Let $f(\underset{\sim}{x}) = x_1^2 + x_2^2$ and $\underset{\sim}{x} = (1,0)^\top$, so

$$\begin{aligned} \underset{\sim}{g} &= (2x_1,\ 2x_2)^\top \\ &= (2,0)^\top. \end{aligned}$$

- $\underset{\sim}{d} = (-1,1)^\top$ is a descent direction:

$$\begin{aligned} \underset{\sim}{g} \cdot \underset{\sim}{d} &= -2 + 0 \\ &= -2 \\ &< 0. \end{aligned}$$

- $\underset{\sim}{d} = (0,1)^\top$ is not: $\underset{\sim}{g} \cdot \underset{\sim}{d} = 0$.
- $-\underset{\sim}{g} = (-2,0)^\top$ is: $\underset{\sim}{g} \cdot (-\underset{\sim}{g}) = -4 < 0$.

Theorem 0.1 (A short enough step downhill decreases the function)**.** *Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives that are continuous on $\mathbb{R}^p$, and let $\underset{\sim}{x} \in \mathbb{R}^p$ be a point with $\underset{\sim}{g} \overset{def}{=} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) \neq \underset{\sim}{0}_{p\times 1}$. Then there is an $\bar{s} > 0$ with $f(\underset{\sim}{x} - s\underset{\sim}{g}) < f(\underset{\sim}{x})$ for every $s \in (0, \bar{s})$: a short enough step in the descent direction $-\underset{\sim}{g}$ (Definition 0.5) decreases f.*

Proof

Proof. Let $h(s) \overset{\text{def}}{=} f(\underset{\sim}{x} - s\underset{\sim}{g})$. The inner function $s \mapsto \underset{\sim}{x} - s\underset{\sim}{g}$ has derivative $-\underset{\sim}{g}^\top$, so by the vector chain rule

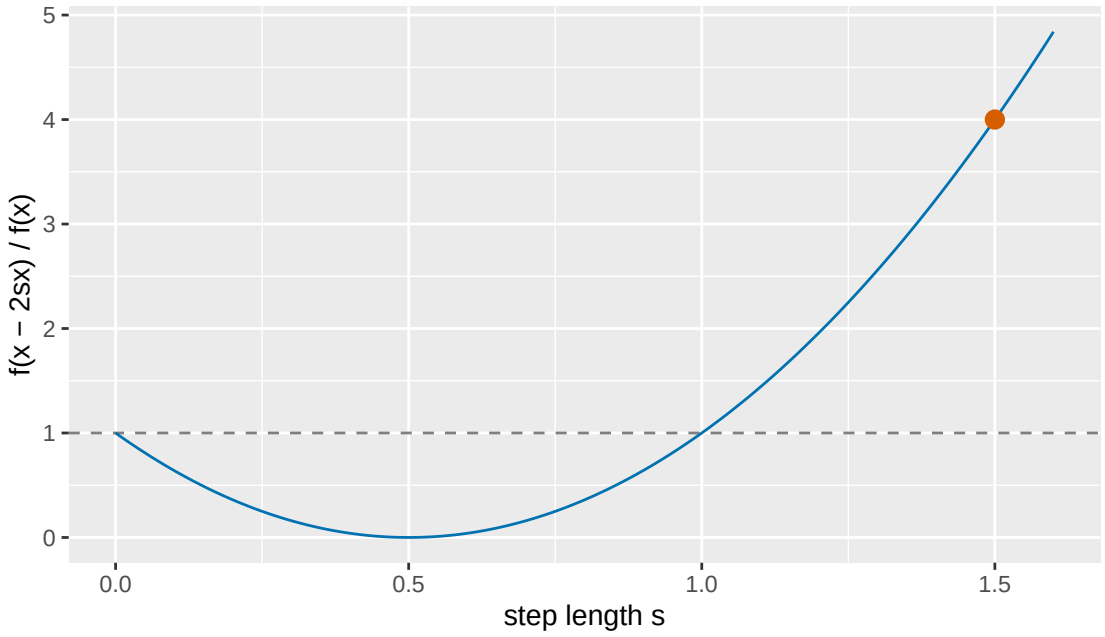
$$
\begin{aligned}
h'(s) &= -\underset{\sim}{g}^\top \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x} - s\underset{\sim}{g}), && \text{(chain rule)} \\
h'(0) &= -\underset{\sim}{g}^\top \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) && \text{(set } s = 0\text{)} \\
&= -\underset{\sim}{g} \cdot \underset{\sim}{g} && \text{(definition of } \underset{\sim}{g}\text{)} \\
&= -\|\underset{\sim}{g}\|^2 && \text{(squared length)} \\
&< 0. && (\underset{\sim}{g} \neq \underset{\sim}{0}_{p\times 1})
\end{aligned}
$$

Since $h'(0) = \lim_{s\to 0}(h(s) - h(0))/s$ is negative, taking $\varepsilon = |h'(0)|$ in the definition of the limit gives an $\bar{s} > 0$ such that $(h(s) - h(0))/s < h'(0) + |h'(0)| = 0$ for $0 < s < \bar{s}$. Multiplying by $s > 0$, $h(s) < h(0)$, that is, $f(\underset{\sim}{x} - s\underset{\sim}{g}) < f(\underset{\sim}{x})$. □

Example 0.5 (How short is short enough)**.** In Example 0.1, $f(x) = x^2$ at $x \neq 0$ has $g = 2x$, and $f(x - 2sx) = (1 - 2s)^2x^2 < x^2$ exactly when $|1 - 2s| < 1$, that is, when $0 < s < 1$. So Theorem 0.1 holds with $\bar{s} = 1$, and the step $s = 1.5$ of that example was too long.
Figure 28 plots the ratio $f(x - 2sx)/f(x)$ against s.

R

```
dd_s_dd <- seq(0, 1.6, length.out = 201)
dd_ratio_dd <- (1 - 2 * dd_s_dd)^2
dd_curve_dd <- tibble::tibble(s = dd_s_dd, ratio = dd_ratio_dd)
dd_long_dd <- tibble::tibble(s = 1.5, ratio = 4)
dd_aes_dd <- ggplot2::aes(s, ratio)
dd_dash_dd <- "dashed"
dd_curve_dd |>
  ggplot2::ggplot() +
  dd_aes_dd +
  ggplot2::geom_hline(
    yintercept = 1, linetype = dd_dash_dd, colour = "grey50"
  ) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(data = dd_long_dd, colour = "#D55E00", size = 3) +
  ggplot2::labs(x = "step length s", y = "f(x - 2sx) / f(x)")
```



Python

```
import matplotlib.pyplot as plt
import numpy as np

dd_s_py = np.linspace(0, 1.6, 201)
dd_ratio_py = (1 - 2 * dd_s_py)**2
fig, ax = plt.subplots(figsize=(6, 3.5), layout="constrained")
ax.plot(dd_s_py, dd_ratio_py, color="#0072B2")
#> [<matplotlib.lines.Line2D object at 0x7fa977f43110>]
ax.axhline(1, linestyle="--", color="grey")
#> <matplotlib.lines.Line2D object at 0x7fa977f42fd0>
ax.plot([1.5], [4], "o", color="#D55E00")
#> [<matplotlib.lines.Line2D object at 0x7fa977f42e90>]
ax.set_xlabel("step length s")
```

Example 0.6 (Gradient descent zigzags)**.** Let $f(\underset{\sim}{x}) = \frac{1}{2}(x_1^2 + b\,x_2^2)$ with $b > 0$, so $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) = (x_1,\ bx_2)^\top$ and one gradient step of length s from $\underset{\sim}{x}$ gives $((1-s)x_1,\ (1-bs)x_2)^\top$. For exact line search (Definition 0.4), let $\phi(s) \overset{\text{def}}{=} f$ at that point:

$$\begin{aligned}
\phi(s) &= \tfrac{1}{2}((1-s)^2x_1^2 + b(1-bs)^2x_2^2), && \text{(definition of } f\text{)}\\
\phi'(s) &= -(1-s)x_1^2 - b^2(1-bs)x_2^2 && \text{(chain rule, term by term)}\\
&= -x_1^2 + s\,x_1^2 - b^2x_2^2 + b^3 s\,x_2^2 && \text{(expand the products)}\\
&= s\,(x_1^2 + b^3x_2^2) - (x_1^2 + b^2x_2^2). && \text{(collect the terms in } s\text{)}
\end{aligned}$$

For $\underset{\sim}{x} \neq \underset{\sim}{0}_{2\times 1}$, $x_1^2 + b^3x_2^2 > 0$, so ϕ' is increasing in s with a single zero, and ϕ is minimized at $s^* = \dfrac{x_1^2 + b^2x_2^2}{x_1^2 + b^3x_2^2}$. From $\underset{\sim}{x}^{(0)} = (b, 1)^\top$:

$$\begin{aligned}
s^{(0)} &= \frac{b^2 + b^2}{b^2 + b^3} && \text{(substitute } x_1 = b, x_2 = 1\text{)}\\
&= \frac{2}{1+b}, && \text{(cancel } b^2\text{)}
\end{aligned}$$

and, with $r \overset{\text{def}}{=} \frac{b-1}{b+1}$,

$$\begin{aligned}
1 - s^{(0)} &= \frac{1 + b - 2}{1+b}\\
&= r, && \text{(common denominator)}\\
1 - b\,s^{(0)} &= \frac{1 + b - 2b}{1+b}\\
&= -r, && \text{(common denominator)}\\
\underset{\sim}{x}^{(1)} &= (r\,b,\ -r\cdot 1)^\top\\
&= r\,(b, -1)^\top. && \text{(the gradient step from } (b,1)^\top\text{)}
\end{aligned}$$

For $b \neq 1$ the same holds at every step, by induction (when $b = 1$, $r = 0$ and $\underset{\sim}{x}^{(1)}$ is already the minimizer): if $\underset{\sim}{x}^{(t)} = (c\,b,\ \pm c)^\top$ for a number $c \neq 0$, then s^*, which depends only on $x_1^2 = c^2b^2$ and $x_2^2 = c^2$, is again $\frac{2}{1+b}$ (the factor c^2 cancels), and the step multiplies the first entry by r and the second by $-r$. So

$$\begin{aligned}
\underset{\sim}{x}^{(t)} &= (b\,r^t,\ (-r)^t)^\top,\\
f(\underset{\sim}{x}^{(t)}) &= \tfrac{1}{2}(b^2 + b)\,r^{2t}\\
&= r^{2t} f(\underset{\sim}{x}^{(0)}).
\end{aligned}$$

For $b \neq 1$, exactly one entry changes sign at every step: the second when $b > 1$ (then $r > 0$), and the first when $b < 1$ (then $r < 0$). So the iterates zigzag toward the minimizer $\underset{\sim}{0}_{2\times 1}$.

- With $b = 3$, $r = \frac{1}{2}$: $\underset{\sim}{x}^{(1)} = (1.5, -0.5)^\top$ and $\underset{\sim}{x}^{(2)} = (0.75, 0.25)^\top$, and f falls from 6 to 1.5 to 0.375, a quarter at each step.
- With $b = 1$, $r = 0$ and one step reaches the minimizer.
- With $b = 0.01$, $r^2 = (0.99/1.01)^2 \approx 0.96$, so f shrinks by only about 4% per step. Newton's method reaches the minimizer of any such f in one step (Theorem 0.3).

Definition 0.6 (Gradient flow)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives. A **gradient flow** of f is a differentiable curve $\underset{\sim}{x} : [0, \infty) \to \mathbb{R}^p$ that satisfies

$$\frac{d\underset{\sim}{x}(t)}{dt} = -\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}(t)).$$

Example 0.7 (Gradient descent follows the gradient flow)**.** Let $f(x) = \frac{1}{2}x^2$ on $\mathbb{R}$, so $\frac{\partial}{\partial x} f(x) = x$. The gradient flow (Definition 0.6) from $x(0) = x_0$ solves $\frac{dx(t)}{dt} = -x(t)$, whose solution is

$$x(t) = x_0 \exp\{-t\}.$$

Gradient descent with a fixed step length s multiplies x by $1 - s$ at each step, so after k steps it has reached time $t = ks$ with

$$x^{(k)} = x_0 \, (1 - s)^k.$$

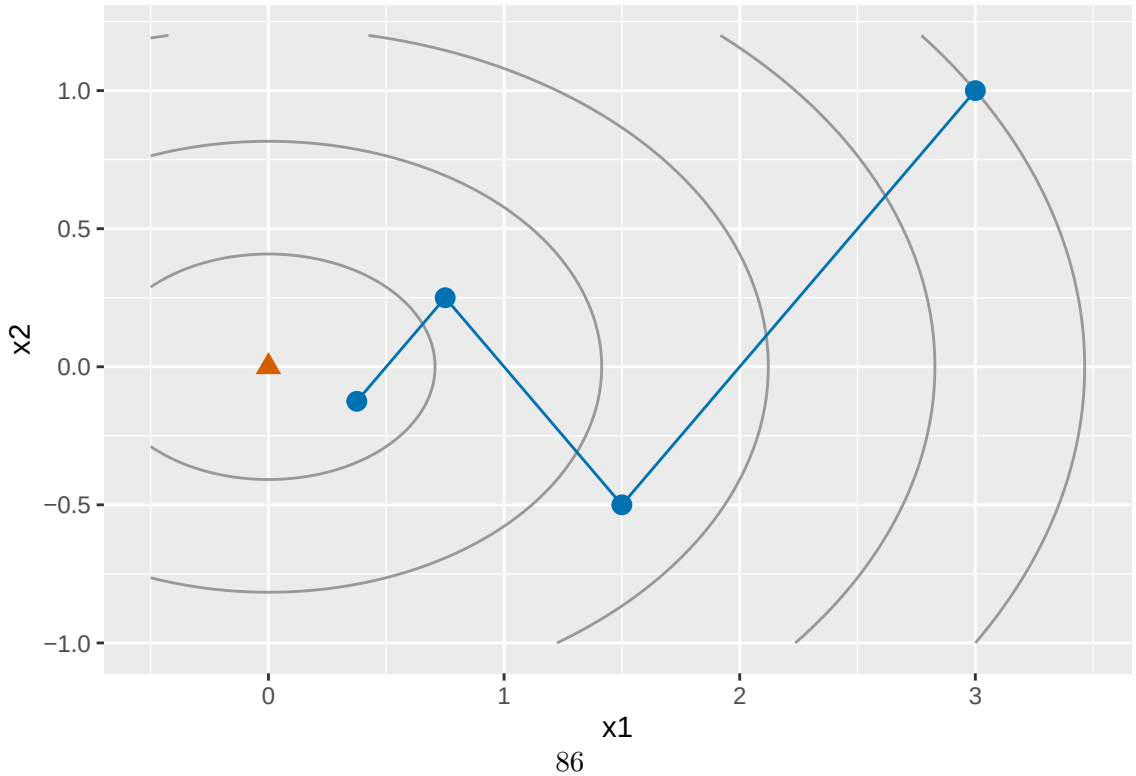
Start at $x_0 = 1$ and run to time $t = 1$:

R

```
x0_gf <- 1
t_gf <- 1
s_gf <- c(0.5, 0.01)
descent_gf <- x0_gf * (1 - s_gf)^(t_gf / s_gf)
flow_gf <- x0_gf * exp(-t_gf)
values_gf <- c(descent_gf, flow = flow_gf)
round(values_gf, 4)
#>                 flow
#> 0.2500 0.3660 0.3679
```

R

```
gz_x1_gz <- seq(-0.5, 3.5, length.out = 201)
gz_x2_gz <- seq(-1, 1.2, length.out = 201)
gz_grid_gz <- expand.grid(x1 = gz_x1_gz, x2 = gz_x2_gz)
gz_grid_gz$f <- 0.5 * (gz_grid_gz$x1^2 + 3 * gz_grid_gz$x2^2)
gz_steps_gz <- 0:3
gz_path_x1_gz <- 3 * 0.5^gz_steps_gz
gz_path_x2_gz <- c(1, -0.5, 0.25, -0.125)
gz_path_gz <- tibble::tibble(x1 = gz_path_x1_gz, x2 = gz_path_x2_gz)
gz_origin_gz <- tibble::tibble(x1 = 0, x2 = 0)
gz_aes_gz <- ggplot2::aes(x1, x2)
gz_aes_z_gz <- ggplot2::aes(z = f)
gz_breaks_gz <- c(0.25, 1, 2.25, 4, 6)
ggplot2::ggplot(gz_grid_gz, gz_aes_gz) +
  ggplot2::geom_contour(
    gz_aes_z_gz, colour = "grey60", breaks = gz_breaks_gz
  ) +
  ggplot2::geom_path(data = gz_path_gz, colour = "#0072B2") +
  ggplot2::geom_point(data = gz_path_gz, colour = "#0072B2", size = 3) +
  ggplot2::geom_point(
    data = gz_origin_gz, colour = "#D55E00", size = 3, shape = 17
  ) +
  ggplot2::labs(x = "x1", y = "x2")
```



Python

```
import matplotlib.pyplot as plt
import numpy as np

gz_x1_gz, gz_x2_gz = np.meshgrid(np.linspace(-0.5, 3.5, 201),
```

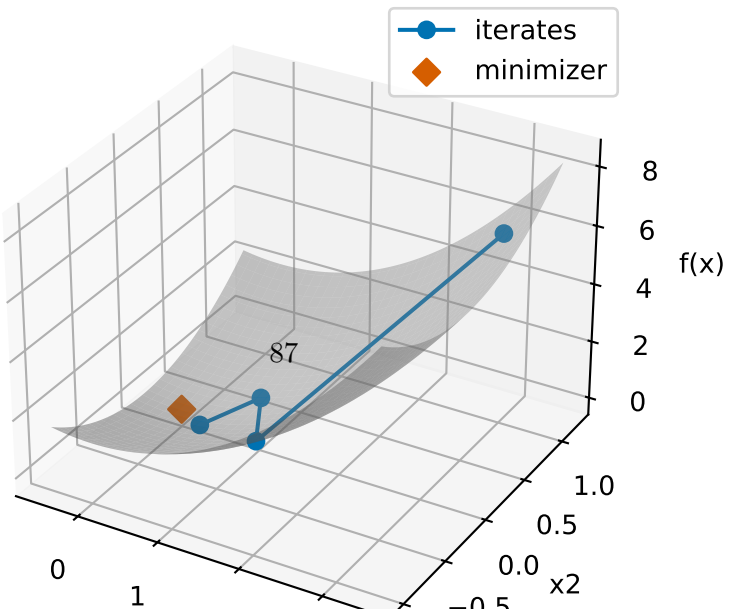
R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

gz_x1_3d, gz_x2_3d = np.meshgrid(np.linspace(-0.5, 3.5, 101),
                                 np.linspace(-1, 1.2, 101))
gz_f_3d = 0.5 * (gz_x1_3d**2 + 3 * gz_x2_3d**2)
gz_path_x1_3d = 3 * 0.5 ** np.arange(4)
gz_path_x2_3d = np.array([1, -0.5, 0.25, -0.125])
gz_path_f_3d = 0.5 * (gz_path_x1_3d**2 + 3 * gz_path_x2_3d**2)
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(gz_x1_3d, gz_x2_3d, gz_f_3d, color="grey", alpha=0.4)
#> <mpl_toolkits.mplot3d.art3d.Poly3DCollection object at 0x7fa977eb9450>
ax.plot(gz_path_x1_3d, gz_path_x2_3d, gz_path_f_3d, "o-", color="#0072B2",
        label="iterates")
#> [<mpl_toolkits.mplot3d.art3d.Line3D object at 0x7fa977eb9310>]
ax.scatter([0], [0], [0], marker="D", color="#D55E00", s=50, label="minimizer")
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa977eb9590>
ax.set_xlabel("x1")
#> Text(0.5, 0, 'x1')
ax.set_ylabel("x2")
#> Text(0.5, 0.5, 'x2')
ax.set_zlabel("f(x)")
#> Text(0.5, 0, 'f(x)')
ax.legend(loc="upper right")
#> <matplotlib.legend.Legend object at 0x7fa977eb96d0>
plt.show();
```



Python

```
import numpy as np

x0_gf, t_gf = 1.0, 1.0
s_gf = np.array([0.5, 0.01])
descent_gf = x0_gf * (1 - s_gf) ** (t_gf / s_gf)
flow_gf = x0_gf * np.exp(-t_gf)
print(np.round(np.append(descent_gf, flow_gf), 4));
#> [0.25   0.366  0.3679]
```

With step length $s = 0.5$ the descent reaches 0.25, and with $s = 0.01$ it reaches 0.366, against 0.3679 for the flow. The smaller the step length, the closer the descent stays to the flow.

Remark (Gradient descent is Euler's method for the gradient flow)

Remark 0.2 (Gradient descent is Euler's method for the gradient flow). Gradient descent with a fixed step length s updates $\underset{\sim}{x}^{(t+1)} = \underset{\sim}{x}^{(t)} - s \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$. This is Euler's method with time step s for the gradient flow equation.
For a linear function f, the velocity of the flow is the negative normal vector.
By the level sets of a linear function, that vector is orthogonal to every level set, so the flow is too.

Definition 0.7 (Relative change stopping criterion)**.** Let f and $\underset{\sim}{x}^{(0)}, \underset{\sim}{x}^{(1)}, \ldots$ be as in gradient descent, and let $\epsilon > 0$. The **relative change stopping criterion** with tolerance ϵ stops the iteration at the first t for which $f(\underset{\sim}{x}^{(t)}) \neq 0$ and

$$\frac{\left|f(\underset{\sim}{x}^{(t+1)}) - f(\underset{\sim}{x}^{(t)})\right|}{\left|f(\underset{\sim}{x}^{(t)})\right|} < \epsilon.$$

Example 0.8 (Stopping a descent on $f(x) = x^2 + 1$)**.** Let $f(x) = x^2 + 1$. Run gradient descent on f from $x^{(0)} = 2$ with step length $s = 0.25$, and stop at tolerance $\epsilon = 0.01$. Each step maps x to $x - 0.25 \cdot 2x = x/2$:

R

```r
x_rc <- 2
tol_rc <- 0.01
for (t_rc in 0:100) {
  x_next_rc <- x_rc - 0.25 * 2 * x_rc
  cost_rc <- x_rc^2 + 1
  cost_next_rc <- x_next_rc^2 + 1
  diff_rc <- cost_next_rc - cost_rc
  change_rc <- abs(diff_rc) / cost_rc
  below_tol_rc <- change_rc < tol_rc
  if (below_tol_rc) break
  x_rc <- x_next_rc
}
c(t = t_rc, change = change_rc)
#>           t      change
#> 5.000000000 0.002918288
```

Python

```python
f_rc = lambda x: x**2 + 1
x_rc, tol_rc = 2.0, 0.01
for t_rc in range(101):
    x_next_rc = x_rc - 0.25 * 2 * x_rc
    change_rc = abs(f_rc(x_next_rc) - f_rc(x_rc)) / abs(f_rc(x_rc))
    if change_rc < tol_rc:
        break
    x_rc = x_next_rc
print(t_rc, change_rc);
#> 5 0.0029182879377431907
```

The criterion first holds at $t = 5$, where the relative change is 0.00292.

Definition 0.8 (Rosenbrock function)**.** The **Rosenbrock function** on $\mathbb{R}^2$ is

$$f(\underset{\sim}{x}) \stackrel{\text{def}}{=} 100(x_2 - x_1^2)^2 + (1 - x_1)^2, \qquad \underset{\sim}{x} = (x_1, x_2)^\top.$$

It is a standard test problem for optimizers [19].

Example 0.9 (Values of the Rosenbrock function)**.** We evaluate the Rosenbrock function at the minimizer $(1, 1)$, at the origin, and at the standard starting point $(-1.2, 1)$. Then we evaluate it along the curve $x_2 = x_1^2$, which is the floor of a narrow, curved valley.

R

```
rosenbrock_rf <- function(x1, x2) 100 * (x2 - x1^2)^2 + (1 - x1)^2

points_rf <- list(minimizer = c(1, 1), origin = c(0, 0), start = c(-1.2, 1))
sapply(points_rf, function(p) round(rosenbrock_rf(p[1], p[2]), 3))
#> minimizer    origin     start
#>       0.0       1.0      24.2

# On the curve x2 = x1^2 the first term vanishes, leaving (1 - x1)^2.
x_curve_rf <- c(-1, 0, 0.5, 1, 2)
rbind(
  on_curve = rosenbrock_rf(x_curve_rf, x_curve_rf^2),
  formula = (1 - x_curve_rf)^2
)
#>          [,1] [,2] [,3] [,4] [,5]
#> on_curve    4    1 0.25    0    1
#> formula     4    1 0.25    0    1
```

Python

```
import numpy as np

def rosenbrock_rf(x1, x2):
    return 100 * (x2 - x1 ** 2) ** 2 + (1 - x1) ** 2

points_rf = {"minimizer": (1, 1), "origin": (0, 0), "start": (-1.2, 1)}
print({name: round(rosenbrock_rf(*p), 3) for name, p in points_rf.items()})
#> {'minimizer': 0, 'origin': 1, 'start': 24.2}

# On the curve x2 = x1^2 the first term vanishes, leaving (1 - x1)^2.
x_curve_rf = np.array([-1.0, 0.0, 0.5, 1.0, 2.0])
print("on_curve:", rosenbrock_rf(x_curve_rf, x_curve_rf ** 2))
#> on_curve: [4.   1.   0.25 0.   1.  ]
print("formula: ", (1 - x_curve_rf) ** 2)
#> formula:  [4.   1.   0.25 0.   1.  ]
```

The function is 0 at $(1, 1)$ and positive everywhere else, because both of its parts are non-negative and vanish together only when $x_1 = 1$ and $x_2 = 1$. So $(1, 1)$ is the global minimizer. At the starting point the value is 24.2, mostly from the first term, because $(-1.2, 1)$ is off the curve $x_2 = x_1^2$. On the curve the function is $(1 - x_1)^2$, which varies slowly compared with the steep walls.

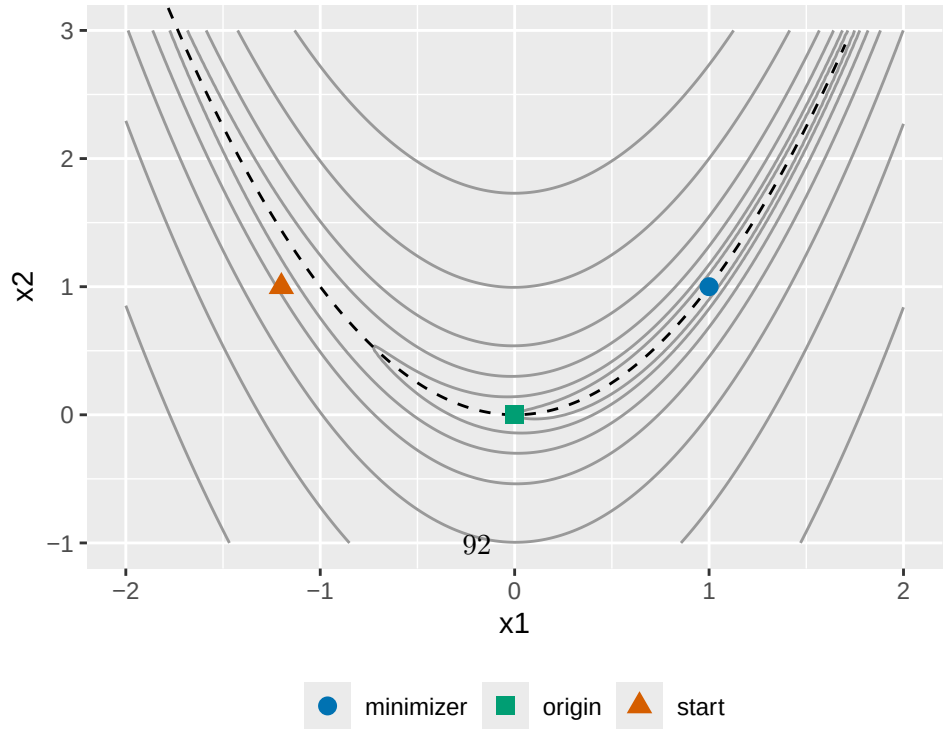
Definition 0.9 (Basin of attraction)**.** For given step lengths, the **basin of attraction** of a local minimizer $\underset{\sim}{x}^*$ under gradient descent is the set of starting points $\underset{\sim}{x}^{(0)}$ from which the iterates converge to $\underset{\sim}{x}^*$ [20].

Example 0.10 (Basins of a double-well function)**.** Let $f(x) = x^4 - 2x^2$, so $\frac{\partial}{\partial x} f(x) = 4x(x-1)(x+1)$ and the stationary points are -1, 0 and 1. Both -1 and 1 are local minimizers, with $f(\pm 1) = -1$, and 0 is a local maximizer. We run gradient descent with the fixed step length $s = 0.05$ for 200 steps from the starting points -1.5, -0.5, -0.1, 0.1, 0.5, 1.5 and 3, and for three steps from $x^{(0)} = 4$.

R

R

```
library(ggplot2)
rf_grid <- expand.grid(
  x1 = seq(-2, 2, length.out = 201),
  x2 = seq(-1, 3, length.out = 201)
)
rf_grid$f <- rosenbrock_rf(rf_grid$x1, rf_grid$x2)
rf_points <- data.frame(
  name = names(points_rf),
  x1 = sapply(points_rf, `[`, 1),
  x2 = sapply(points_rf, `[`, 2)
)
rf_curve <- data.frame(x1 = seq(-2, 1.7, length.out = 200))
rf_curve$x2 <- rf_curve$x1^2
ggplot(rf_grid, aes(x1, x2)) +
  geom_contour(
    aes(z = f),
    colour = "grey60",
    breaks = c(1, 3, 10, 30, 100, 300, 1000)
  ) +
  geom_line(data = rf_curve, linetype = "dashed") +
  geom_point(aes(colour = name, shape = name), data = rf_points, size = 3) +
  scale_colour_manual(
    values = c(minimizer = "#0072B2", origin = "#009E73", start = "#D55E00")
  ) +
  scale_shape_manual(values = c(minimizer = 16, origin = 15, start = 17)) +
  coord_cartesian(xlim = c(-2, 2), ylim = c(-1, 3)) +
  labs(x = "x1", y = "x2", colour = NULL, shape = NULL) +
  theme(legend.position = "bottom")
```



R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

Python

```
import matplotlib.pyplot as plt
import numpy as np

rf_x1_3d, rf_x2_3d = np.meshgrid(np.linspace(-2, 2, 101),
                                 np.linspace(-1, 3, 101))


def rf_f_3d(x1, x2):
    return 100 * (x2 - x1**2) ** 2 + (1 - x1) ** 2


rf_curve_x1_3d = np.linspace(-1.7, 1.7, 200)
rf_curve_x2_3d = rf_curve_x1_3d**2
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(rf_x1_3d, rf_x2_3d, rf_f_3d(rf_x1_3d, rf_x2_3d),
                color="grey", alpha=0.4)
#> <mpl_toolkits.mplot3d.art3d.Poly3DCollection object at 0x7fa978051f90>
ax.plot(rf_curve_x1_3d, rf_curve_x2_3d,
        rf_f_3d(rf_curve_x1_3d, rf_curve_x2_3d), "--", color="black")
#> [<mpl_toolkits.mplot3d.art3d.Line3D object at 0x7fa978051e50>]
rf_style_3d = {"minimizer": ("#0072B2", "o", (1, 1)),
               "origin": ("#009E73", "s", (0, 0)),
               "start": ("#D55E00", "D", (-1.2, 1))}
for name, (color, marker, p) in rf_style_3d.items():
    ax.scatter(p[0], p[1], rf_f_3d(*p), color=color, marker=marker, s=50,
               label=name)
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa9780520d0>
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa978052210>
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa978052350>
ax.set_xlabel("x1")
#> Text(0.5, 0, 'x1')
ax.set_ylabel("x2")
#> Text(0.5, 0.5, 'x2')
ax.set_zlabel("f(x)")
#> Text(0.5, 0, 'f(x)')
ax.legend(loc="upper right")
#> <matplotlib.legend.Legend object at 0x7fa9780525d0>
plt.show();
```




```r
basin_step_bs <- 0.05
basin_starts_bs <- c(-1.5, -0.5, -0.1, 0.1, 0.5, 1.5, 3)

# 200 steps from every starting point at once.
basin_final_bs <- basin_starts_bs
for (i in 1:200) {
  basin_slope_bs <- 4 * basin_final_bs^3 - 4 * basin_final_bs
  basin_final_bs <- basin_final_bs - basin_step_bs * basin_slope_bs
}
basin_end_bs <- signif(basin_final_bs, 3)
tibble::tibble(start = basin_starts_bs, end = basin_end_bs)
#> # A tibble: 7 x 2
#>   start   end
#>   <dbl> <dbl>
#> 1  -1.5    -1
#> 2  -0.5    -1
#> 3  -0.1    -1
#> 4   0.1     1
#> 5   0.5     1
#> 6   1.5     1
#> 7   3      -1

# The first step from 3.
basin_from_three_bs <- 3 - basin_step_bs * (4 * 3^3 - 4 * 3)

# Three steps from 4: the iterates grow in size.
basin_far_bs <- numeric(4)
basin_far_bs[1] <- 4
for (i in 1:3) {
  basin_slope_bs <- 4 * basin_far_bs[i]^3 - 4 * basin_far_bs[i]
  basin_far_bs[i + 1] <- basin_far_bs[i] - basin_step_bs * basin_slope_bs
}
signif(basin_far_bs, 3)
#> [1]       4.0      -8.0      92.8 -160000.0
```

Python

```
basin_step_bs = 0.05

def basin_next_bs(x):
    return x - basin_step_bs * (4 * x ** 3 - 4 * x)

def basin_gd_bs(x0, n_steps=200):
    x = x0
    for _ in range(n_steps):
        x = basin_next_bs(x)
    return x

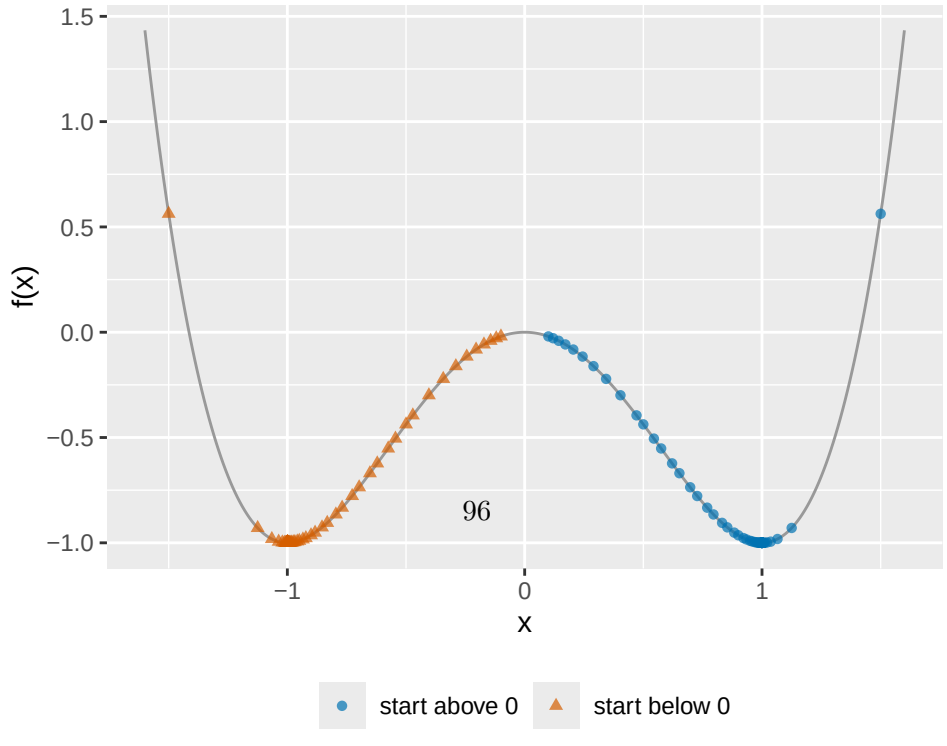
basin_starts_bs = [-1.5, -0.5, -0.1, 0.1, 0.5, 1.5, 3]
for start in basin_starts_bs:
    print(start, float(f"{basin_gd_bs(start):.3g}"))
#> -1.5 -1.0
#> -0.5 -1.0
#> -0.1 -1.0
#> 0.1 1.0
#> 0.5 1.0
#> 1.5 1.0
#> 3 -1.0

# Three steps from 4: the iterates grow in size.
basin_far_bs = [4.0]
for _ in range(3):
    basin_far_bs.append(basin_next_bs(basin_far_bs[-1]))
print([float(f"{v:.3g}") for v in basin_far_bs])
#> [4.0, -8.0, 92.8, -160000.0]
```

Each of the six starts between -1.5 and 1.5 ends at -1 or 1, whichever lies on its own side of 0, so for these starts the sign of the starting point decides which of the two the run reaches. A run from 0 itself never moves, because $\frac{\partial}{\partial x} f(0) = 0$. The basin of attraction of 1 is not all of $(0, \infty)$: the first step from $x^{(0)} = 3$ lands at -1.8, on the other side of 0, and the run ends at -1. From $x^{(0)} = 4$ the iterates, starting with $x^{(0)}$ itself, are 4, -8, 92.8 and -1.6×10^5, which grow in absolute value at every step shown. Both of these starts are far from 0, where the derivative is large and so the step $s \frac{\partial}{\partial x} f(x)$ is long. From $x^{(0)} = 3$ that step is $96\,s$ long, so the first step stays on the positive side of 0 only when s is below 0.03125. So the basin of 1 depends on s, and with $s = 0.05$ the basin is not a half-line.

R

```
# Uses basin_step_bs and basin_starts_bs from the example above.
basin_runs_bf <- matrix(0, nrow = 31, ncol = 6)
basin_runs_bf[1, ] <- basin_starts_bs[1:6]
for (i in 1:30) {
  basin_prev_bf <- basin_runs_bf[i, ]
  basin_slope_bf <- 4 * basin_prev_bf^3 - 4 * basin_prev_bf
  basin_runs_bf[i + 1, ] <- basin_prev_bf - basin_step_bs * basin_slope_bf
}
basin_side_bf <- ifelse(
  basin_starts_bs[1:6] > 0, "start above 0", "start below 0"
)
basin_x_bf <- as.vector(basin_runs_bf)
basin_side_all_bf <- rep(basin_side_bf, each = 31)
basin_paths_bf <- tibble::tibble(x = basin_x_bf, side = basin_side_all_bf)
basin_paths_bf$cost <- basin_paths_bf$x^4 - 2 * basin_paths_bf$x^2
basin_grid_bf <- seq(-1.6, 1.6, length.out = 321)
basin_curve_bf <- tibble::tibble(
  x = basin_grid_bf, cost = basin_grid_bf^4 - 2 * basin_grid_bf^2
)
basin_colours_bf <- c("start above 0" = "#0072B2", "start below 0" = "#D55E00")
ggplot2::ggplot(basin_curve_bf, ggplot2::aes(x, cost)) +
  ggplot2::geom_line(colour = "grey60") +
  ggplot2::geom_point(
    data = basin_paths_bf,
    ggplot2::aes(colour = side, shape = side), alpha = 0.7
  ) +
  ggplot2::scale_colour_manual(values = basin_colours_bf) +
  ggplot2::labs(x = "x", y = "f(x)", colour = NULL, shape = NULL) +
  ggplot2::theme(legend.position = "bottom")
```



Proposition 0.1 (Divergence of gradient descent on the double-well function)**.** *Let* $f(x) = x^4 - 2x^2$*, and run gradient descent with a fixed step length* $s > 0$*. If* $x^{(0)}$ *satisfies*

$$x^{(0)^2} > 1 + \frac{1}{2s},$$

then $\left|x^{(t)}\right| \to \infty$ *as* $t \to \infty$*.*

Proof

Proof. The derivative is $\frac{\partial}{\partial x} f(x) = 4x^3 - 4x$, so one step of gradient descent gives

$$\begin{aligned} x^{(t+1)} &= x^{(t)} - s\left(4x^{(t)^3} - 4x^{(t)}\right) && \text{(gradient descent step)} \\ &= -x^{(t)}\left(4sx^{(t)^2} - 4s - 1\right). && \text{(factor out } -x^{(t)}\text{)} \end{aligned}$$

Write $m(x) \stackrel{\text{def}}{=} 4sx^2 - 4s - 1$. Then $\left|x^{(t+1)}\right| = \left|x^{(t)}\right| m(x^{(t)})$ whenever $m(x^{(t)}) \geq 0$. Now $m(x) > 1$ exactly when $x^2 > 1 + 1/(2s)$, and $m(x)$ increases with x^2. So if $x^{(t)}$ satisfies the condition, then $\left|x^{(t+1)}\right| > \left|x^{(t)}\right|$, and $x^{(t+1)}$ satisfies it too. By induction on t, every iterate satisfies it. Since $\left|x^{(t)}\right|$ increases at every step, $x^{(t)^2} \geq x^{(0)^2}$, and so $m(x^{(t)}) \geq m(x^{(0)}) > 1$. Hence $\left|x^{(t)}\right| \geq m(x^{(0)})^t \left|x^{(0)}\right|$, which tends to ∞. □

Corollary 0.1 (Basins of the double-well function are bounded)**.** *Let* $f(x) = x^4 - 2x^2$*, and run gradient descent with a fixed step length* $s > 0$*. Every start* $x^{(0)}$ *in the basin of attraction of a local minimizer of* f *satisfies*

$$x^{(0)^2} \leq 1 + \frac{1}{2s}.$$

Proof

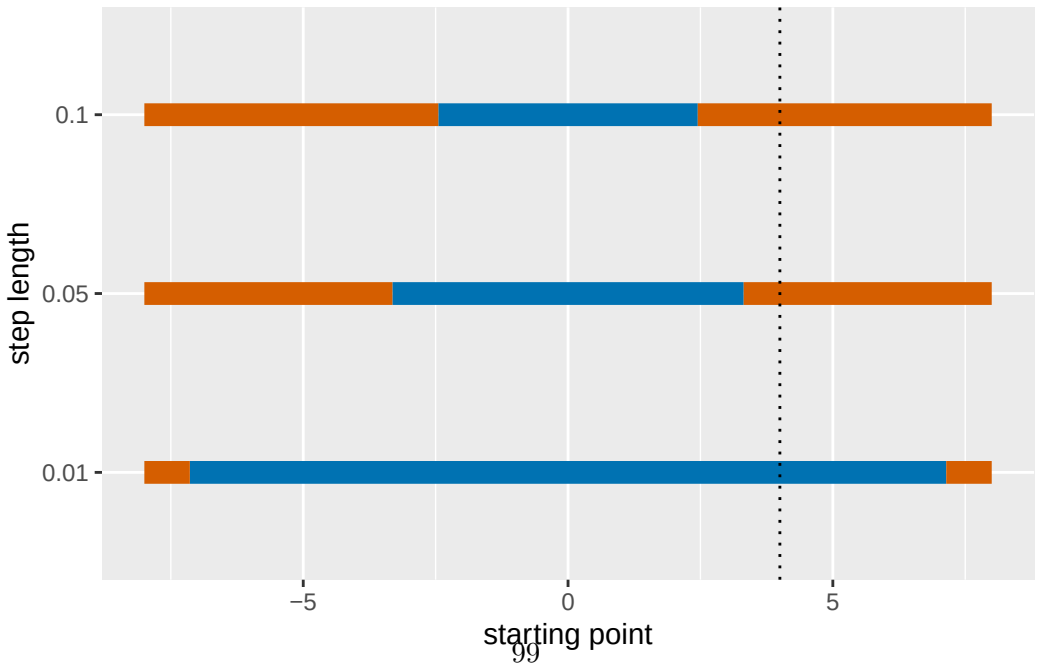
Proof. The iterates from a start in the basin converge to a local minimizer, so they do not diverge to ∞. By Proposition 0.1, the start therefore does not satisfy $x^{(0)^2} > 1 + 1/(2s)$. □

Example 0.11 (Divergent starts on the double-well function for three step lengths)**.** For $f(x) = x^4 - 2x^2$, Proposition 0.1 says that gradient descent diverges to ∞ from every start with $\left|x^{(0)}\right|$ above $\sqrt{1 + 1/(2s)}$. This bound is 7.14 for $s = 0.01$, 3.32 for $s = 0.05$, and 2.45 for $s = 0.1$. The derivative $4x^3 - 4x$ is unbounded, as is the second derivative

$12x^2 - 4$, so the step $s\frac{\partial}{\partial x}f(x)$ grows without bound with x, and a far start overshoots. By Proposition 0.1, every fixed step length fails to converge from some starts. A larger step length lowers the bound, so more starts are guaranteed to diverge. By Corollary 0.1, each basin of attraction lies within the same bounded interval, which narrows as s grows. In the earlier basin example, $s = 0.05$, and the start 4 lies above the bound 3.32, which is why its iterates grow. Figure 34 shows the three bounds.

R

```r
dwf_step_fg <- c(0.01, 0.05, 0.1)
dwf_cut_fg <- sqrt(1 + 1 / (2 * dwf_step_fg))
dwf_level_fg <- factor(dwf_step_fg)
dwf_bars_fg <- tibble::tibble(step = dwf_level_fg, cut = dwf_cut_fg)
dwf_aes_fg <- ggplot2::aes(y = step)
dwf_mid_fg <- ggplot2::aes(x = -cut, xend = cut, yend = step)
dwf_left_fg <- ggplot2::aes(x = -8, xend = -cut, yend = step)
dwf_right_fg <- ggplot2::aes(x = cut, xend = 8, yend = step)
dwf_bars_fg |>
  ggplot2::ggplot() +
  dwf_aes_fg +
  ggplot2::geom_segment(dwf_mid_fg, colour = "#0072B2", linewidth = 4) +
  ggplot2::geom_segment(dwf_left_fg, colour = "#D55E00", linewidth = 4) +
  ggplot2::geom_segment(dwf_right_fg, colour = "#D55E00", linewidth = 4) +
  ggplot2::geom_vline(xintercept = 4, linetype = "dotted") +
  ggplot2::labs(x = "starting point", y = "step length")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

dwf_step_py = [0.01, 0.05, 0.1]
```

Definition 0.10 (Plateau)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ be a function with a gradient at every point of a region, and let $\epsilon > 0$ be a tolerance. The region is a **plateau** of f if the Euclidean norm of the gradient satisfies

$$0 < \left\| \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) \right\| < \epsilon$$

at every point $\underset{\sim}{x}$ of the region. The function is then nearly flat on the region, and the region contains no stationary point.

Example 0.12 (A cost with no stationary point)**.**
Let $f(x) = \log(1 + \mathrm{e}^{-x})$. Its derivative is $\frac{\partial}{\partial x} f(x) = -1/(1 + \mathrm{e}^{x})$, which is negative for every x, so f has no stationary point at all.
At $x = 0$ the gradient has size 0.5. At $x = 10$ it has size 4.54×10^{-5}, which is small but not zero. A step of gradient descent subtracts the gradient times the step length. The gradient is negative, so the iterates keep moving to the right, more and more slowly. With tolerance $\epsilon = 10^{-4}$, the region $x \geq 10$ is a plateau (Definition 0.10). A run that enters this plateau has not converged; the run has only slowed down.

Definition 0.11 (Armijo condition)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x} \in \mathbb{R}^p$, and let $g \overset{\text{def}}{=} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$. A step length $s > 0$ satisfies the **Armijo condition** [21] with constant $c \in (0, 1)$ at $\underset{\sim}{x}$ if

$$f(\underset{\sim}{x} - s\underset{\sim}{g}) \leq f(\underset{\sim}{x}) - c\, s\, \|\underset{\sim}{g}\|^2.$$

Example 0.13 (The Armijo condition on a parabola)**.** Let $f(x) = x^2$ and $x = 2$, so $g = f'(2) = 4$ and $\|g\|^2 = 16$. With $c = 1/4$, the condition on a step length $s > 0$ is

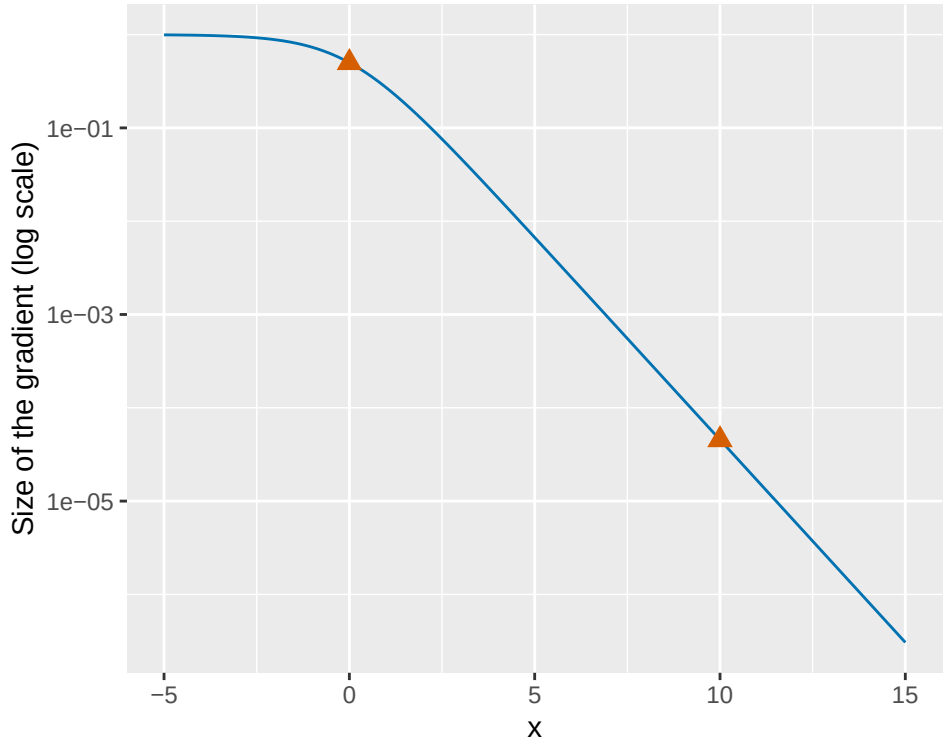
$$\begin{aligned} (2 - 4s)^2 &\leq 4 - 4s && (f(2 - 4s) \leq f(2) - c\, s\, \|g\|^2) \\ 4 - 16s + 16s^2 &\leq 4 - 4s && (\text{expand the square}) \\ 16s^2 &\leq 12s && (\text{add } 16s - 4 \text{ to both sides}) \\ s &\leq 3/4. && (\text{divide by } 16s > 0) \end{aligned}$$

So $s = 1/2$ satisfies the condition: $f(0) = 0 \leq 4 - 2 = 2$. And $s = 1$ does not: $f(-2) = 4 > 4 - 4 = 0$.

Definition 0.12 (Backtracking line search)**.** Let $s_0 > 0$ be an initial step length, $\beta \in (0, 1)$ a shrink factor, and $c \in (0, 1)$. **Backtracking line search** [21] is the line search that chooses $s^{(t)}$ to be the largest of $s_0, \beta s_0, \beta^2 s_0, \ldots$ that satisfies the Armijo condition with constant c at $\underset{\sim}{x}^{(t)}$.

R

```
plateau_grid_pf <- seq(-5, 15, length.out = 401)
plateau_exp_pf <- exp(plateau_grid_pf)
plateau_size_pf <- 1 / (1 + plateau_exp_pf)
plateau_curve_pf <- tibble::tibble(x = plateau_grid_pf, size = plateau_size_pf)
plateau_marks_x_pf <- c(0, 10)
plateau_marks_exp_pf <- exp(plateau_marks_x_pf)
plateau_marks_size_pf <- 1 / (1 + plateau_marks_exp_pf)
plateau_marks_pf <- tibble::tibble(
  x = plateau_marks_x_pf, size = plateau_marks_size_pf
)
plateau_aes_pf <- ggplot2::aes(x, size)
ggplot2::ggplot(plateau_curve_pf, plateau_aes_pf) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_point(
    data = plateau_marks_pf, colour = "#D55E00", shape = 17, size = 3
  ) +
  ggplot2::scale_y_log10() +
  ggplot2::labs(x = "x", y = "Size of the gradient (log scale)")
```



Python

```
import matplotlib.pyplot as plt
import numpy as np

plateau_grid_pf = np.linspace(-5, 15, 401)
plateau_marks_pf = np.array([0, 10])
fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
ax.plot(plateau_grid_pf, 1 / (1 + np.exp(plateau_grid_pf)), color="#0072B2")
#> [<matplotlib.lines.Line2D object at 0x7fa980ab5950>]
```

Example 0.14 (Backtracking on a parabola)**.** For $f(x) = x^2$ at $x = 2$ with $c = 1/4$, the Armijo condition holds exactly when $s \le 3/4$ (Example 0.13). With $s_0 = 1$ and $\beta = 1/2$, the trial step lengths are $1, 1/2, 1/4, \ldots$. The first, $s = 1$, exceeds $3/4$ and is rejected. The second, $s = 1/2$, is at most $3/4$ and is accepted. So backtracking chooses $s^{(0)} = 1/2$, after two evaluations of f.

Theorem 0.2 (A bound on the error of gradient descent on a quadratic)**.** *Let* $\mathbf{S}$ *be a* $p \times p$ *positive definite matrix with eigenvalues* $\lambda_1, \ldots, \lambda_p$*, let* $\underset{\sim}{c} \in \mathbb{R}^p$*, and let* $f(\underset{\sim}{x}) = \frac{1}{2}\, \underset{\sim}{x}^\top \mathbf{S} \underset{\sim}{x} - \underset{\sim}{c} \cdot \underset{\sim}{x}$*, whose unique minimizer is* $\underset{\sim}{x}^* = \mathbf{S}^{-1} \underset{\sim}{c}$ *(Theorem 0.3). Gradient descent with a fixed step length* $s > 0$ *from any* $\underset{\sim}{x}^{(0)}$ *has errors satisfying, for* $t = 1, 2, \ldots$*,*

$$\left\| \underset{\sim}{x}^{(t)} - \underset{\sim}{x}^* \right\| \le \rho(s)^t \left\| \underset{\sim}{x}^{(0)} - \underset{\sim}{x}^* \right\|, \qquad \rho(s) \overset{def}{=} \max_{1 \le i \le p} |1 - s\,\lambda_i|.$$

Proof

Proof. Let $\underset{\sim}{e}^{(t)} \overset{\text{def}}{=} \underset{\sim}{x}^{(t)} - \underset{\sim}{x}^*$ be the error at step t. The gradient of f is $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) = \mathbf{S}\underset{\sim}{x} - \underset{\sim}{c}$ (as in the proof of Theorem 0.3), and $\underset{\sim}{c} = \mathbf{S}\underset{\sim}{x}^*$, so

$$\begin{aligned}
\underset{\sim}{e}^{(t+1)} &= \underset{\sim}{x}^{(t)} - s\,(\mathbf{S}\underset{\sim}{x}^{(t)} - \underset{\sim}{c}) - \underset{\sim}{x}^* && \text{(definition of gradient descent)} \\
&= \underset{\sim}{x}^{(t)} - s\,\mathbf{S}(\underset{\sim}{x}^{(t)} - \underset{\sim}{x}^*) - \underset{\sim}{x}^* && (\underset{\sim}{c} = \mathbf{S}\underset{\sim}{x}^*) \\
&= (\mathbf{I}_p - s\,\mathbf{S})\, \underset{\sim}{e}^{(t)}. && \text{(collect the terms in } \underset{\sim}{e}^{(t)})
\end{aligned}$$

By induction on t, $\underset{\sim}{e}^{(t)} = (\mathbf{I}_p - s\,\mathbf{S})^t\, \underset{\sim}{e}^{(0)}$. By the spectral theorem, write $\mathbf{S} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$. Then $\mathbf{I}_p - s\,\mathbf{S} = \mathbf{Q}(\mathbf{I}_p - s\,\mathbf{\Lambda})\mathbf{Q}^\top$ because $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}_p$, and the tth power cancels each inner $\mathbf{Q}^\top\mathbf{Q} = \mathbf{I}_p$, so

$$\begin{aligned}
\left\| \underset{\sim}{e}^{(t)} \right\| &= \left\| \mathbf{Q}(\mathbf{I}_p - s\,\mathbf{\Lambda})^t \mathbf{Q}^\top \underset{\sim}{e}^{(0)} \right\| && \text{(the factorization above)} \\
&= \left\| (\mathbf{I}_p - s\,\mathbf{\Lambda})^t \mathbf{Q}^\top \underset{\sim}{e}^{(0)} \right\| && \text{(orthogonal matrices preserve length)} \\
&\le \rho(s)^t \left\| \mathbf{Q}^\top \underset{\sim}{e}^{(0)} \right\| && \text{(each coordinate is multiplied by } (1 - s\lambda_i)^t) \\
&= \rho(s)^t \left\| \underset{\sim}{e}^{(0)} \right\|. && \text{(orthogonal matrices preserve length)}
\end{aligned}$$

The length-preserving steps apply orthogonal matrices preserve length. The inequality holds because $\mathbf{I}_p - s\,\mathbf{\Lambda}$ is diagonal, so its tth power multiplies coordinate i by $(1 - s\,\lambda_i)^t$, whose absolute value is at most $\rho(s)^t$. □

Example 0.15 (The error multiplier for a step length of one quarter)**.** Let $\mathbf{S} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ with eigenvalues 3 and 1, and $\underset{\sim}{c} = \underset{\sim}{0}_{2\times 1}$, so $\underset{\sim}{x}^* = \underset{\sim}{0}_{2\times 1}$. With $s = 0.25$, the factors $1 - s\,\lambda_i$ are 0.25 and 0.75, so $\rho(s) = 0.75$ and

$$\|\underset{\sim}{x}^{(t)}\| \le 0.75^t \, \|\underset{\sim}{x}^{(0)}\|.$$

Along the eigenvector for $\lambda = 3$ the error shrinks fast, and along the eigenvector for $\lambda = 1$ it shrinks slowly; the bound follows the slower one.

Theorem 0.3 (The best constant step length)**.** *In the setting of Theorem 0.2, let* $\lambda_{\min}$ *and* $\lambda_{\max}$ *be the smallest and largest eigenvalues of* $\mathbf{S}$*. The factor* $\rho(s) = \max_i |1 - s\,\lambda_i|$ *of Theorem 0.2 is smallest over* $s > 0$ *at*

$$s^* = \frac{2}{\lambda_{\min} + \lambda_{\max}},$$

where

$$\rho(s^*) = \frac{\kappa - 1}{\kappa + 1}, \qquad \kappa = \kappa(\mathbf{S}),$$

the condition number of $\mathbf{S}$*.*

Proof

Proof. For $s > 0$, each $1 - s\,\lambda_i$ lies between $u_{\min} \overset{\text{def}}{=} 1 - s\,\lambda_{\max}$ and $u_{\max} \overset{\text{def}}{=} 1 - s\,\lambda_{\min}$, so $|1 - s\,\lambda_i| \le \max(u_{\max}, -u_{\min})$. Both $\lambda_{\min}$ and $\lambda_{\max}$ occur among the λ_i, and $|u_{\max}| \ge u_{\max}$ and $|u_{\min}| \ge -u_{\min}$, so the maximum is attained:

$$\rho(s) = \max(1 - s\,\lambda_{\min},\ s\,\lambda_{\max} - 1).$$

The first expression is strictly decreasing in s, and the second is strictly increasing, and they coincide when

$$\begin{aligned} 1 - s\,\lambda_{\min} &= s\,\lambda_{\max} - 1 && \text{(set the two expressions equal)} \\ 2 &= s\,(\lambda_{\min} + \lambda_{\max}) && \text{(collect the terms in } s\text{)} \\ s &= \frac{2}{\lambda_{\min} + \lambda_{\max}} = s^*. && \text{(divide)} \end{aligned}$$

For $s < s^*$, $\rho(s)$ is at least the decreasing expression, which exceeds its value at s^*. For $s > s^*$, $\rho(s)$ is at least the increasing expression, which exceeds its value at s^*. So s^* is the minimizer, and

$$\begin{aligned}
\rho(s^*) &= 1 - \frac{2\,\lambda_{\min}}{\lambda_{\min} + \lambda_{\max}} && \text{(first expression at } s^*\text{)} \\
&= \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} && \text{(common denominator)} \\
&= \frac{\kappa - 1}{\kappa + 1}. && \text{(divide the numerator and denominator by } \lambda_{\min}\text{)}
\end{aligned}$$

□

Example 0.16 (The best step length for eigenvalues 3 and 1)**.** For $\mathbf{S} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ with eigenvalues 3 and 1, Theorem 0.3 gives $s^* = 2/(3+1) = 0.5$, the condition number $\kappa = 3$, and $\rho(s^*) = (3-1)/(3+1) = 0.5$. A grid search over $s = 0.01, 0.02, \ldots, 1$ finds the smallest $\rho(s)$ at $s = 0.5$, the same step length. Each step then multiplies the error by at most 0.5.

Exercise 0.1 (Which step lengths shrink the error bound)**.** In the setting of Theorem 0.2, find all $s > 0$ with $\rho(s) < 1$, and write the answer as a condition on s that involves $\lambda_{\max}$.

Solution

Solution 0.1. $\rho(s) < 1$ says that $|1 - s\,\lambda_i| < 1$ for every i, which is $-1 < 1 - s\,\lambda_i < 1$. The right inequality holds for every $s > 0$ because $\lambda_i > 0$. The left inequality is $s\,\lambda_i < 2$, that is, $s < 2/\lambda_i$. These p inequalities, one for each eigenvalue, all hold exactly when the inequality for the largest eigenvalue holds, since it is the most restrictive:

$$0 < s < \frac{2}{\lambda_{\max}}.$$

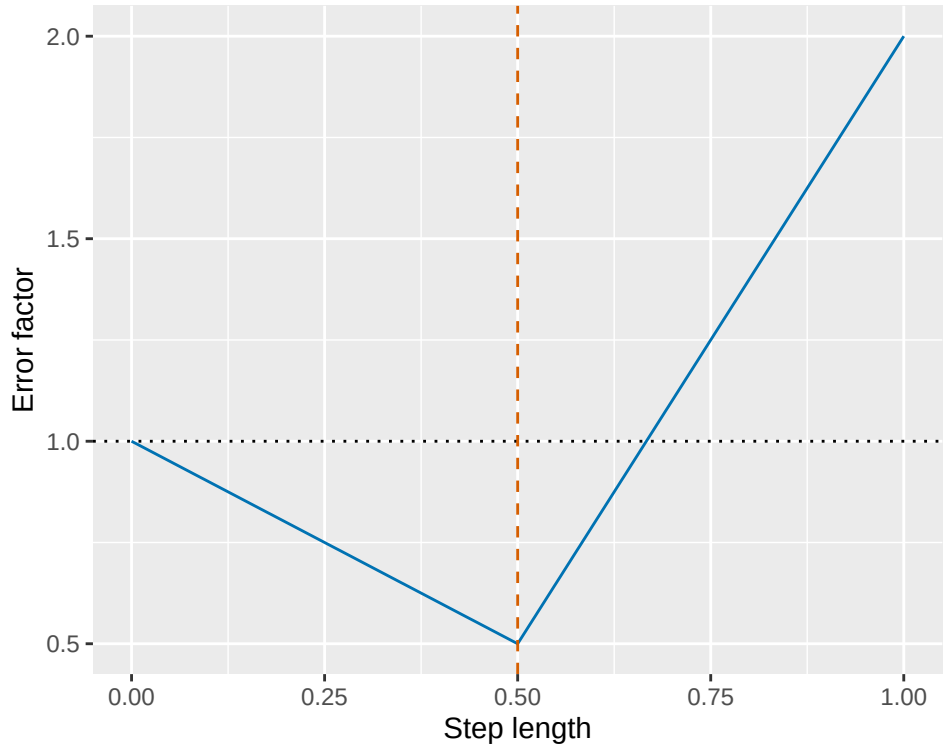
Exercise 0.2 (The best step length for eigenvalues 1 and 9)**.** In gradient descent, a fixed step length that is too short converges slowly, and one that is too long diverges. Theorem 0.3 gives the best one from the eigenvalues of $\mathbf{S}$.
Let $\mathbf{S} = \begin{bmatrix} 1 & 0 \\ 0 & 9 \end{bmatrix}$. Using Theorem 0.3, find s^*.

Solution

Solution 0.2. $\mathbf{S}$ is diagonal, so its eigenvalues are its diagonal entries 1 and 9. Then

R

```r
bcs_grid_pf <- seq(0, 1, length.out = 401)
bcs_first_pf <- abs(1 - 3 * bcs_grid_pf)
bcs_second_pf <- abs(1 - bcs_grid_pf)
bcs_rho_pf <- pmax(bcs_first_pf, bcs_second_pf)
bcs_curve_pf <- tibble::tibble(s = bcs_grid_pf, rho = bcs_rho_pf)
bcs_aes_pf <- ggplot2::aes(s, rho)
ggplot2::ggplot(bcs_curve_pf, bcs_aes_pf) +
  ggplot2::geom_line(colour = "#0072B2") +
  ggplot2::geom_hline(yintercept = 1, linetype = "dotted") +
  ggplot2::geom_vline(
    xintercept = 0.5, colour = "#D55E00", linetype = "dashed"
  ) +
  ggplot2::labs(x = "Step length", y = "Error factor")
```



Python

```python
import matplotlib.pyplot as plt
import numpy as np

bcs_grid_pf = np.linspace(0, 1, 401)
bcs_rho_pf = np.maximum(np.abs(1 - 3 * bcs_grid_pf), np.abs(1 - bcs_grid_pf))
fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
ax.plot(bcs_grid_pf, bcs_rho_pf, color="#0072B2")
#> [<matplotlib.lines.Line2D object at 0x7fa980aa8f50>]
ax.axhline(1, color="black", linestyle=":")
#> <matplotlib.lines.Line2D object at 0x7fa980aa9090>
ax.axvline(0.5, color="#D55E00", linestyle="--")
#> <matplotlib.lines.Line2D object at 0x7fa980aa91d0>
ax.set_xlabel("Step length")
```

$$s^* = \frac{2}{1+9} = 0.2.$$

Exercise 0.3 (The best error factor for eigenvalues 1 and 9)**.** In the setting of Exercise 0.2, find $\rho(s^*)$.

Solution

Solution 0.3. The condition number is $\kappa(\mathbf{S}) = 9/1 = 9$, so by Theorem 0.3

$$\rho(s^*) = \frac{9-1}{9+1} = 0.8.$$

Exercise 0.4 (Steps to cut the error bound by a factor of ten)**.** In the setting of Exercise 0.2 and Exercise 0.3, find the smallest $t = 1, 2, \ldots$ with $\rho(s^*)^t \leq 0.1$.

Solution

Solution 0.4. From Solution 0.3, $\rho(s^*) = 0.8$. Taking logarithms, $\rho(s^*)^t \leq 0.1$ becomes $t \ln \rho(s^*) \leq \ln 0.1$. Here $\ln \rho(s^*) < 0$, so dividing by it flips the inequality: $t \geq \ln 0.1 / \ln \rho(s^*) \approx 10.32$. The smallest such t is 11.

Definition 0.13 (Momentum coefficient)**.** Let $\underset{\sim}{g}^{(1)}, \underset{\sim}{g}^{(2)}, \ldots$ be vectors in $\mathbb{R}^p$, and let $\underset{\sim}{v}^{(t)}$ be a running **velocity** built from them. The **momentum coefficient** $\beta \in [0, 1)$ is the factor that multiplies the previous velocity in

$$\underset{\sim}{v}^{(t)} \stackrel{\text{def}}{=} \beta \underset{\sim}{v}^{(t-1)} + \underset{\sim}{g}^{(t)},$$

for $t = 1, 2, \ldots$, with $\underset{\sim}{v}^{(0)} \stackrel{\text{def}}{=} \underset{\sim}{0}$.

Example 0.17 (How far back the velocity remembers)**.** Unrolling the recursion, the vector $g^{(t-k)}$ enters $\underset{\sim}{v}^{(t)}$ with factor β^k, for $k < t$. These factors add up to at most $\sum_{k=0}^{\infty} \beta^k = 1/(1-\beta)$. As a rule of thumb, the velocity is therefore dominated by the last $1/(1-\beta)$ vectors.

R

```r
beta_mc <- c(0, 0.5, 0.9, 0.99)
k_mc <- 10
weight_mc <- beta_mc^k_mc
memory_mc <- 1 / (1 - beta_mc)
weight_rounded_mc <- signif(weight_mc, 2)
# nolint start: undesirable_function_linter.
rbind(beta = beta_mc, weight = weight_rounded_mc, memory = memory_mc)
#>        [,1]    [,2]  [,3]   [,4]
#> beta      0 0.50000  0.90   0.99
#> weight    0 0.00098  0.35   0.90
#> memory    1 2.00000 10.00 100.00
# nolint end
```

Python

```python
beta_mc = [0, 0.5, 0.9, 0.99]
k_mc = 10
for b_mc in beta_mc:
    print(b_mc, float(f"{b_mc**k_mc:.2g}"), round(1 / (1 - b_mc)));
#> 0 0.0 1
#> 0.5 0.00098 2
#> 0.9 0.35 10
#> 0.99 0.9 100
```

At $\beta = 0$ the velocity is the current vector alone, and at $\beta = 0.99$ it combines about 100 vectors.

Definition 0.14 (Momentum (heavy-ball method))**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x}^{(0)}$ be a starting point, let $s > 0$ be a step length, and let $\beta \in [0, 1)$ be a momentum coefficient. **Momentum** (the **heavy-ball method**) updates, for $t = 1, 2, ...$, by

$$\underset{\sim}{v}^{(t)} \stackrel{\text{def}}{=} \beta \underset{\sim}{v}^{(t-1)} + \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t-1)})$$
$$\underset{\sim}{x}^{(t)} \stackrel{\text{def}}{=} \underset{\sim}{x}^{(t-1)} - s \underset{\sim}{v}^{(t)},$$

with $\underset{\sim}{v}^{(0)} \stackrel{\text{def}}{=} \underset{\sim}{0}$.

Example 0.18 (Momentum on a one-dimensional quadratic)**.** Let $f(x) = \frac{1}{2}x^2$, so $\frac{\partial}{\partial x} f(x) = x$. Start at $x^{(0)} = 1$ with $s = 0.1$, and take 20 updates with $\beta = 0$, which is

plain gradient descent, and with $\beta = 0.9$:

R

```
x0_mo <- 1
s_mo <- 0.1
beta_mo <- c(plain = 0, momentum = 0.9)
n_mo <- 20
x_mo <- rep(x0_mo, 2)
v_mo <- c(0, 0)
path_mo <- matrix(NA_real_, nrow = n_mo, ncol = 2)
for (k_mo in seq_len(n_mo)) {
  v_mo <- beta_mo * v_mo + x_mo
  x_mo <- x_mo - s_mo * v_mo
  path_mo[k_mo, ] <- x_mo
}
last_mo <- path_mo[n_mo, ]
lowest_mo <- apply(path_mo, 2, min)
# nolint start: undesirable_function_linter.
summary_mo <- rbind(last = last_mo, lowest = lowest_mo)
# nolint end
round(summary_mo, 4)
#>          [,1]    [,2]
#> last   0.1216  0.3447
#> lowest 0.1216 -0.6037
```

Python

```
s_mo, n_mo = 0.1, 20
for beta_mo in (0.0, 0.9):
    x_mo, v_mo, path_mo = 1.0, 0.0, []
    for _ in range(n_mo):
        v_mo = beta_mo * v_mo + x_mo
        x_mo = x_mo - s_mo * v_mo
        path_mo.append(x_mo)
    print(beta_mo, round(path_mo[-1], 4), round(min(path_mo), 4));
#> 0.0 0.1216 0.1216
#> 0.9 0.3447 -0.6037
```

After 20 updates, plain descent has reached 0.122 and momentum 0.345. Momentum overshoots the minimizer 0, down to -0.604 at update 9, and then swings back. In this

run, with a small step length, the swing leaves momentum behind plain descent after 20 updates.

Definition 0.15 (Adaptive method)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x}^{(0)}$ be a starting point, let $s > 0$ be a step length, let $\rho \in [0, 1)$ be a decay factor, and let $\epsilon > 0$. Write $g_j^{(t)}$ for coordinate j of $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t-1)})$. An **adaptive method** is defined, for $t = 1, 2, \ldots$ and each coordinate $j = 1, \ldots, p$,

$$u_j^{(t)} \overset{\text{def}}{=} \rho\, u_j^{(t-1)} + (1 - \rho) \left(g_j^{(t)}\right)^2$$

$$x_j^{(t)} \overset{\text{def}}{=} x_j^{(t-1)} - s\, \frac{g_j^{(t)}}{\sqrt{u_j^{(t)}} + \epsilon},$$

with $u_j^{(0)} \overset{\text{def}}{=} 0$.

Example 0.19 (Two coordinates on very different scales)**.** Suppose the two coordinates of the gradient start at $g_1^{(1)} = 100$ and $g_2^{(1)} = 0.01$, and both shrink by a factor of 0.5 at each update. Take $\rho = 0.9$, and look at the first 5 updates. Dividing by the root of the running average gives the factor $g_j^{(t)} / \sqrt{u_j^{(t)}}$ that multiplies s in the update of each coordinate:

R

```r
s_am <- c(100, 0.01)
rho_am <- 0.9
q_am <- 0.5
n_am <- 5
k_am <- seq_len(n_am) - 1
g_am <- outer(q_am^k_am, s_am)
u_am <- matrix(0, n_am, 2)
u_prev_am <- c(0, 0)
for (t_am in seq_len(n_am)) {
  u_prev_am <- rho_am * u_prev_am + (1 - rho_am) * g_am[t_am, ]^2
  u_am[t_am, ] <- u_prev_am
}
factor_am <- g_am / sqrt(u_am)
round(factor_am, 3)
#>       [,1]  [,2]
#> [1,] 3.162 3.162
#> [2,] 1.474 1.474
#> [3,] 0.755 0.755
#> [4,] 0.395 0.395
#> [5,] 0.208 0.208
```

Python

```python
import numpy as np

s_am, rho_am, q_am, n_am = [100, 0.01], 0.9, 0.5, 5
g_am = np.outer(q_am ** np.arange(n_am), s_am)
u_am = np.zeros(2)
factor_am = []
for t_am in range(n_am):
    u_am = rho_am * u_am + (1 - rho_am) * g_am[t_am] ** 2
    factor_am.append(g_am[t_am] / np.sqrt(u_am))
print(np.round(factor_am, 3));
#> [[3.162 3.162]
#>  [1.474 1.474]
#>  [0.755 0.755]
#>  [0.395 0.395]
#>  [0.208 0.208]]
```

Plain gradient descent would move the first coordinate 10^4 times as far as the second. The factors are the same for both coordinates, so the scale of a coordinate no longer

matters. They also shrink from 3.16 to 0.21, because the running average still holds the earlier, larger values. Dividing by the current gradient alone would give a factor of 1 at every update.

Remark (Early adaptive methods)

Remark 0.3 (Early adaptive methods). RMSProp is an early example of an adaptive method [22]. AdaGrad, an earlier method, replaces the exponentially weighted average by a plain sum of squared gradients, $u_j^{(t)} = u_j^{(t-1)} + \left(g_j^{(t)}\right)^2$.

Definition 0.16 (Adam)**.** Let f, $\underset{\sim}{x}^{(0)}$, s and ϵ be as in an adaptive method, and let $\beta_1, \beta_2 \in [0, 1)$, where β_2 plays the role of the decay factor ρ. Write $\underset{\sim}{g}^{(t)}$ for $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t-1)})$. The **Adam** method [23] is defined, for $t = 1, 2, ...,$

$$
\begin{aligned}
\underset{\sim}{\mu}^{(t)} &\stackrel{\text{def}}{=} \beta_1 \underset{\sim}{\mu}^{(t-1)} + (1 - \beta_1)\, \underset{\sim}{g}^{(t)} \\
u_j^{(t)} &\stackrel{\text{def}}{=} \beta_2\, u_j^{(t-1)} + (1 - \beta_2) \left(g_j^{(t)}\right)^2 \\
\underset{\sim}{\hat{\mu}}^{(t)} &\stackrel{\text{def}}{=} \frac{\underset{\sim}{\mu}^{(t)}}{1 - \beta_1^t} \\
\hat{u}_j^{(t)} &\stackrel{\text{def}}{=} \frac{u_j^{(t)}}{1 - \beta_2^t} \\
x_j^{(t)} &\stackrel{\text{def}}{=} x_j^{(t-1)} - s\, \frac{\hat{m}_j^{(t)}}{\sqrt{\hat{u}_j^{(t)}} + \epsilon},
\end{aligned}
$$

for each coordinate $j = 1, \ldots, p$, with $\underset{\sim}{\mu}^{(0)} \stackrel{\text{def}}{=} \underset{\sim}{0}$ and $u_j^{(0)} \stackrel{\text{def}}{=} 0$.

Example 0.20 (The first Adam step ignores the size of the gradient)**.** Let $f(x) = \frac{1}{2}x^2$, so $\frac{\partial}{\partial x} f(x) = x$, and take $s = 0.1$, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. At $t = 1$ the corrections for the zero start of $\underset{\sim}{\mu}$ and u_j cancel the factors $1 - \beta_1$ and $1 - \beta_2$, so $\hat{m}^{(1)} = g^{(1)}$ and $\hat{u}^{(1)} = \left(g^{(1)}\right)^2$. The first step is then $s\, g^{(1)} / (|g^{(1)}| + \epsilon)$, which is about s in size. Start at $x^{(0)} = 2$ and at $x^{(0)} = 2000$:

R

```r
s_ad <- 0.1
beta1_ad <- 0.9
beta2_ad <- 0.999
eps_ad <- 1e-8
x0_ad <- c(2, 2000)
g_ad <- x0_ad
m_ad <- (1 - beta1_ad) * g_ad
u_ad <- (1 - beta2_ad) * g_ad^2
m_hat_ad <- m_ad / (1 - beta1_ad)
u_hat_ad <- u_ad / (1 - beta2_ad)
step_ad <- s_ad * m_hat_ad / (sqrt(u_hat_ad) + eps_ad)
round(step_ad, 4)
#> [1] 0.1 0.1
```

Python

```python
import numpy as np

s_ad, beta1_ad, beta2_ad, eps_ad = 0.1, 0.9, 0.999, 1e-8
g_ad = np.array([2.0, 2000.0])
m_hat_ad = (1 - beta1_ad) * g_ad / (1 - beta1_ad)
u_hat_ad = (1 - beta2_ad) * g_ad ** 2 / (1 - beta2_ad)
print(np.round(s_ad * m_hat_ad / (np.sqrt(u_hat_ad) + eps_ad), 4));
#> [0.1 0.1]
```

Both first steps have size 0.1, though the gradients differ by a factor of 1000. Plain gradient descent would move $s\,g^{(1)}$, which is 0.2 and 200.

Definition 0.17 (Piecewise constant schedule)**.** Let $s^{(0)} > 0$ be the step length before update 1, let $\gamma \in (0,1)$ be a drop factor, and let $t_1 < t_2 < \cdots$ be update counts. The **piecewise constant schedule** is

$$s^{(t)} \stackrel{\text{def}}{=} s^{(0)}\,\gamma^{k(t)}, \qquad k(t) \stackrel{\text{def}}{=} \big|\{j : t_j \le t\}\big|.$$

Example 0.21 (A ten-fold drop twice)**.** Take $s^{(0)} = 0.1$ and $\gamma = 0.1$, with drops at $t_1 = 30$ and $t_2 = 60$. The step length just before and at each drop is:

R

```
s0_pc <- 0.1
decay_pc <- 0.1
drops_pc <- c(30, 60)
t_pc <- c(0, 29, 30, 59, 60, 90)
dropped_pc <- outer(drops_pc, t_pc, "<=")
k_pc <- colSums(dropped_pc)
step_pc <- s0_pc * decay_pc^k_pc
tibble::tibble(t = t_pc, step_length = step_pc)
#> # A tibble: 6 x 2
#>       t step_length
#>   <dbl>       <dbl>
#> 1     0       0.1
#> 2    29       0.1
#> 3    30       0.01
#> 4    59       0.01
#> 5    60       0.001
#> 6    90       0.001
```

Python

```
s0_pc, decay_pc, drops_pc = 0.1, 0.1, [30, 60]
for t_pc in [0, 29, 30, 59, 60, 90]:
    k_pc = sum(d_pc <= t_pc for d_pc in drops_pc)
    print(t_pc, round(s0_pc * decay_pc ** k_pc, 6));
#> 0 0.1
#> 29 0.1
#> 30 0.01
#> 59 0.01
#> 60 0.001
#> 90 0.001
```

The step length stays fixed between drops. It falls by the factor γ exactly at $t = 30$ and $t = 60$.

Definition 0.18 (Linear decay)**.** Let τ be a number of updates, and let $s^{(0)}, s^{(\tau)} > 0$ be the step lengths before update 1 and after update τ. The **linear decay** schedule is

$$s^{(t)} \stackrel{\text{def}}{=} (1 - \zeta)\, s^{(0)} + \zeta\, s^{(\tau)}, \qquad \zeta \stackrel{\text{def}}{=} \min\left(\frac{t}{\tau}, 1\right).$$

Example 0.22 (Linear decay between two step lengths)**.** Take $s^{(0)} = 0.1$, $s^{(\tau)} = 0.001$ and $\tau = 90$. The step length at $t = 0, 45, 90, 120$ is:

R

```
s0_ld <- 0.1
s_end_ld <- 0.001
tau_ld <- 90
t_ld <- c(0, 45, 90, 120)
frac_ld <- pmin(t_ld / tau_ld, 1)
step_ld <- (1 - frac_ld) * s0_ld + frac_ld * s_end_ld
tibble::tibble(t = t_ld, step_length = step_ld)
#> # A tibble: 4 x 2
#>       t step_length
#>   <dbl>       <dbl>
#> 1     0      0.1
#> 2    45      0.0505
#> 3    90      0.001
#> 4   120      0.001
```

Python

```
s0_ld, sT_ld, tau_ld = 0.1, 0.001, 90
for t_ld in [0, 45, 90, 120]:
    frac_ld = min(t_ld / tau_ld, 1)
    print(t_ld, round((1 - frac_ld) * s0_ld + frac_ld * sT_ld, 6));
#> 0 0.1
#> 45 0.0505
#> 90 0.001
#> 120 0.001
```

The step length falls by the same amount at each update. It then stays at 0.001 for $t \geq \tau$.

Definition 0.19 (Exponential decay)**.** Let $s^{(0)} > 0$ be the step length before update 1, and let $\gamma \in (0, 1)$ be a decay factor. The **exponential decay** schedule is

$$s^{(t)} \stackrel{\text{def}}{=} s^{(0)} \gamma^t.$$

Example 0.23 (The same factor over two run lengths)**.** Take $s^{(0)} = 0.1$ and $\gamma = 0.99$. The step length at $t = 1000$ and at $t = 100000$ is:

R

```
s0_ed <- 0.1
decay_ed <- 0.99
t_ed <- c(1000, 100000)
step_ed <- s0_ed * decay_ed^t_ed
tibble::tibble(t = t_ed, step_length = step_ed)
#> # A tibble: 2 x 2
#>        t step_length
#>    <dbl>       <dbl>
#> 1   1000  0.00000432
#> 2 100000  0
```

Python

```
s0_ed, decay_ed = 0.1, 0.99
for t_ed in [1000, 100000]:
    print(t_ed, s0_ed * decay_ed ** t_ed);
#> 1000 4.317124741065786e-06
#> 100000 0.0
```

At $t = 1000$ the step length is about 4.3×10^{-6}. At $t = 100000$ it is 0 in floating-point arithmetic. One factor gives two very different schedules.

Definition 0.20 (Cosine decay)**.** Let τ be a number of updates, and let $s^{(0)} > 0$ be the step length before update 1. The **cosine decay** schedule [24] is

$$s^{(t)} \stackrel{\text{def}}{=} \frac{s^{(0)}}{2}\left(1 + \cos\frac{\pi t}{\tau}\right), \qquad t = 0, 1, \ldots, \tau.$$

Example 0.24 (Cosine decay over a fixed run)**.** Take $s^{(0)} = 0.1$ and $\tau = 90$. The step length at $t = 0, 15, 45, 75, 90$ is:

R

```r
s0_cd <- 0.1
tau_cd <- 90
t_cd <- c(0, 15, 45, 75, 90)
angle_cd <- pi * t_cd / tau_cd
step_cd <- s0_cd / 2 * (1 + cos(angle_cd))
tibble::tibble(t = t_cd, step_length = step_cd)
#> # A tibble: 5 x 2
#>       t step_length
#>   <dbl>       <dbl>
#> 1     0     0.1
#> 2    15     0.0933
#> 3    45     0.05
#> 4    75     0.00670
#> 5    90     0
```

Python

```python
import math

s0_cd, tau_cd = 0.1, 90
for t_cd in [0, 15, 45, 75, 90]:
    print(t_cd, round(s0_cd / 2 * (1 + math.cos(math.pi * t_cd / tau_cd)), 6));
#> 0 0.1
#> 15 0.093301
#> 45 0.05
#> 75 0.006699
#> 90 0.0
```

The step length is 0.1 at the start, half of that at $t = 45$, and zero at $t = 90$. It changes slowly near both ends and fastest in the middle.

Remark (Restarting the cosine schedule)

Remark 0.4 (Restarting the cosine schedule). Restarting the schedule periodically is a common variant of cosine decay [24]: jump the step length back to $s^{(0)}$ and decay again. One motivation is that the jump can move the iterates out of one basin of attraction and into a better one.

Quasi-Newton methods

i Source and license

This section is adapted from the quasi-Newton lecture of Zhou [25], used under the MIT License. The license text is:

> Copyright (c) 2025 ucla-biostat-257
>
> Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
>
> The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
>
> THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

From the source, this section takes the order of ideas: the cost of Newton's method, the secant condition, and the BFGS and limited-memory BFGS updates. The source states the updates as solutions of nearest-matrix problems without proof; here the BFGS update is written out. The Davidon-Fletcher-Powell and rank-1 updates, the nearest-matrix derivation, the history, and the Julia code are not part of this section.

Definition 0.1 (Secant condition)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, and let $\underset{\sim}{x}, \underset{\sim}{x}^+ \in \mathbb{R}^p$. Write $\underset{\sim}{s} \overset{\text{def}}{=} \underset{\sim}{x}^+ - \underset{\sim}{x}$ for the step and $\underset{\sim}{y} \overset{\text{def}}{=} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^+) - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$ for the change in the gradient. A symmetric matrix $\mathbf{B}$ satisfies the **secant condition** [26, Ch. 6] for $(\underset{\sim}{s}, \underset{\sim}{y})$ when

$$\mathbf{B}\underset{\sim}{s} = \underset{\sim}{y}.$$

Example 0.1 (The Hessian of a quadratic function satisfies the secant condition)**.** Let $f(\underset{\sim}{x}) = \frac{1}{2}\underset{\sim}{x}^\top \mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b}\cdot\underset{\sim}{x}$ with $\mathbf{Q}$ symmetric. Then $\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) = \mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b}$, so for any $\underset{\sim}{x}$ and $\underset{\sim}{x}^+$

$$\begin{aligned}
\underset{\sim}{y} &= (\mathbf{Q}\underset{\sim}{x}^+ - \underset{\sim}{b}) - (\mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b}) && \text{(gradient at each point)}\\
&= \mathbf{Q}(\underset{\sim}{x}^+ - \underset{\sim}{x}) && \text{(collect terms)}\\
&= \mathbf{Q}\underset{\sim}{s}. && \text{(definition of } \underset{\sim}{s}\text{)}
\end{aligned}$$

So the Hessian $\mathbf{Q}$ of the quadratic function satisfies the secant condition for every step. For a general f, $\underset{\sim}{y} \approx \mathbf{H}_f \underset{\sim}{s}$ holds for a short step, with $\mathbf{H}_f$ the Hessian of f near the step, which is why a matrix that satisfies the secant condition is a reasonable stand-in for the Hessian.

Definition 0.2 (Quasi-Newton method)**.** Let $f : \mathbb{R}^p \to \mathbb{R}$ have first partial derivatives, let $\underset{\sim}{x}^{(0)}$ be a starting point, and let $\mathbf{H}^{(0)}$ be a positive definite matrix. A **quasi-Newton method** [26, Ch. 6] produces, for $t = 0, 1, 2, \ldots$,

$$\underset{\sim}{x}^{(t+1)} \overset{\text{def}}{=} \underset{\sim}{x}^{(t)} - \alpha^{(t)} \mathbf{H}^{(t)} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)}),$$

where $\alpha^{(t)} > 0$ is a step length chosen by a line search, and $\mathbf{H}^{(t+1)}$ is a symmetric positive definite matrix whose inverse satisfies the secant condition, that is, $\mathbf{H}^{(t+1)}\underset{\sim}{y}^{(t)} = \underset{\sim}{s}^{(t)}$, with $\underset{\sim}{s}^{(t)} = \underset{\sim}{x}^{(t+1)} - \underset{\sim}{x}^{(t)}$ and $\underset{\sim}{y}^{(t)} = \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t+1)}) - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$.

Example 0.2 (One quasi-Newton step on a quadratic function)**.** Take $f(\underset{\sim}{x}) = \frac{1}{2}\underset{\sim}{x}^\top \mathbf{Q}\underset{\sim}{x} - \underset{\sim}{b}\cdot\underset{\sim}{x}$ with $\mathbf{Q} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$ and $\underset{\sim}{b} = (1, 2)^\top$ (Example 0.1), $\underset{\sim}{x}^{(0)} = \underset{\sim}{0}$ and $\mathbf{H}^{(0)} = \mathbf{I}_2$. For a quadratic function, the step length that minimizes f along a direction $\underset{\sim}{d}$ is $\alpha = -\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) \cdot \underset{\sim}{d} / \underset{\sim}{d}^\top \mathbf{Q}\underset{\sim}{d}$.

```
q_entries <- c(4, 1, 1, 3)
Q <- matrix(q_entries, nrow = 2)
b <- c(1, 2)
x0 <- c(0, 0)
H0 <- diag(2)
q_x0 <- drop(Q %*% x0)
grad0 <- q_x0 - b
d <- -drop(H0 %*% grad0)
q_d <- drop(Q %*% d)
step <- -sum(grad0 * d) / sum(d * q_d)
x1 <- x0 + step * d
q_x1 <- drop(Q %*% x1)
grad1 <- q_x1 - b
f0 <- 0.5 * sum(x0 * q_x0) - sum(b * x0)
f1 <- 0.5 * sum(x1 * q_x1) - sum(b * x1)
s <- x1 - x0
y <- grad1 - grad0
```

The step length is 0.25, which moves the iterate to $\underset{\sim}{x}^{(1)} = (0.25, 0.5)$ and lowers f from 0 to -0.625. The difference of the gradients, $\underset{\sim}{y}^{(0)} = (1.5, 1.75)$, is $\mathbf{Q}\underset{\sim}{s}^{(0)}$, which is the secant condition for a quadratic function.

ℹ Remark (Why a quasi-Newton method is used)

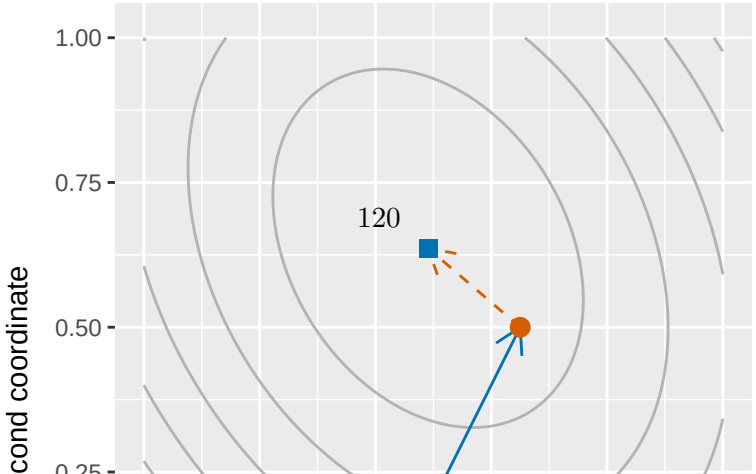
Remark 0.1 (Why a quasi-Newton method is used). Because $\mathbf{H}^{(t)}$ is positive definite, $-\mathbf{H}^{(t)}\frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$ is a descent direction, so, when the gradient is not $\underset{\sim}{0}$, a line search finds a step that decreases f. Newton's method needs the Hessian and the solution of a system of equations at every iteration, which costs about p^3 operations for a dense $p \times p$ Hessian, and requires the formulas for its entries. A quasi-Newton method needs only the gradient and an update of $\mathbf{H}^{(t)}$ costing about p^2 operations. Near a minimizer the method speeds up superlinearly: faster than gradient descent, slower than Newton's method [26, Ch. 6]. Replacing $\mathbf{H}^{(t)}$ by $\mathbf{I}_p$ in the step formula gives gradient descent.

Definition 0.3 (BFGS update (Broyden-Fletcher-Goldfarb-Shanno))**.** Let $\mathbf{H}$, an approximation of the inverse Hessian, be a $p \times p$ positive definite matrix, and let $\underset{\sim}{s}, \underset{\sim}{y} \in \mathbb{R}^p$ satisfy $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$. With $\rho \stackrel{\text{def}}{=} 1/\underset{\sim}{y} \cdot \underset{\sim}{s}$, the **BFGS update** [26, Ch. 6] of $\mathbf{H}$ is

$$\mathbf{H}^{+} \stackrel{\text{def}}{=} (\mathbf{I}_p - \rho \underset{\sim}{s}\underset{\sim}{y}^{\top})\mathbf{H}(\mathbf{I}_p - \rho \underset{\sim}{y}\underset{\sim}{s}^{\top}) + \rho \underset{\sim}{s}\underset{\sim}{s}^{\top}.$$

R

```
qn_x_qn <- seq(-0.4, 0.6, length.out = 101)
qn_y_qn <- seq(-0.2, 1, length.out = 101)
qn_grid_qn <- tidyr::expand_grid(x = qn_x_qn, y = qn_y_qn)
qn_gx_qn <- qn_grid_qn$x
qn_gy_qn <- qn_grid_qn$y
qn_quad_qn <- 0.5 * (4 * qn_gx_qn^2 + 2 * qn_gx_qn * qn_gy_qn + 3 * qn_gy_qn^2)
qn_lin_qn <- qn_gx_qn + 2 * qn_gy_qn
qn_grid_qn$z <- qn_quad_qn - qn_lin_qn
qn_star_qn <- c(1, 7) / 11
qn_start_qn <- tibble::tibble(x = 0, y = 0)
qn_next_qn <- tibble::tibble(x = 0.25, y = 0.5)
qn_min_qn <- tibble::tibble(x = qn_star_qn[1], y = qn_star_qn[2])
qn_step_qn <- tibble::tibble(x = 0, y = 0, xend = 0.25, yend = 0.5)
qn_rest_qn <- tibble::tibble(
  x = 0.25, y = 0.5, xend = qn_star_qn[1], yend = qn_star_qn[2]
)
qn_len_qn <- grid::unit(0.15, "inches")
qn_arrow_qn <- grid::arrow(length = qn_len_qn)
qn_aes_qn <- ggplot2::aes(x, y)
qn_seg_qn <- ggplot2::aes(xend = xend, yend = yend)
ggplot2::ggplot(qn_grid_qn, qn_aes_qn) +
  ggplot2::geom_contour(ggplot2::aes(z = z), colour = "grey70", bins = 12) +
  ggplot2::geom_segment(
    data = qn_step_qn, mapping = qn_seg_qn, colour = "#0072B2",
    arrow = qn_arrow_qn
  ) +
  ggplot2::geom_segment(
    data = qn_rest_qn, mapping = qn_seg_qn, colour = "#D55E00",
    linetype = "dashed", arrow = qn_arrow_qn
  ) +
  ggplot2::geom_point(data = qn_start_qn, colour = "#0072B2", size = 3) +
  ggplot2::geom_point(data = qn_next_qn, colour = "#D55E00", size = 3) +
  ggplot2::geom_point(
    data = qn_min_qn, colour = "#0072B2", size = 3, shape = 15
  ) +
  ggplot2::coord_equal() +
  ggplot2::labs(x = "first coordinate", y = "second coordinate")
```



R

(This 3D surface is interactive; see the HTML or slide version of this page to view it.)

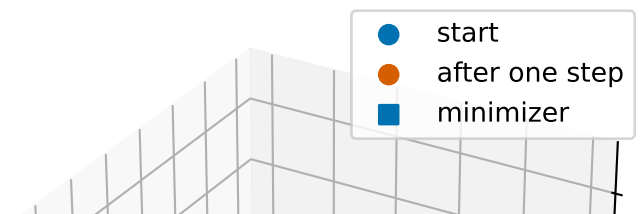
Python

```
import matplotlib.pyplot as plt
import numpy as np

qn_x_3d = np.linspace(-0.4, 0.6, 101)
qn_y_3d = np.linspace(-0.2, 1, 101)
qn_xx_3d, qn_yy_3d = np.meshgrid(qn_x_3d, qn_y_3d)


def qn_f_3d(x, y):
    return 0.5 * (4 * x**2 + 2 * x * y + 3 * y**2) - x - 2 * y


qn_pts_x_3d = np.array([0, 0.25, 1 / 11])
qn_pts_y_3d = np.array([0, 0.5, 7 / 11])
qn_pts_z_3d = qn_f_3d(qn_pts_x_3d, qn_pts_y_3d)
fig = plt.figure(figsize=(6, 4.5))
ax = fig.add_subplot(projection="3d")
ax.plot_surface(qn_xx_3d, qn_yy_3d, qn_f_3d(qn_xx_3d, qn_yy_3d),
                color="grey", alpha=0.4)
#> <mpl_toolkits.mplot3d.art3d.Poly3DCollection object at 0x7fa982bfb9d0>
ax.scatter(qn_pts_x_3d[0], qn_pts_y_3d[0], qn_pts_z_3d[0], marker="o",
           color="#0072B2", s=50, label="start")
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa982bfb390>
ax.scatter(qn_pts_x_3d[1], qn_pts_y_3d[1], qn_pts_z_3d[1], marker="o",
           color="#D55E00", s=50, label="after one step")
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa982bfb890>
ax.scatter(qn_pts_x_3d[2], qn_pts_y_3d[2], qn_pts_z_3d[2], marker="s",
           color="#0072B2", s=50, label="minimizer")
#> <mpl_toolkits.mplot3d.art3d.Path3DCollection object at 0x7fa982bfb750>
ax.set_xlabel("first coordinate")
#> Text(0.5, 0, 'first coordinate')
ax.set_ylabel("second coordinate")
#> Text(0.5, 0.5, 'second coordinate')
ax.set_zlabel("f(x)")
#> Text(0.5, 0, 'f(x)')
ax.legend(loc="upper right")
#> <matplotlib.legend.Legend object at 0x7fa982bfb610>
plt.show();
```

Example 0.3 (A BFGS update from the quasi-Newton step)**.** Continue Example 0.2 with $\mathbf{H} = \mathbf{I}_2$, $\underset{\sim}{s} = \underset{\sim}{s}^{(0)}$, and $\underset{\sim}{y} = \underset{\sim}{y}^{(0)}$.

```
rho <- 1 / sum(y * s)
left <- diag(2) - rho * s %*% t(y)
H1 <- left %*% H0 %*% t(left) + rho * s %*% t(s)
H1
#>         [,1]   [,2]
#> [1,]  0.6625 -0.425
#> [2,] -0.4250  0.650
drop(H1 %*% y)
#> [1] 0.25 0.50
```

Here $\underset{\sim}{y} \cdot \underset{\sim}{s} = 1.25 > 0$, so $\rho = 0.8$. The updated matrix satisfies $\mathbf{H}^{+}\underset{\sim}{y} = (0.25, 0.5)$, which is $\underset{\sim}{s}$ = (0.25, 0.5), and its smallest eigenvalue is 0.231204, so $\mathbf{H}^{+}$ is positive definite.

Remark (The curvature condition $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$)

Remark 0.2 (The curvature condition $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$). If the strict inequality of Theorem 0.4 holds for distinct points, then $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$ for $\underset{\sim}{s} = \underset{\sim}{x}^{+} - \underset{\sim}{x} \neq \underset{\sim}{0}$ and $\underset{\sim}{y} = \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{+}) - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x})$. Add the versions at $(\underset{\sim}{x}^{+}, \underset{\sim}{x})$ and at $(\underset{\sim}{x}, \underset{\sim}{x}^{+})$:

$$f(\underset{\sim}{x}^{+}) + f(\underset{\sim}{x}) > f(\underset{\sim}{x}) + f(\underset{\sim}{x}^{+}) + \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}) \cdot \underset{\sim}{s} - \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{+}) \cdot \underset{\sim}{s},$$

so $0 > -\underset{\sim}{y} \cdot \underset{\sim}{s}$. For a general f, a line search that enforces a curvature condition on the step length (the Wolfe conditions [26, Ch. 3]) guarantees $\underset{\sim}{y} \cdot \underset{\sim}{s} > 0$. Such a step can be found along a descent direction when f cannot keep decreasing forever.

Definition 0.4 (Limited-memory BFGS (L-BFGS))**. Limited-memory BFGS** (**L-BFGS**) [26, Ch. 7] is the quasi-Newton method that never forms $\mathbf{H}^{(t)}$. It stores only the m most recent pairs $(\underset{\sim}{s}^{(j)}, \underset{\sim}{y}^{(j)})$, with m small, such as 5 or 20, and obtains $\mathbf{H}^{(t)} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(t)})$ implicitly, through a few vector operations, as if the BFGS update for those m pairs were applied to a simple starting matrix, such as a multiple of $\mathbf{I}_p$.

Example 0.4 (L-BFGS applies the update without forming the matrix)**.**
Continue Example 0.3 with $m = 1$ stored pair $(\underset{\sim}{s}^{(0)}, \underset{\sim}{y}^{(0)})$ and starting matrix $\mathbf{I}_2$. Below, $\mathbf{H}^{+} \frac{\partial}{\partial \underset{\sim}{x}} f(\underset{\sim}{x}^{(1)})$ is formed from vectors alone, starting from `H0`, then compared with the matrix product.

```
g1 <- grad1
alpha <- rho * sum(s * g1)
q <- g1 - alpha * y
r <- drop(H0 %*% q)
beta <- rho * sum(y * r)
r <- r + s * (alpha - beta)
h1_g1 <- H1 %*% g1
from_matrix <- drop(h1_g1)
tibble::tibble(from_vectors = r, from_matrix = from_matrix)
#> # A tibble: 2 x 2
#>   from_vectors from_matrix
#>          <dbl>       <dbl>
#> 1        0.438       0.438
#> 2       -0.375      -0.375
```

The two columns agree to within 5.6e-17, although the first never forms the matrix. BFGS stores a dense $p \times p$ matrix and spends about p^2 operations per iteration; L-BFGS stores $2m$ vectors with p entries and spends about mp operations per iteration, so it remains usable when p is in the millions.

Further reading

- Boyd and Vandenberghe [27] is the standard reference on convex optimization, covering convex sets, convex functions, convex optimization problems, duality, unconstrained minimization, and interior-point methods.
- Nocedal and Wright [26] is a comprehensive reference on continuous numerical optimization, covering line search methods, trust-region methods, conjugate gradient methods, quasi-Newton methods, and nonlinear equations.
- M/CS 335, Linear Optimization at Western Washington University covers optimizing linear functions under linear constraints, including linear programming and duality.
- M/CS 375, Numerical Computation at Western Washington University covers computer arithmetic, solving nonlinear equations, optimization in a single variable, matrix factorization, and iterative methods for matrices.
- Hua Zhou's course page for "UCLA Biostat 257 - Statistical Computing" (2025 Spring) has lectures on optimization, from root finding to constrained problems. Its source is on GitHub under the MIT License.
- Hua Zhou's lecture notes for "UCLA Biostat 216 - Mathematical Methods for Biostatistics" (2024 Fall)
- Neural Networks is a YouTube playlist by Grant Sanderson (3Blue1Brown); its chapters on gradient descent and backpropagation calculus visually illustrate how gradients of

multivariate cost functions are computed and used for optimization.

References

[1] Wikipedia contributors, "Stationary point — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Stationary_point

[2] S. J. Miller, *The probability lifesaver: Calculus review problems*. 2016. Available: https://web.williams.edu/Mathematics/sjmiller/public_html/probabilitylifesaver/index.htm#:~:text=http%3A//web.williams.edu/Mathematics/sjmiller/public_html/probabilitylifesaver/supplementalchap_calcreview.pdf

[3] H. Zhou, "Optimization and multivariate calculus." Lecture notes for Biostat 216, Mathematical Methods for Biostatistics, University of California, Los Angeles, 2024. Accessed: Oct. 05, 2026. [Online]. Available: https://ucla-biostat-216.github.io/2024fall/slides/13-optim/13-optim.html

[4] Wikipedia contributors, "Fermat's theorem (stationary points) — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Fermat%27s_theorem_(stationary_points)

[5] Wikipedia contributors, "Taylor's theorem — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Taylor%27s_theorem

[6] W. Rudin, *Principles of mathematical analysis*, 3rd ed. in International series in pure and applied mathematics. New York: McGraw-Hill, 1976.

[7] Wikipedia contributors, "Saddle point — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Saddle_point

[8] Wikipedia contributors, "Convex set — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Convex_set

[9] Wikipedia contributors, "Epigraph (mathematics) — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Epigraph_(mathematics)

[10] Wikipedia contributors, "Tangent space — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Tangent_space

[11] Wikipedia contributors, "Feasible region — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Feasible_region

[12] Wikipedia contributors, "Mathematical optimization — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Mathematical_optimization

[13] Wikipedia contributors, "Lagrange multiplier — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Lagrange_multiplier

[14] Wikipedia contributors, "Iterative method — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Iterative_method

[15] Wikipedia contributors, "Newton's method in optimization — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Newton%27s_method_in_optimization

[16] Wikipedia contributors, "Rate of convergence — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Rate_of_convergence

[17] Wikipedia contributors, "Gradient descent — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Gradient_descent

[18] Wikipedia contributors, "Line search — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Line_search

[19] Wikipedia contributors, "Rosenbrock function — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Rosenbrock_function

[20] Wikipedia contributors, "Attractor — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Attractor

[21] Wikipedia contributors, "Backtracking line search — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Backtracking_line_search

[22] Wikipedia contributors, "Stochastic gradient descent — Wikipedia, the free encyclopedia." 2026. Available: https://en.wikipedia.org/wiki/Stochastic_gradient_descent

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd international conference on learning representations*, 2015. doi: 10.48550/arXiv.1412.6980.

[24] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *International conference on learning representations*, 2017. doi: 10.48550/arXiv.1608.03983.

[25] H. Zhou, "Quasi-Newton methods." Lecture notes for Biostat 257, Statistical Computing, University of California, Los Angeles, 2025. Accessed: Oct. 26, 2026. [Online]. Available: https://ucla-biostat-257.github.io/2025spring/slides/24-quasinewton/quasinewton.html

[26] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. in Springer series in operations research and financial engineering. Springer, 2006. doi: 10.1007/978-0-387-40065-5.

[27] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge University Press, 2004. doi: 10.1017/CBO9780511804441.